



نام دانشجو :

رضا ندیمی

شماره دانشجویی:

832204022

دانشکده سامانه های هوشمند - مکترونیک

تمرینات مکترونیک 2 سری چهارم

تاریخ:

1405/03/03

استاد مربوطه:

دکتر تارویردی زاده

فهرست

- فضای توسعه ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی..... 3
- 1_Polling. از طریق ترمینال نرم افزار Realterm اعداد دلخواهی (در بازه 0 تا 240) را به میکروکنترلر ارسال کنید؛
عدد دریافت شده در میکروکنترلر را با عدد ۱۰ جمع کرده و دوباره به ترمینال نرم افزار Realterm، بر گردانید..... 4
- 1.1 روش Poling 4
- 1_IT. از طریق ترمینال نرم افزار Realterm اعداد دلخواهی (در بازه 0 تا 240) را به میکروکنترلر ارسال کنید؛ عدد
دریافت شده در میکروکنترلر را با عدد ۱۰ جمع کرده و دوباره به ترمینال نرم افزار Realterm، بر گردانید..... 10
- 1.2 روش interrupt 10
- 1_DMA. از طریق ترمینال نرم افزار Realterm اعداد دلخواهی (در بازه 0 تا 240) را به میکروکنترلر ارسال کنید؛ عدد
دریافت شده در میکروکنترلر را با عدد ۱۰ جمع کرده و دوباره به ترمینال نرم افزار Realterm، بر گردانید..... 17
- 1.3 روش DMA 17
- 2_Polling. تمرین فوق را به جای ترمینال نرم افزار Realterm، با نرم افزار Matlab انجام دهید. 24
- 2IT. تمرین فوق را به جای ترمینال نرم افزار Realterm، با نرم افزار Matlab انجام دهید. 28
- 2_DMA. تمرین فوق را به جای ترمینال نرم افزار Realterm، با نرم افزار Matlab انجام دهید. 29
- 3_Polling. شمارندهای در میکرو کنترلر ایجاد کنید که اعداد از ۰ تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵
رسید از ۲۵۵ تا ۰ را مجدد بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال
دندانه ارهای). این اعداد را در همان حین تولید در میکروکنترلر، به کامپیوتر ارسال کرده و در نرمافزار Matlab، دریافت
نموده و در یک آرایه بریزید و بعد از چند سری دریافت در نرم افزار Matlab، آن را رسم نمایید. 31
- 3.1 روش Poling 31
- 3_IT. شمارندهای در میکرو کنترلر ایجاد کنید که اعداد از ۰ تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از
۲۵۵ تا ۰ را مجدد بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندانه
ارهای). این اعداد را در همان حین تولید در میکروکنترلر، به کامپیوتر ارسال کرده و در نرمافزار Matlab، دریافت نموده و
در یک آرایه بریزید و بعد از چند سری دریافت در نرم افزار Matlab، آن را رسم نمایید. 41
- 3.2 روش Intrupt 41
- 3_DMA. شمارندهای در میکرو کنترلر ایجاد کنید که اعداد از ۰ تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید
از ۲۵۵ تا ۰ را مجدد بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندانه
ارهای). این اعداد را در همان حین تولید در میکروکنترلر، به کامپیوتر ارسال کرده و در نرمافزار Matlab، دریافت نموده و
در یک آرایه بریزید و بعد از چند سری دریافت در نرم افزار Matlab، آن را رسم نمایید. 47
- 3.3 روش DMA 47
- 4_Polling. شمارنده ای در کامپیوتر در نرم افزار Matlab ایجاد کنید که اعداد از 0 تا ۲۵۵ را بصورت متوالی تولید کند
وقتی به ۲۵۵ رسید از ۲۵۵ تا ۰ را مجدد بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد

(تولید سیگنال دندان‌اره‌ای). این اعداد را در همان حین تولید در نرم‌افزار Matlab، به میکروکنترلر ارسال کرده و از طریق میکروکنترلر روی LCD، نمایش دهید. (سرعت ارسال و نمایش اعداد روی LCD باید طوری باشد که با چشم قابل تشخیص باشد). دقت: دریافت داده در میکروکنترلر را با ۳ روش، Polling، وقفه و DMA، انجام دهید. 54	
4.1 روش Poling 54	
4_IT. شماره‌دهی در کامپیوتر در نرم افزار Matlab ایجاد کنید که اعداد از 0 تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از ۰ تا ۲۵۵ را مجدداً بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندان‌اره‌ای). این اعداد را در همان حین تولید در نرم‌افزار Matlab، به میکروکنترلر ارسال کرده و از طریق میکروکنترلر روی LCD، نمایش دهید. (سرعت ارسال و نمایش اعداد روی LCD باید طوری باشد که با چشم قابل تشخیص باشد). دقت: دریافت داده در میکروکنترلر را با ۳ روش، Polling، وقفه و DMA، انجام دهید. 61	
4.2 روش IT 61	
4_DMA. شماره‌دهی در کامپیوتر در نرم افزار Matlab ایجاد کنید که اعداد از 0 تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از ۰ تا ۲۵۵ را مجدداً بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندان‌اره‌ای). این اعداد را در همان حین تولید در نرم‌افزار Matlab، به میکروکنترلر ارسال کرده و از طریق میکروکنترلر روی LCD، نمایش دهید. (سرعت ارسال و نمایش اعداد روی LCD باید طوری باشد که با چشم قابل تشخیص باشد). دقت: دریافت داده در میکروکنترلر را با ۳ روش، Polling، وقفه و DMA، انجام دهید. 69	
4.3 روش DMA 69	
5_Polling. اسم خودتان را به تعداد ۱۰ بار، از میکروکنترلر به نرم‌افزار Matlab، ارسال نمایید آن را در نرم افزار Matlab، دریافت و نمایش دهید. 77	
5.1 روش Poling 77	
5_IT. اسم خودتان را به تعداد ۱۰ بار، از میکروکنترلر به نرم‌افزار Matlab، ارسال نمایید آن را در نرم‌افزار Matlab، دریافت و نمایش دهید. 83	
5.2 روش IT 83	
5_DMA. اسم خودتان را به تعداد ۱۰ بار، از میکروکنترلر به نرم‌افزار Matlab، ارسال نمایید آن را در نرم‌افزار Matlab، دریافت و نمایش دهید. 88	
5.3 روش DMA 88	
6_Polling. اسم خودتان را به تعداد ۱۰ بار، از نرم‌افزار Matlab به میکروکنترلر، ارسال نمایید آن را در میکروکنترلر، دریافت و روی LCD نمایش دهید. 95	
6.1 روش Poling 95	
6_IT. اسم خودتان را به تعداد ۱۰ بار، از نرم‌افزار Matlab به میکروکنترلر، ارسال نمایید آن را در میکروکنترلر، دریافت و روی LCD نمایش دهید. 103	
6.2 روش IT 103	

6_DMA	اسم خودتان را به تعداد ۱۰ بار، از نرمافزار Matlab به میکروکنترلر، ارسال نمایید آن را در میکروکنترلر، دریافت و روی LCD نمایش دهید.....	111
6.3	روشن DMA.....	111

فضای توسعه ی مورد استفاده با توجه به عدم دسترسی به مخازن نرم افزار بین المللی

1. برای STM32 CubeMX پک F1 نسخه ی V1.8.0
2. برای Keil پک Keil.STM32F1xx_DFP.2.4.0
3. برای Keil پک ARM.CMSIS.4.5.0 متعلق به سال 2015 (آخرین نسخه ای که در نت ملی توانستم پیدا کنم. الان سال 2026 می باشد یعنی چیزی حدود 11 سال پیش)
4. برای ارتباط سریال با میکرو کنترلر از virtual serial driver pro استفاده شده است که از لینک زیر دانلود شده است.
<https://p30download.ir/fa/entry/65323/virtual-serial-port-driver>

1_Polling. از طریق ترمینال نرم افزار Realterm اعداد دلخواهی (در بازه 0 تا 240) را به میکروکنترلر ارسال کنید؛ عدد دریافت شده در میکروکنترلر را با عدد ۱۰ جمع کرده و دوباره به ترمینال نرم افزار Realterm، بر گردانید.

1.1 روش Poling

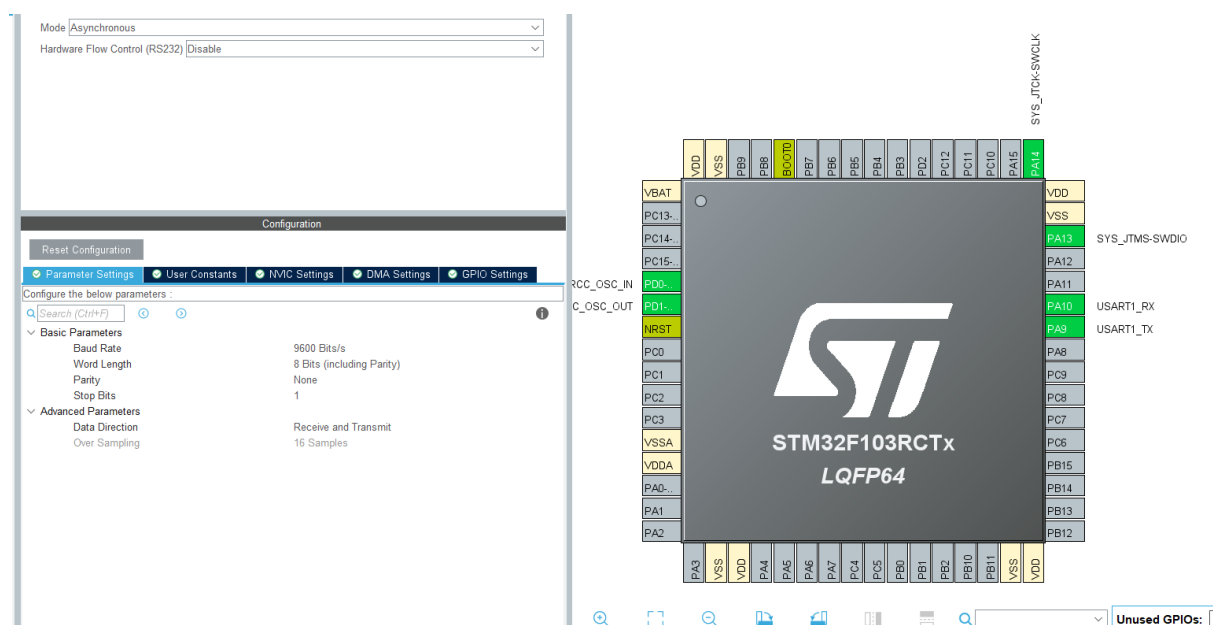
در این روش CPU به طور مداوم وضعیت رجیستر UART را بررسی می کند.

(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



(ii) بخش Keil

در روش پولینگ درداخل حلقه while با کمک دستور HAL_UART_Recevie در بازه های زمانی کوتاه بررسی میکنیم که ایا داده جدیدی از طریق UART دریافت شده است یا خیر.

در داخل main:

huart1 را معرفی میکنیم و همچنین دو متغیر برای دریافت و ارسال اطلاعات به صورت uint_8 معرفی می کنیم

```
/* USER CODE BEGIN 2 */
uint8_t polling_received_data;
uint8_t polling_to_send_data;
/* USER CODE END 2 */

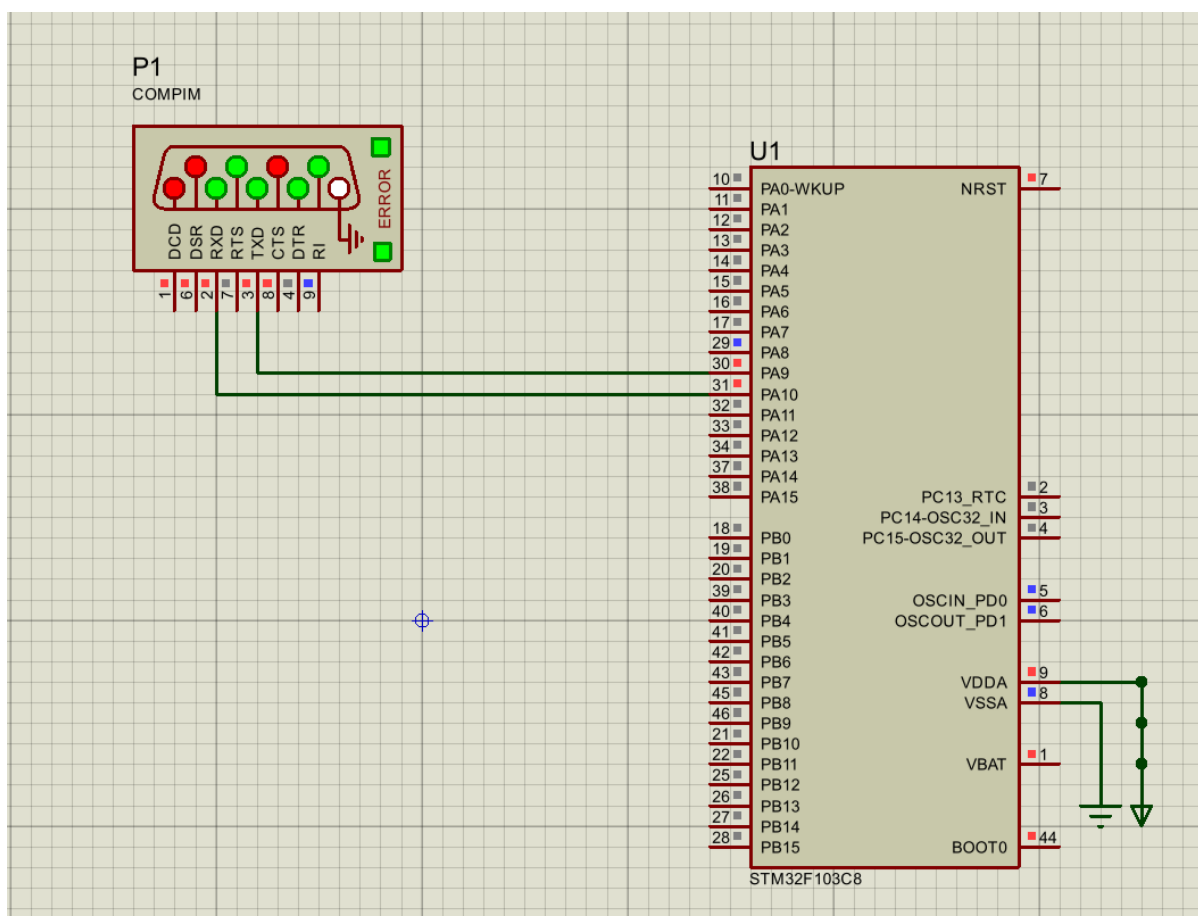
/* Infinite loop */
```

در داخل while:

بررسی میکنیم که آیا اطلاعات به درستی دریافت شده است یا خیر سپس بررسی میکنیم که عدد دریافت شده در رنج 0 تا 240 باشد که در این صورت آن عدد را با 10 جمع کرده و ارسال میکنیم در غیر این صورت عدد 255 را به صورت دیفالت ارسال می کنیم.

```
/* USER CODE BEGIN WHILE */
while(1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if(HAL_UART_Receive(&huart1, &polling_received_data, 1, 100) == HAL_OK){
        // Check if received value is within 0-240
        if(polling_received_data <= 240){
            polling_to_send_data = polling_received_data + 10;
        } else {
            polling_to_send_data = 255;
        }
        // Send the single byte back
        HAL_UART_Transmit(&huart1, &polling_to_send_data, 1, HAL_MAX_DELAY);
    }
}
/* USER CODE END 3 */
```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

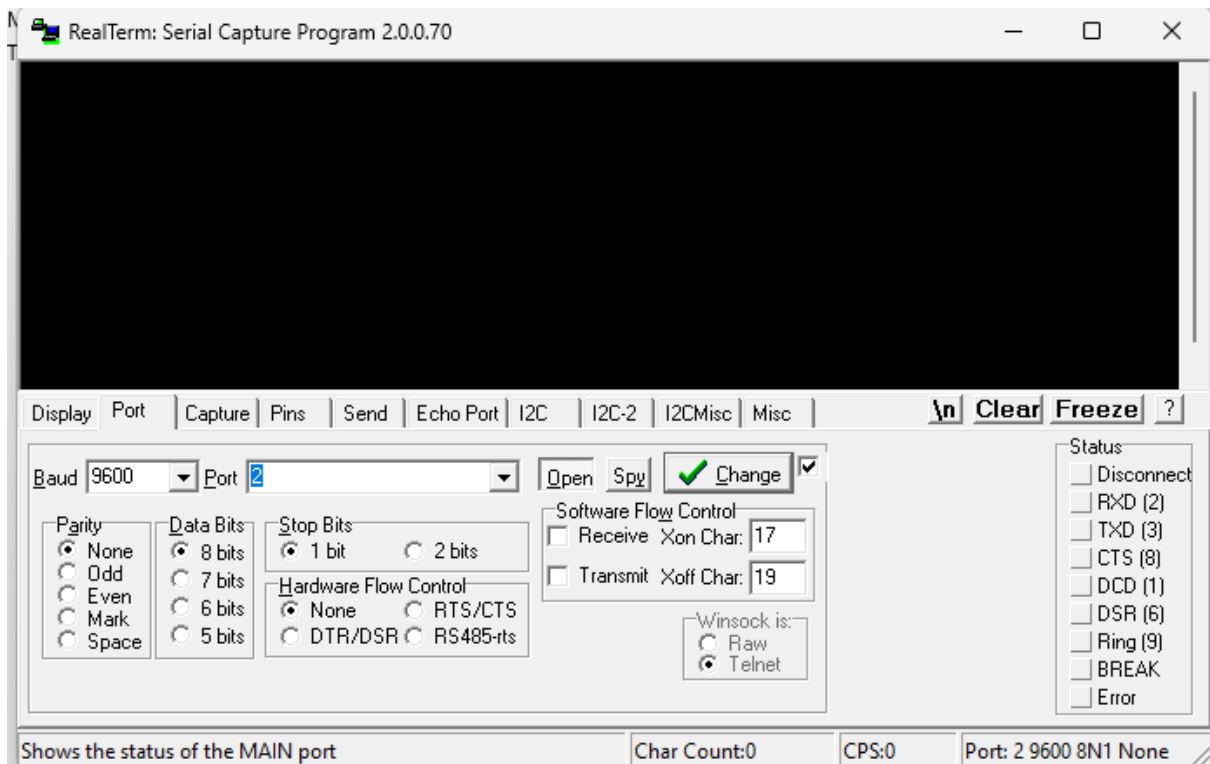
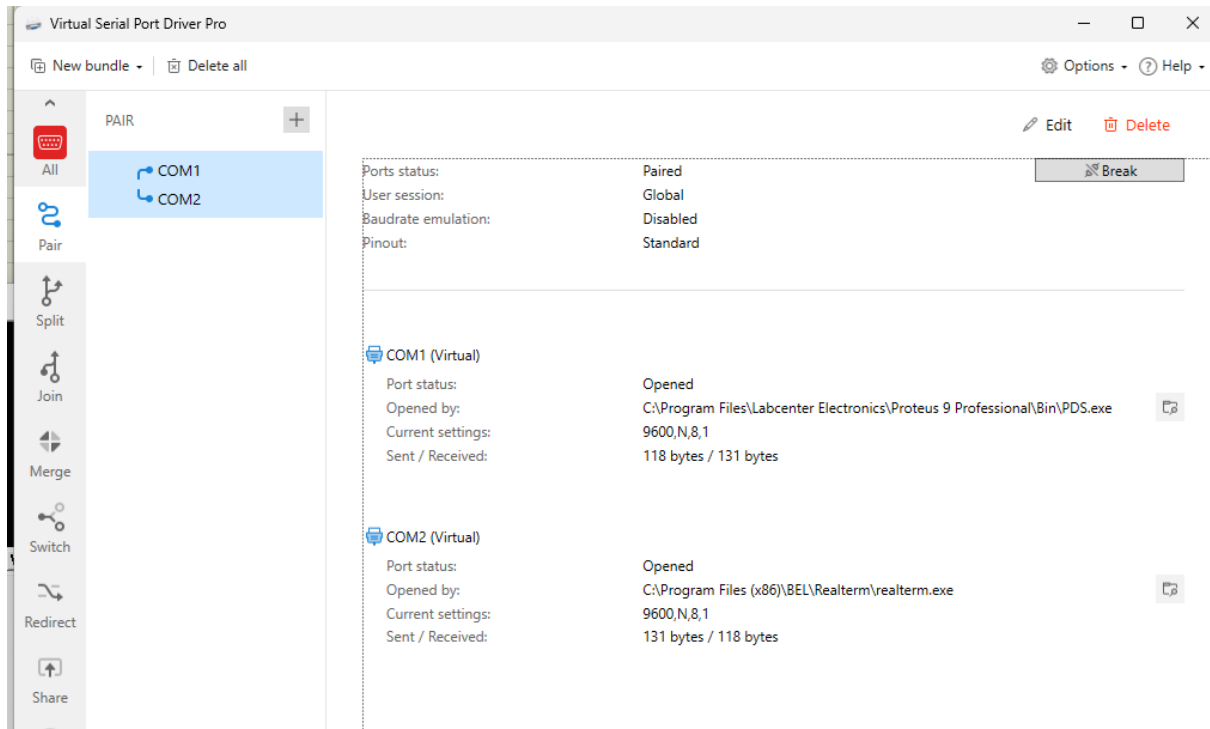
COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.



4. بخش Virtual serial port pro و RealTerm

یک کانیکشن pair می سازیم



Edit Component ? X

Part Reference:	P1	Hidden: ☐
Part Value:	COMPIM	Hidden: ☐
Element:	New	
VSM Model:	COMPIM.DLL	Hide All v
Physical port:	COM1	Hide All v
Physical Baud Rate:	9600	Hide All v
Physical Data Bits:	8	Hide All v
Physical Parity:	NONE	Hide All v
Virtual Baud Rate:	9600	Hide All v
Virtual Data Bits:	8	Hide All v
Virtual Parity:	NONE	Hide All v
Advanced Properties:		
Physical Stop Bits	1	Hide All v
Other Properties		
☐ Exclude from Simulation	☐ Attach hierarchy module	
☐ Exclude from PCB Layout	☐ Hide common pins	
☐ Exclude from Current Variant	☐ Edit all properties as text	

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

1.IT. از طریق ترمینال نرم افزار Realterm اعداد دلخواهی (در بازه 0 تا 240) را به میکروکنترلر ارسال کنید؛ عدد دریافت شده در میکروکنترلر را با عدد ۱۰ جمع کرده و دوباره به ترمینال نرم افزار Realterm، برگردانید.

1.2 روش interrupt

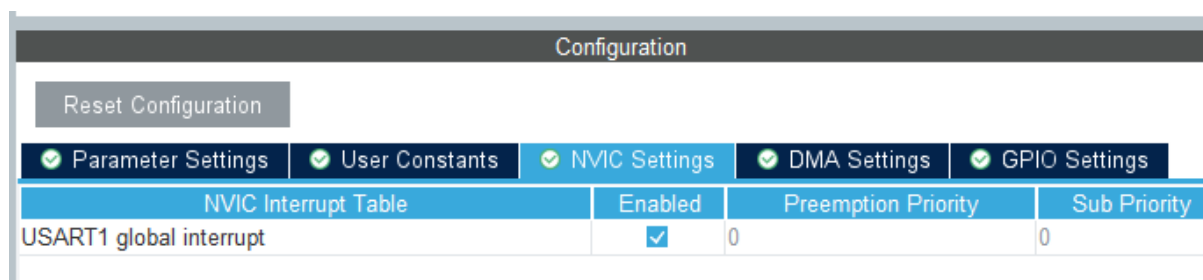
در روش اینترپت، میکروکنترلر بی‌آنکه مدام چک کند (پولینگ)، فقط وقتی داده می‌رسد وقفه ایجاد می‌شود و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

(بخش STM32CubeMX

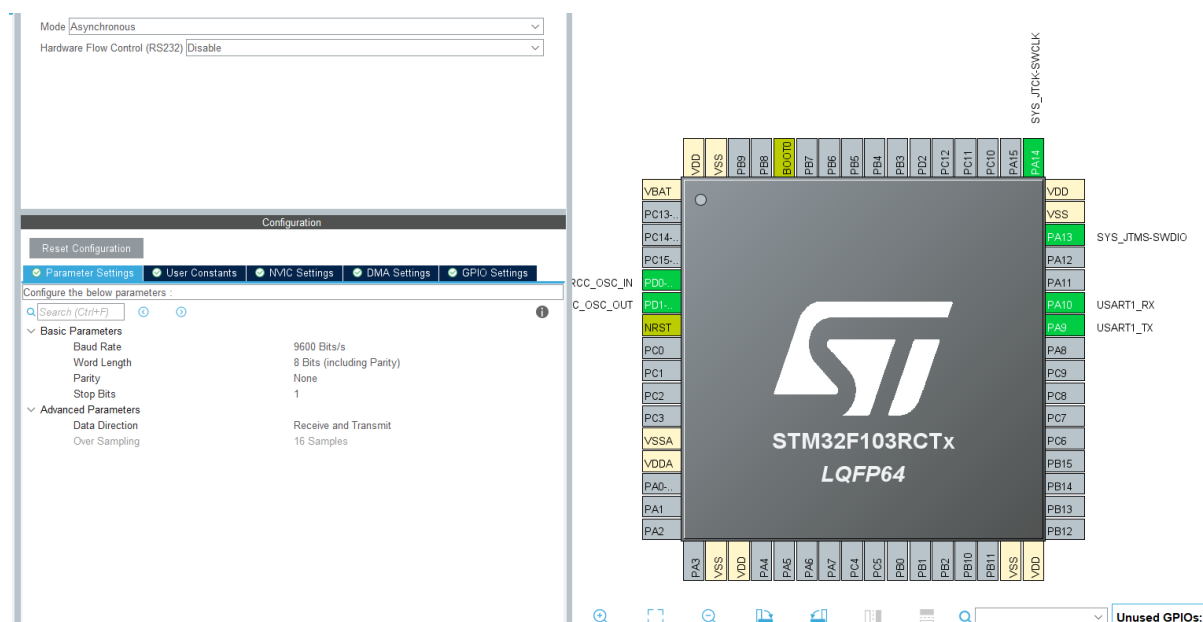
میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می‌کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم

تیک مربوط به وقفه را می‌زنیم



پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



(iii) بخش Keil

در روش وقفه‌ای، CPU هیچ‌گاه مشغول چک کردن مداوم پورت نمی‌شود. به جای آن، سخت‌افزار UART تنظیم می‌شود که به محض رسیدن یک بایت، یک وقفه به CPU بزند. CPU در بقیه زمان‌ها آزاد است و می‌تواند کارهای دیگر را انجام دهد. این روش نسبت به پولینگ (چک کردن حلقه‌ای) بسیار کارآمدتر است و مصرف پردازنده را به شدت کاهش می‌دهد.

معرفی متغیرها

```
/* USER CODE BEGIN 0 */
uint8_t received_byte;
uint8_t result_to_send;

/* USER CODE END 0 */
```

در داخل main:

راه‌اندازی دریافت وقفه‌ای به CPU می‌گوییم بدون اشغال شدن منتظر یک بایت باشد

اگر خطا رخ دهد، تابع Error_Handler صدا زده می‌شود

```
/* USER CODE BEGIN 2 */
if (HAL_UART_Receive_IT(&huart1, &received_byte, 1) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE END 2 */
```

در داخل قسمت 4 user code در پایین فایل main.c:

این تابع هر زمان که یک بایت داده از طریق پورت سریال USART1 به طور کامل دریافت شود، به صورت خودکار توسط سیستم وقفه (اینترپت) فراخوانی می‌شود.

مراحل انجام کار در این تابع:

بررسی منبع وقفه:

خط `if (huart->Instance == USART1)` بررسی می‌کند که آیا وقفه رسیده از همان پورت سریالی است که ما می‌خواهیم (USART1). این کار برای مواقعی است که چند پورت سریال داریم و فقط می‌خواهیم به یکی از آنها پاسخ دهیم.

پردازش داده دریافتی:

متغیر `received_byte` که قبلاً در برنامه اصلی تعریف شده، مقدار بایت دریافتی را دارد. اگر این مقدار بین 0 تا 240 باشد، عدد 10 به آن اضافه می‌شود و در متغیر `result_to_send` ذخیره می‌گردد. در غیر این صورت (یعنی اگر عدد بزرگتر از 240 باشد) مقدار 255 به عنوان علامت خطا در نظر گرفته می‌شود.

ارسال نتیجه به کامپیوتر:

با دستور `HAL_UART_Transmit`، مقدار محاسبه‌شده (نتیجه) از طریق همان پورت سریال به سمت کامپیوتر ارسال می‌شود.

آماده شدن برای دریافت بعدی:

در انتها، دستور `HAL_UART_Receive_IT` دوباره وقفه دریافت را فعال می‌کند تا میکروکنترلر منتظر بایت بعدی بماند. بدون این خط، فقط یک بار دریافت انجام می‌شد و وقفه بعدی هرگز رخ نمی‌داد.

مزیت این روش:

CPU در بقیه زمان‌ها (زمانی که داده‌ای نمی‌رسد) کاملاً آزاد است و می‌تواند کارهای دیگر مانند کنترل LEDها، خواندن سنسورها و ... را انجام دهد. این روش برخلاف پولینگ (حلقه‌های `while` که مدام چک می‌کنند) مصرف پردازنده را بسیار پایین می‌آورد

```
/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (received_byte <= 240)
        {
            result_to_send = received_byte + 10;
        }
        else
        {
            result_to_send = 255;
        }

        HAL_UART_Transmit(&huart1, &result_to_send, 1, HAL_MAX_DELAY);

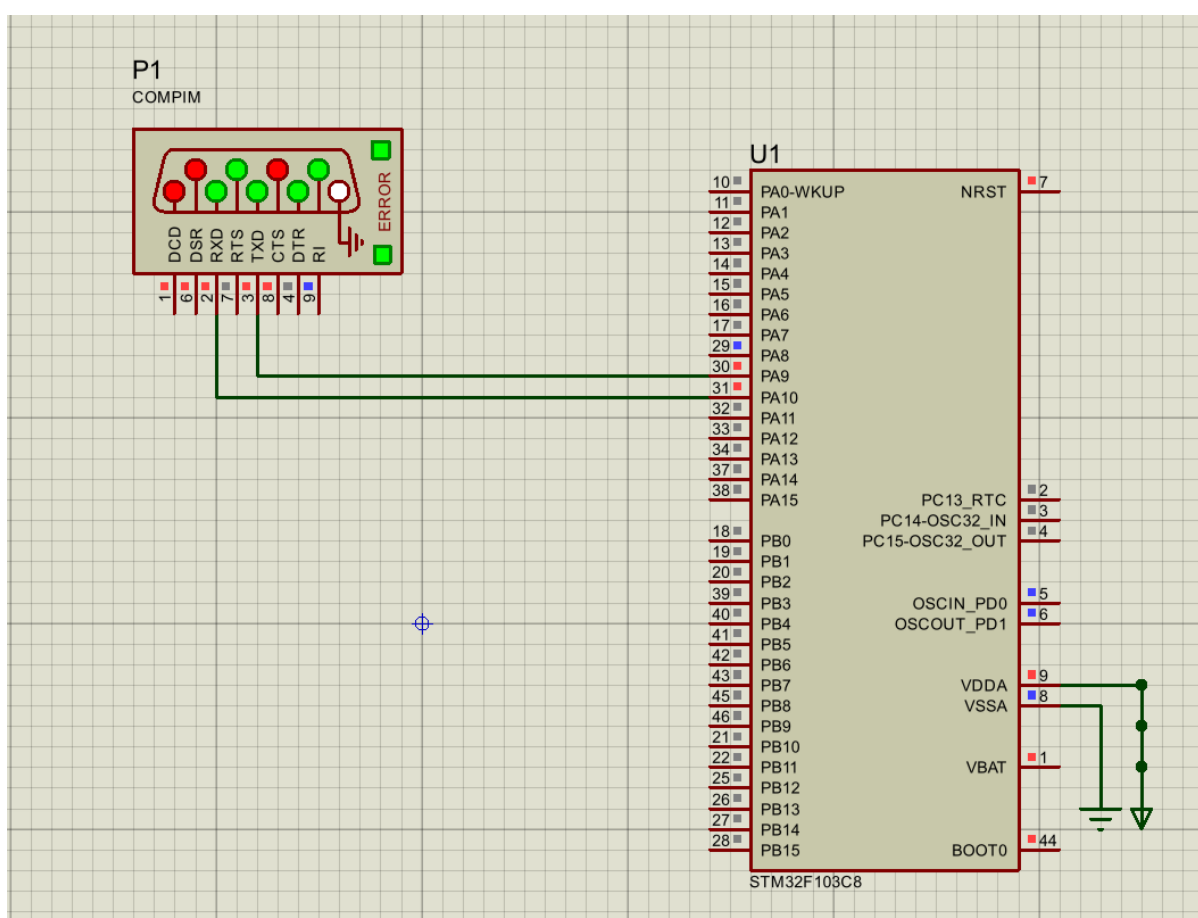
        HAL_UART_Receive_IT(&huart1, &received_byte, 1);
    }
}
```

```

}
}
/* USER CODE END 4 */

```

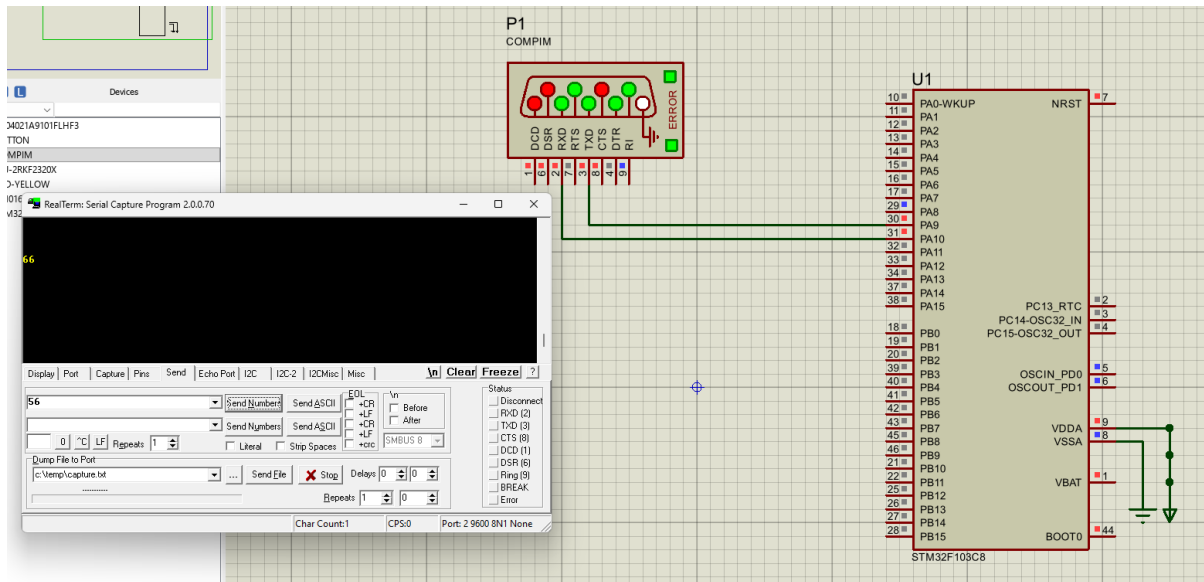
iii) بخش شبیه سازی Proteus



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

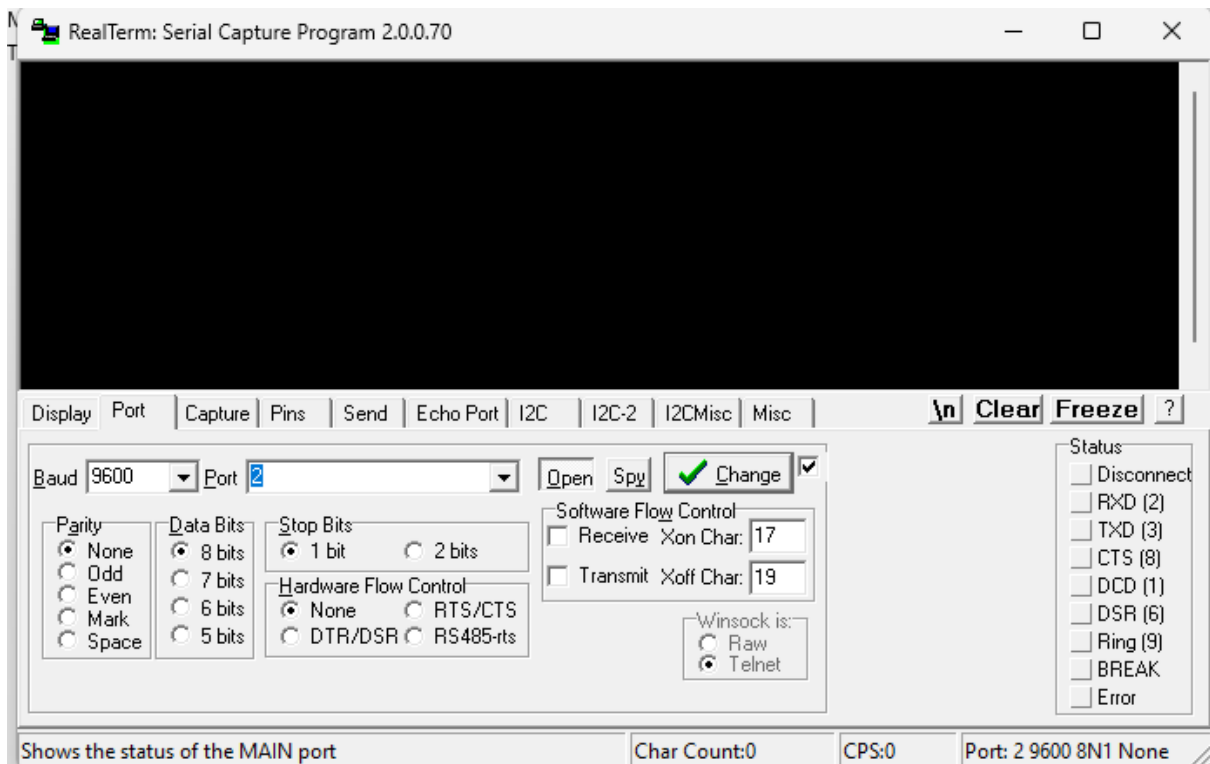
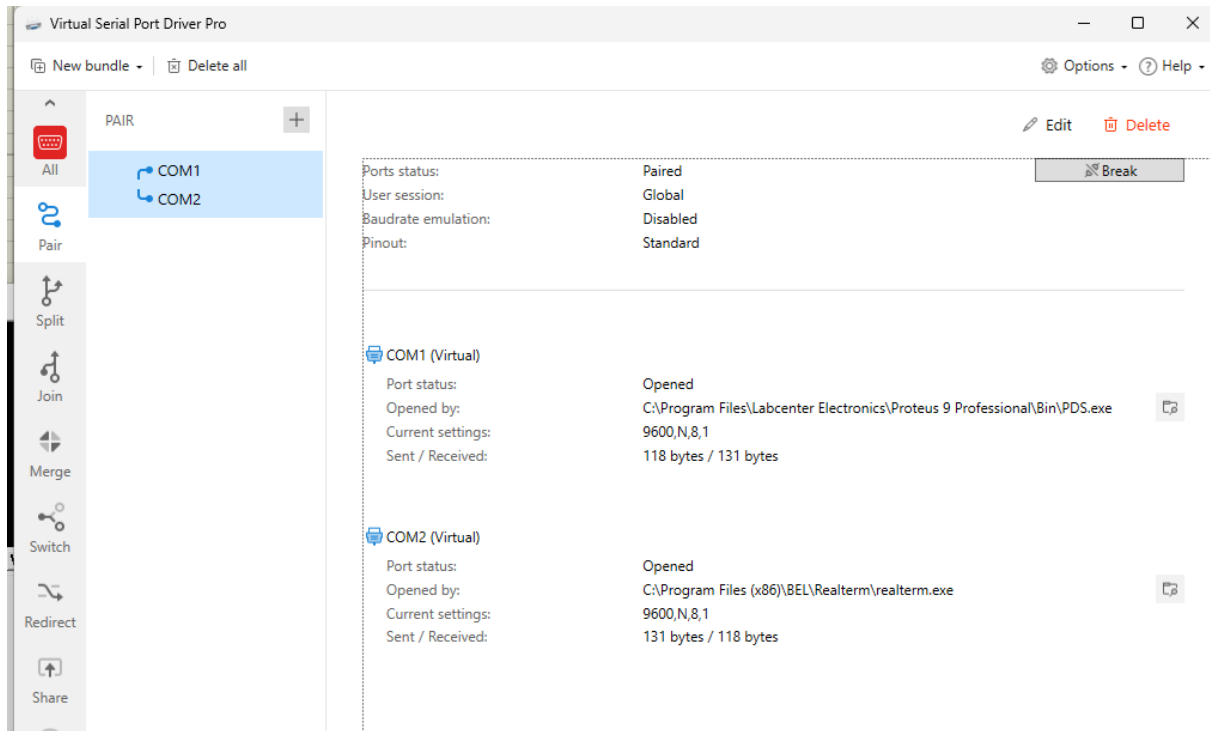
COMPIM را در محیط Proteus قرار می دهیم و پین های Rx و Tx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.



4. بخش Virtual serial port pro و RealTerm

یک کانیکشن pair می سازیم



Edit Component ? X

Part Reference:	P1	Hidden: ☐
Part Value:	COMPIM	Hidden: ☐
Element:	New	
VSM Model:	COMPIM.DLL	Hide All v
Physical port:	COM1	Hide All v
Physical Baud Rate:	9600	Hide All v
Physical Data Bits:	8	Hide All v
Physical Parity:	NONE	Hide All v
Virtual Baud Rate:	9600	Hide All v
Virtual Data Bits:	8	Hide All v
Virtual Parity:	NONE	Hide All v
Advanced Properties:		
Physical Stop Bits	1	Hide All v
Other Properties		
☐ Exclude from Simulation	☐ Attach hierarchy module	
☐ Exclude from PCB Layout	☐ Hide common pins	
☐ Exclude from Current Variant	☐ Edit all properties as text	

OK Cancel Help

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

DMA_1. از طریق ترمینال نرم افزار Realterm اعداد دلخواهی (در بازه 0 تا 240) را به میکروکنترلر ارسال کنید؛ عدد دریافت شده در میکروکنترلر را با عدد ۱۰ جمع کرده و دوباره به ترمینال نرم افزار Realterm، بر گردانید.

1.3 روش DMA

در روش DMA، داده‌ها بدون دخالت CPU و مستقیماً بین پریفرال UART و حافظه جابه‌جا می‌شوند. این روش حتی از وقفه هم کم‌بارتر است، چون CPU فقط در ابتدا و انتهای انتقال بلوک داده درگیر می‌شود، نه برای هر بایت.

(بخش STM32CubeMX)

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می‌کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم

گزینه ی مربوط به dma را می‌زنیم

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

DMA Request	Channel	Direction	Priority
USART1_RX	DMA1 Channel 5	Peripheral To Memory	Low
Select		Select	

Add Delete

DMA Request Settings

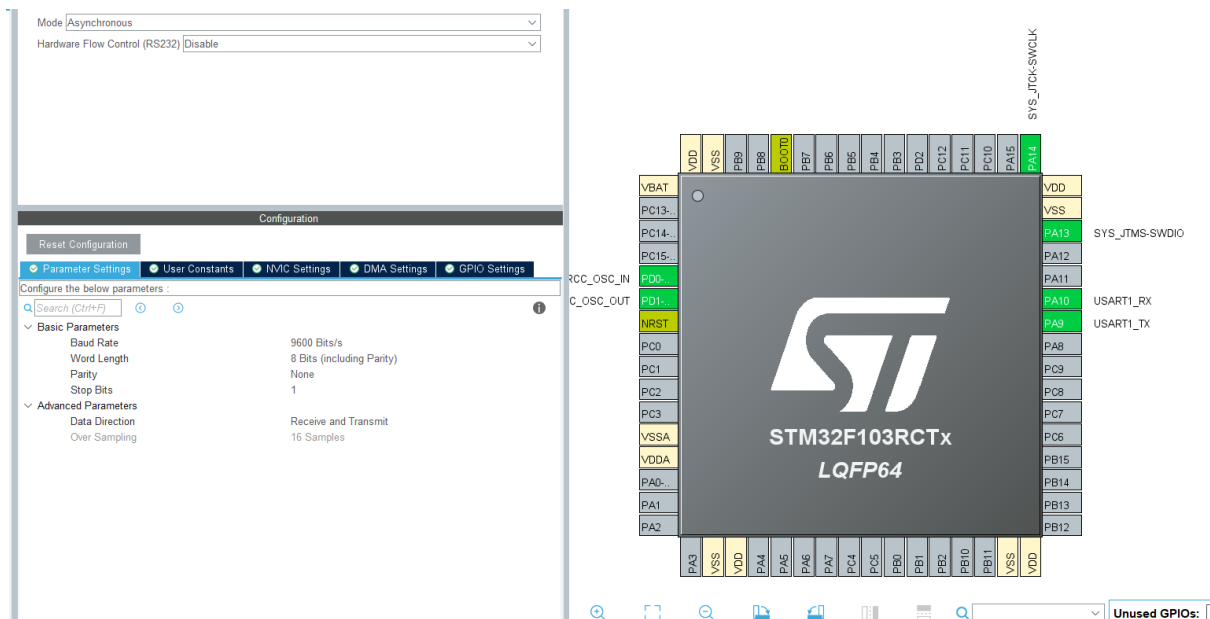
Mode Circular

Increment Address ☐

Data Width Byte

Peripheral ☐ Memory ☒

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



بخش Keil (ii)

در روش DMA ، سخت افزار داده را مستقیماً بدون وقفه و دخالت CPU از UART به حافظه منتقل می کند و CPU تنها پس از تکمیل انتقال بایت، یک بار در تابع فراخوانی (Callback) وارد می شود. این روش حتی از وقفه نیز کمپارتر است، مخصوصاً برای دریافت بلاک های داده بزرگ یا سرعت های بالا.

```
/* USER CODE BEGIN 0 */
UART_HandleTypeDef huart1;
DMA_HandleTypeDef hdma_usart1_rx;

uint8_t dma_rx_buffer[1];
uint8_t result_to_send_dma;
/* USER CODE END 0 */
```

ساختارهای UART_HandleTypeDef و DMA_HandleTypeDef برای پیکربندی سخت افزاری UART و DMA استفاده می شوند. بافر dma_rx_buffer محلی در حافظه است که داده دریافتی توسط DMA مستقیماً در آن ذخیره می شود و result_to_send_dma مقدار نهایی را برای ارسال نگه می دارد.

درداخل main:

دستور HAL_UART_Receive_DMA کانال DMA را برای دریافت خودکار یک بایت از UART به بافر مشخص شده راه اندازی می کند. پس از این خط، CPU کاملاً آزاد است و تا زمانی که داده نرسیده، هیچ وقفه یا اشغالی رخ نمی دهد.

```
/* USER CODE BEGIN 2 */
if (HAL_UART_Receive_DMA(&huart1, dma_rx_buffer, 1) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE END 2 */
```

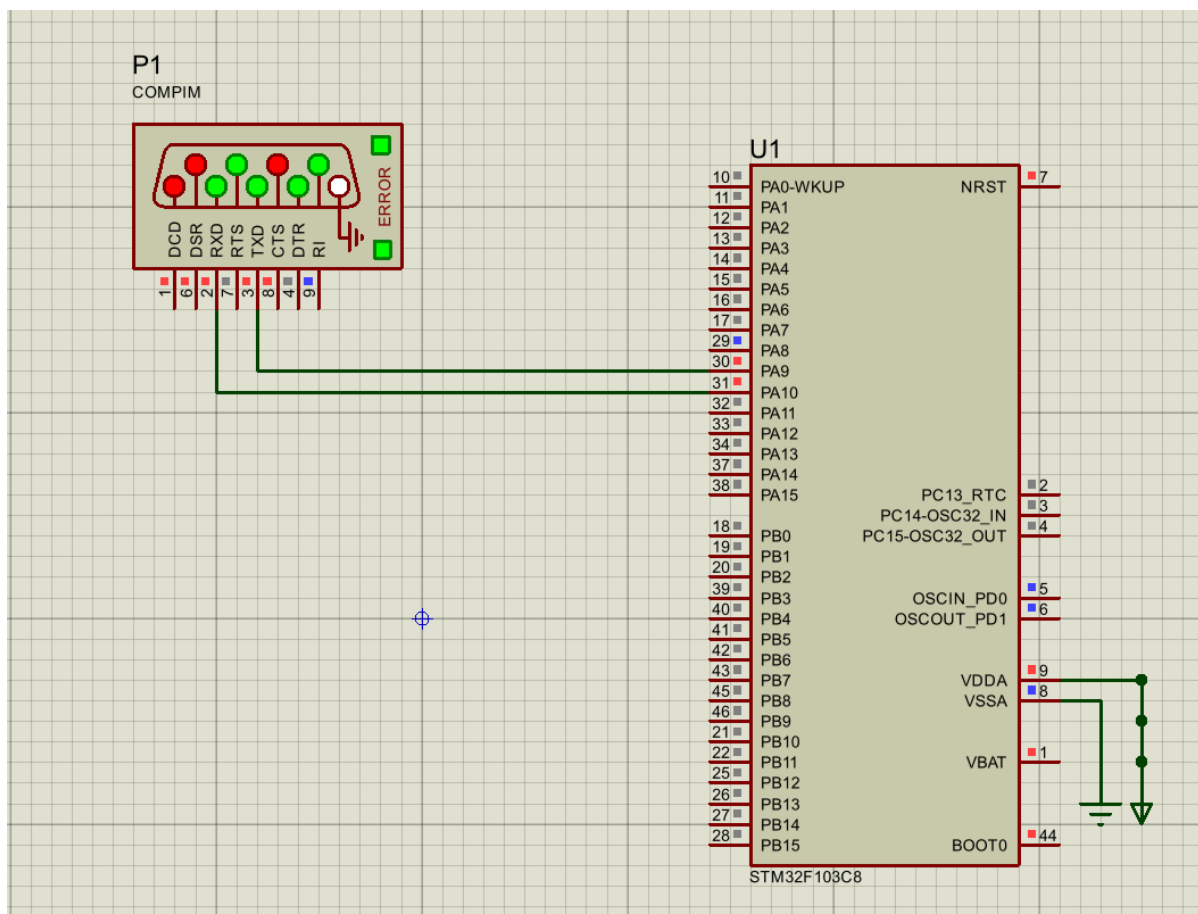
در داخل قسمت 4 user code در پایین فایل main.c:

این تابع به طور خودکار پس از اتمام دریافت یک بایت توسط DMA فراخوانی می‌شود. در آن مقدار بایت دریافتی (از بافر DMA) بررسی، با ۱۰ جمع و از طریق UART ارسال می‌گردد و سپس دریافت DMA دوباره فعال می‌شود تا بایت بعدی بدون دخالت CPU دریافت شود.

```
/* USER CODE BEGIN 4 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (dma_rx_buffer[0] <= 240)
        {
            result_to_send_dma = dma_rx_buffer[0] + 10;
        }
        else
        {
            result_to_send_dma = 255;
        }

        HAL_UART_Transmit(&huart1, &result_to_send_dma, 1, HAL_MAX_DELAY);

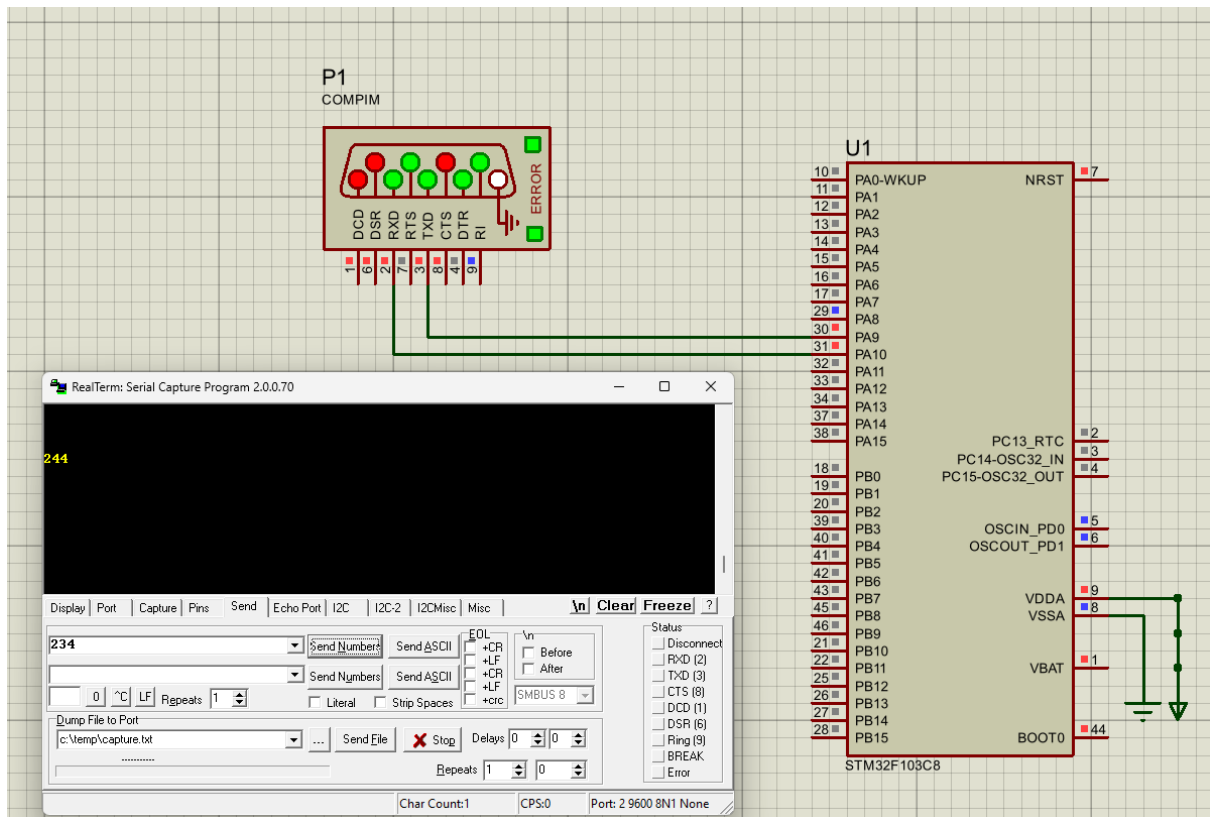
        HAL_UART_Receive_DMA(&huart1, dma_rx_buffer, 1);
    }
}
/* USER CODE END 4 */
```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

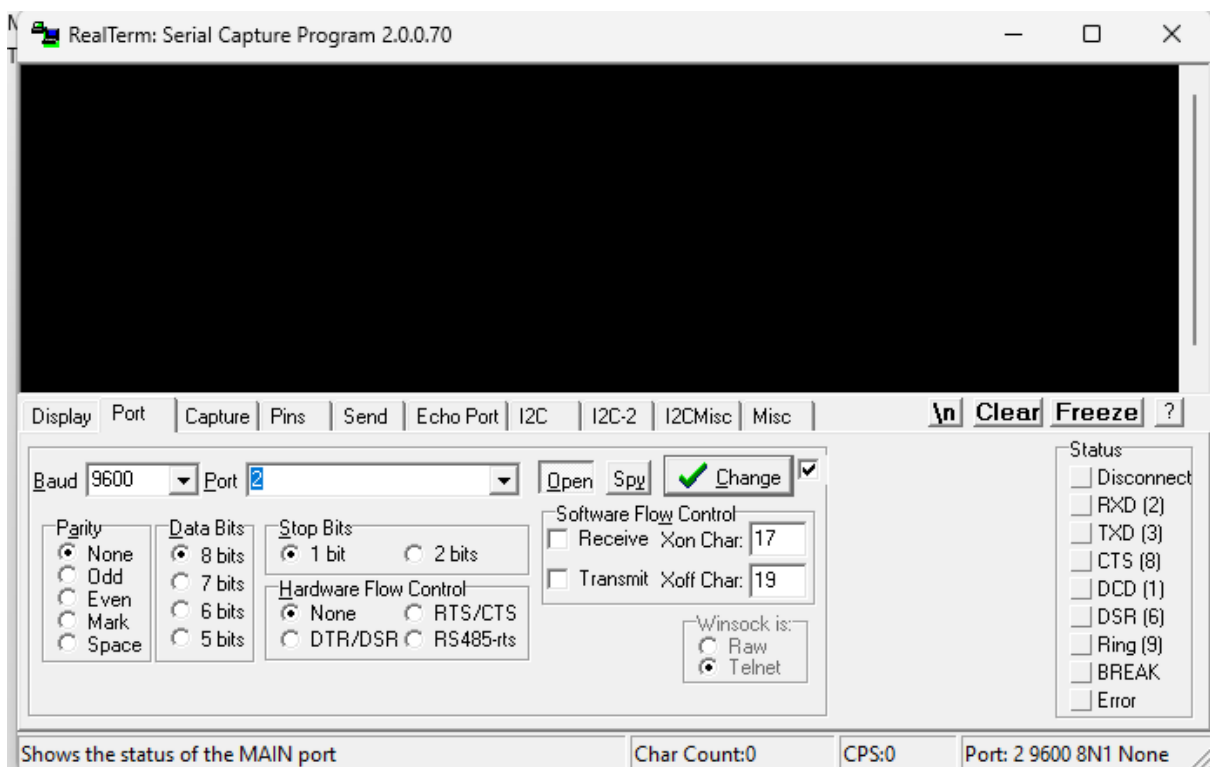
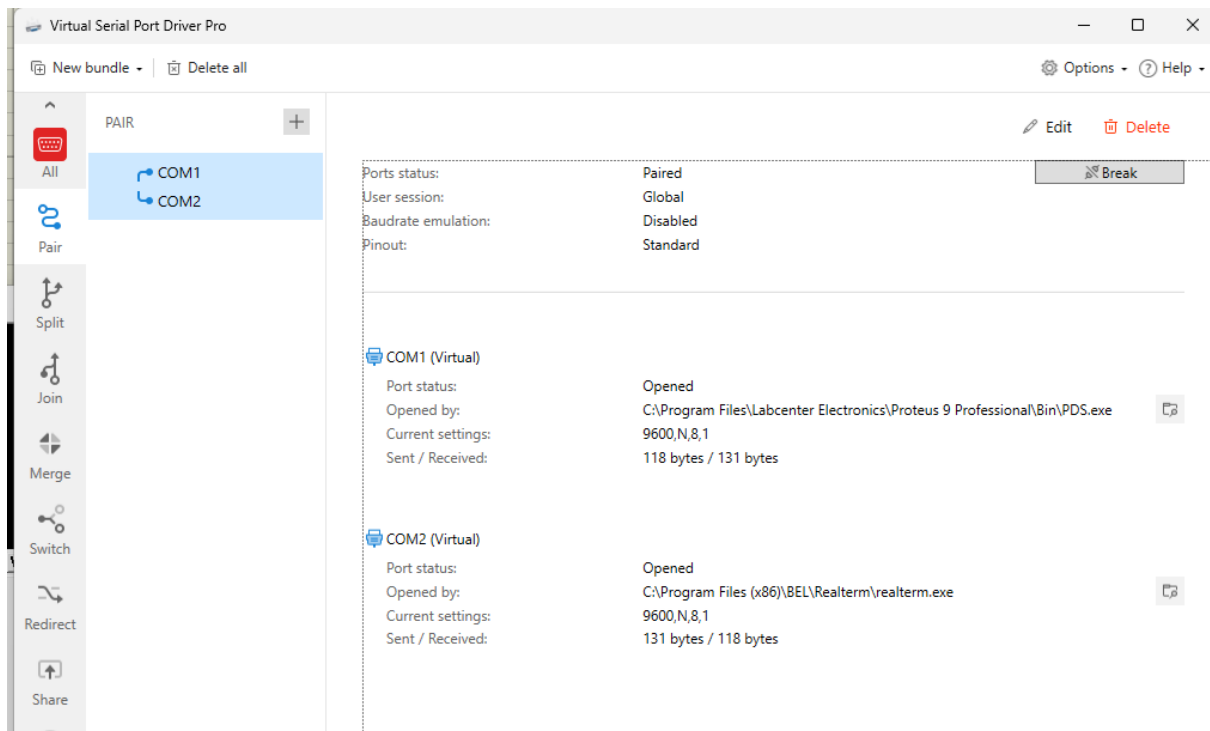
COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.



4. بخش Virtual serial port pro و RealTerm

یک کانیکشن pair می سازیم



Edit Component ? X

Part Reference:	P1	Hidden: ☐
Part Value:	COMPIM	Hidden: ☐
Element:	New	
VSM Model:	COMPIM.DLL	Hide All v
Physical port:	COM1	Hide All v
Physical Baud Rate:	9600	Hide All v
Physical Data Bits:	8	Hide All v
Physical Parity:	NONE	Hide All v
Virtual Baud Rate:	9600	Hide All v
Virtual Data Bits:	8	Hide All v
Virtual Parity:	NONE	Hide All v
Advanced Properties:		
Physical Stop Bits v	1	Hide All v
Other Properties		
☐ Exclude from Simulation	☐ Attach hierarchy module	
☐ Exclude from PCB Layout	☐ Hide common pins	
☐ Exclude from Current Variant	☐ Edit all properties as text	

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

2_Polling. تمرین فوق را به جای ترمینال نرم افزار Realterm ، با نرم افزار Matlab

انجام دهید.

(i) بخش STM32CubeMX

مانند سوال قبل

(ii) بخش Keil

مانند سوال قبل

(iii) بخش شبیه سازی Proteus

مانند سوال قبل

4. بخش MATLAB و Virtual serial port pro

این اسکریپت MATLAB به پورت سریال متصل می‌شود، یک عدد از کاربر گرفته (۰ تا ۲۴۰)، آن را به صورت یک بایت به میکروکنترلر می‌فرستد، پاسخ (عدد + ۱۰) را دریافت کرده و نمایش می‌دهد. حلقه اصلی تا زدن exit ادامه دارد و برای تست هر سه روش (DMA، Interrupt، Polling) یکسان عمل می‌کند، زیرا رفتار نهایی فرستنده/گیرنده در هر سه یکی است.

پاک کردن صفحه و متغیرها

```
clear; clc;
```

این دو دستور صفحه فرمان (Command Window) را پاک می‌کنند (clc) و همه متغیرهای حافظه کار را پاک می‌کنند (clear). برای شروع تازه و جلوگیری از تداخل متغیرهای قبلی استفاده می‌شود.

تنظیمات اولیه پورت سریال

```
serialPortName = 'COM2';
```

```
baudRate = 9600;
```

```
timeoutSeconds = 5;
```

- serialPortName: نام پورت سریالی که میکروکنترلر به آن وصل است (مثلاً COM1 که در COM1PIM یا نرم‌افزار شبیه‌ساز پورت مجازی تنظیم شده).
- baudRate: سرعت تبادل داده که حتماً باید با مقدار تنظیم شده در STM32CubeMX (مثلاً 115200) یکی باشد.
- timeoutSeconds: حداکثر زمانی که متلب منتظر جواب از میکروکنترلر می‌ماند (در اینجا ۵ ثانیه).

بررسی وجود پورت و بستن اتصال قدیمی

```
availablePorts = serialportlist("available");
```

```
if ~ismember(serialPortName, availablePorts)
```

```

        error('Port %s not found. Check virtual COM port or Proteus COMPIM
settings.', serialPortName);
end

```

```

oldPort = serialportfind("Port", serialPortName);
if ~isempty(oldPort)
    delete(oldPort);
    fprintf('Closed existing connection on %s.\n', serialPortName);
end

```

```

fprintf('Connecting to %s at %d baud ...\n', serialPortName, baudRate);

```

- با serialportlist لیست پورت‌های موجود را گرفته و بررسی می‌شود که پورت مورد نظر وجود داشته باشد؛ در غیر این صورت خطا می‌دهد.
- serialportfind اتصال قبلی به همان پورت را پیدا کرده و با delete می‌بندد تا از تداخل جلوگیری شود.
- این کار مخصوصاً وقتی چند بار اسکریپت اجرا می‌شود ضروری است.

اتصال به پورت سریال

```

try
    stm32 = serialport(serialPortName, baudRate);
    stm32.Timeout = timeoutSeconds;
    fprintf('Connected successfully.\n\n');
catch ME
    error('Connection failed: %s\nCheck: Port number, cable, COMPIM settings,
no other app using it.', ME.message);
end

```

- دستور serialport یک شیء برای ارتباط سریال می‌سازد و در متغیر stm32 ذخیره می‌کند.
- خاصیت Timeout مدت انتظار برای عملیات خواندن را تعیین می‌کند.
- اگر اتصال برقرار نشود (پورت نامعتبر، در حال استفاده توسط برنامه دیگر، یا باسد اشتباه)، خطا گرفته شده و برنامه متوقف می‌شود.

حلقه اصلی برای ارسال و دریافت متوالی

```

fprintf('Enter an integer between 0 and 240. Type "exit" to quit.\n');
while true

```

```

userInput = input('Your number (0-240): ', 's');
if strcmpi(userInput, 'exit')
    break;
end

```

- یک حلقه بی‌نهایت (while true) ایجاد شده که کاربر می‌تواند بارها عدد بفرستد و پاسخ بگیرد.
- با تایپ exit حلقه شکسته شده و برنامه خاتمه می‌یابد.
- ورودی به صورت رشته ('s') گرفته می‌شود تا امکان تشخیص دستور exit وجود داشته باشد.

اعتبارسنجی عدد ورودی

```

number = str2double(userInput);
if isnan(number) || number < 0 || number > 240 || mod(number,1) ~= 0
    fprintf('Invalid input. Please enter an integer from 0 to 240.\n');
    continue;
end

```

- رشته ورودی به عدد اعشاری تبدیل می‌شود (str2double).
- شرطها: اگر عدد نباشد (isnan)، یا از بازه خارج باشد، یا اعشاری باشد (باقیمانده تقسیم بر ۱ مخالف صفر)، پیام خطا چاپ کرده و دوباره از ابتدای حلقه می‌پرد (continue).
- این کار از ارسال داده نامعتبر به میکرو جلوگیری می‌کند.

تبدیل عدد به بایت و ارسال

```

dataToSend = uint8(number);
try
    write(stm32, dataToSend, "uint8");
    fprintf('Sent: %d (0x%s)\n', number, dec2hex(dataToSend));
catch ME
    fprintf('Send error: %s\n', ME.message);
    break;
end

```

- عدد صحیح به تایپ uint8 (یک بایت بدون علامت) تبدیل می‌شود.
- تابع write این بایت را روی پورت سریال می‌فرستد.

- اگر خطایی در ارسال پیش آید (مثل قطع شدن پورت)، پیام خطا نشان داده شده و حلقه متوقف می‌شود.

خواندن پاسخ از میکروکنترلر

```
try
    received = read(stm32, 1, "uint8");
    fprintf('Received: %d (0x%s) → Expected: %d\n', ...
        received, dec2hex(received), number + 10);
catch ME
    if contains(ME.message, 'timeout')
        fprintf('Timeout: No response from STM32.\n');
        fprintf('Check: Is the STM32 firmware running? TX/RX wired correctly?\n');
    else
        fprintf('Receive error: %s\n', ME.message);
    end
    break;
end
```

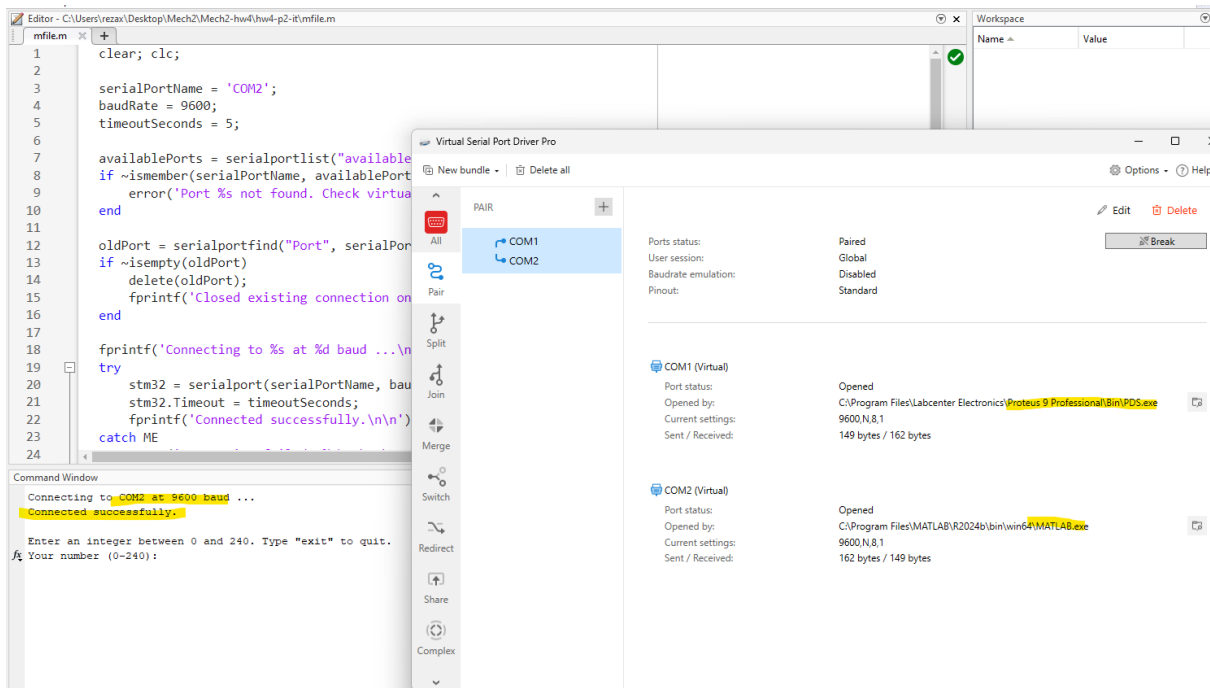
- با `read(stm32, 1, "uint8")` دقیقاً یک بایت از میکرو کنترلر خوانده می‌شود.
- در صورت موفقیت، بایت دریافتی و مقدار مورد انتظار (عدد ارسالی + ۱۰) چاپ می‌گردد.
- اگر خطای زمان انتظار (Timeout) رخ دهد، به کاربر هشدار داده می‌شود که احتمالاً کد میکرو درست کار نمی‌کند یا سیم‌های TX/RX اشتباه وصل شده‌اند.

پاکسازی و بستن پورت

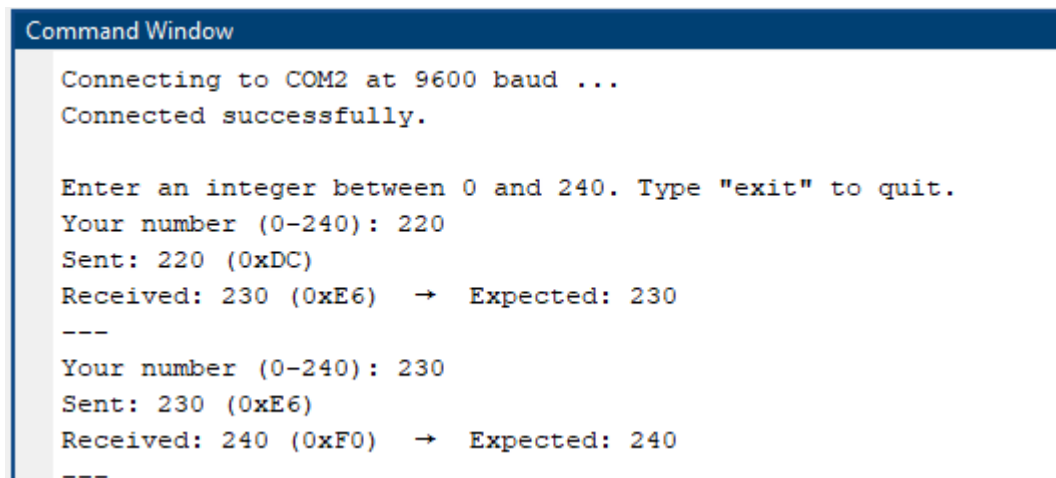
```
clear stm32;
fprintf('\nSerial port closed. Done.\n');
```

- دستور `clear stm32` شیء سریال را از حافظه حذف می‌کند که باعث بسته شدن خودکار پورت می‌شود.
- پیغام پایان کار نمایش داده می‌شود. این بخش بعد از شکستن حلقه اصلی اجرا می‌گردد.

تنظیمات مربوط به بخش virtual serial port driver



تست ارتباط:



2IT. تمرین فوق را به جای ترمینال نرم افزار Realterm ، با نرم افزار Matlab انجام دهید.

بخش STM32CubeMX (i)

مانند سوال قبل

بخش Keil (ii)

مانند سوال قبل

بخش شبیه سازی Proteus (iii)

مانند سوال قبل

بخش MATLAB و Virtual serial port pro 4.

مانند سوال قبل

```
Command Window

Connecting to COM2 at 9600 baud ...
Connected successfully.

Enter an integer between 0 and 240. Type "exit" to quit.
Your number (0-240): 0
Sent: 0 (0x0)
Received: 10 (0xA) → Expected: 10
---
Your number (0-240): -5
Invalid input. Please enter an integer from 0 to 240.
Your number (0-240): 5
Sent: 5 (0x5)
Received: 15 (0xF) → Expected: 15
---
fx Your number (0-240): |
```

2_DMA. تمرین فوق را به جای ترمینال نرم افزار Realterm ، با نرم افزار Matlab انجام

دهید .

بخش STM32CubeMX (i)

مانند سوال قبل

بخش Keil (ii)

مانند سوال قبل

بخش شبیه سازی Proteus (iii)

مانند سوال قبل

بخش MATLAB و Virtual serial port pro 4.

مانند سوال قبل

Command Window

```
Connecting to COM2 at 9600 baud ...  
Connected successfully.
```

```
Enter an integer between 0 and 240. Type "exit" to quit.
```

```
Your number (0-240): 66
```

```
Sent: 66 (0x42)
```

```
Received: 76 (0x4C) → Expected: 76
```

```
---
```

```
Your number (0-240): 245
```

```
Invalid input. Please enter an integer from 0 to 240.
```

```
Your number (0-240): 220
```

```
Sent: 220 (0xDC)
```

```
Received: 230 (0xE6) → Expected: 230
```

```
---
```

```
 Your number (0-240):
```

3_Polling. شمارنده‌ای در میکرو کنترلر ایجاد کنید که اعداد از ۰ تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از ۲۵۵ تا ۰ را مجدد بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندانه اره‌ای). این اعداد را در همان حین تولید در میکروکنترلر، به کامپیوتر ارسال کرده و در نرمافزار Matlab، دریافت نموده و در یک آرایه بریزید و بعد از چند سری دریافت در نرم افزار Matlab، آن را رسم نمایید.

3.1 روش Poling

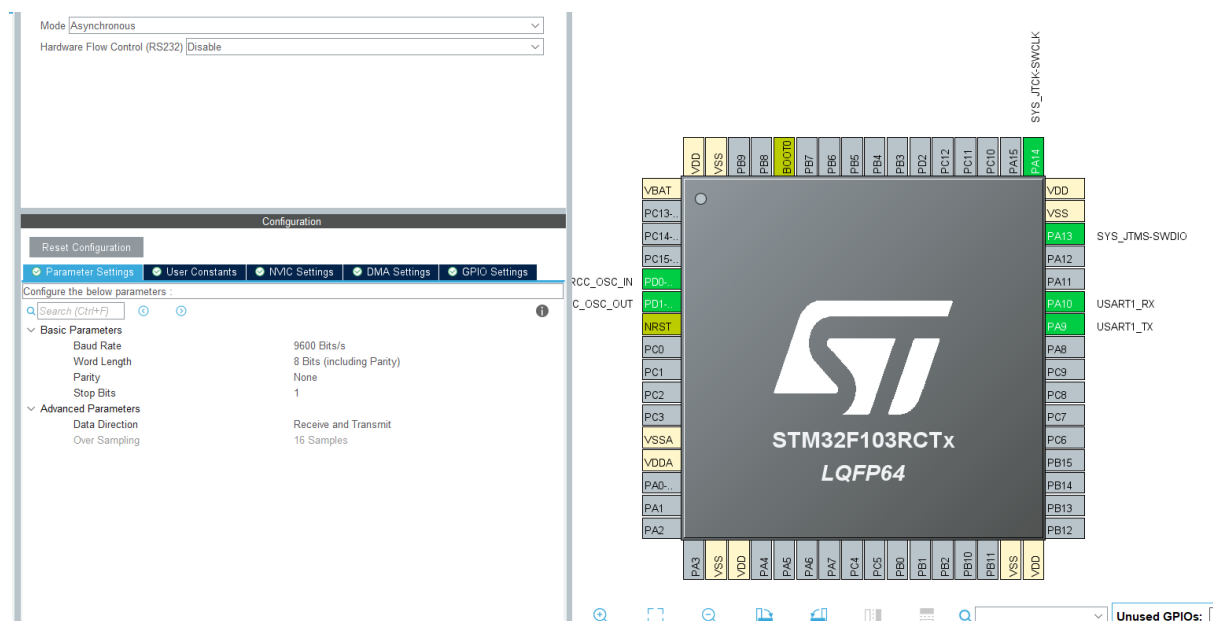
در این روش CPU به طور مداوم وضعیت رجیستر UART را بررسی می کند.

بخش STM32CubeMX

میکرو کنترلر STM32F103RCT دو در نرم افزار انتخاب میکنیم.

UART1, فعال، مے، کنیم و BaudRate، دو، روی 9600 Bits/s تنظیم میکنیم

پروگرام رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



متغیر sawtooth_value مقدار فعلی سیگنال دندان‌اره (۰ تا ۲۵۵) را نگه می‌دارد. تابع generate_next_sawtooth با برگرداندن 1 + current مقدار بعدی را تولید می‌کند؛ به دلیل نوع uint8_t، سرریز از ۲۵۵ به صفر به صورت خودکار انجام می‌شود.

```
/* USER CODE BEGIN 0 */
static uint8_t sawtooth_value = 0;
static inline uint8_t generate_next_sawtooth(uint8_t current) {
    return current + 1;
}
/* USER CODE END 0 */
```

در داخل while:

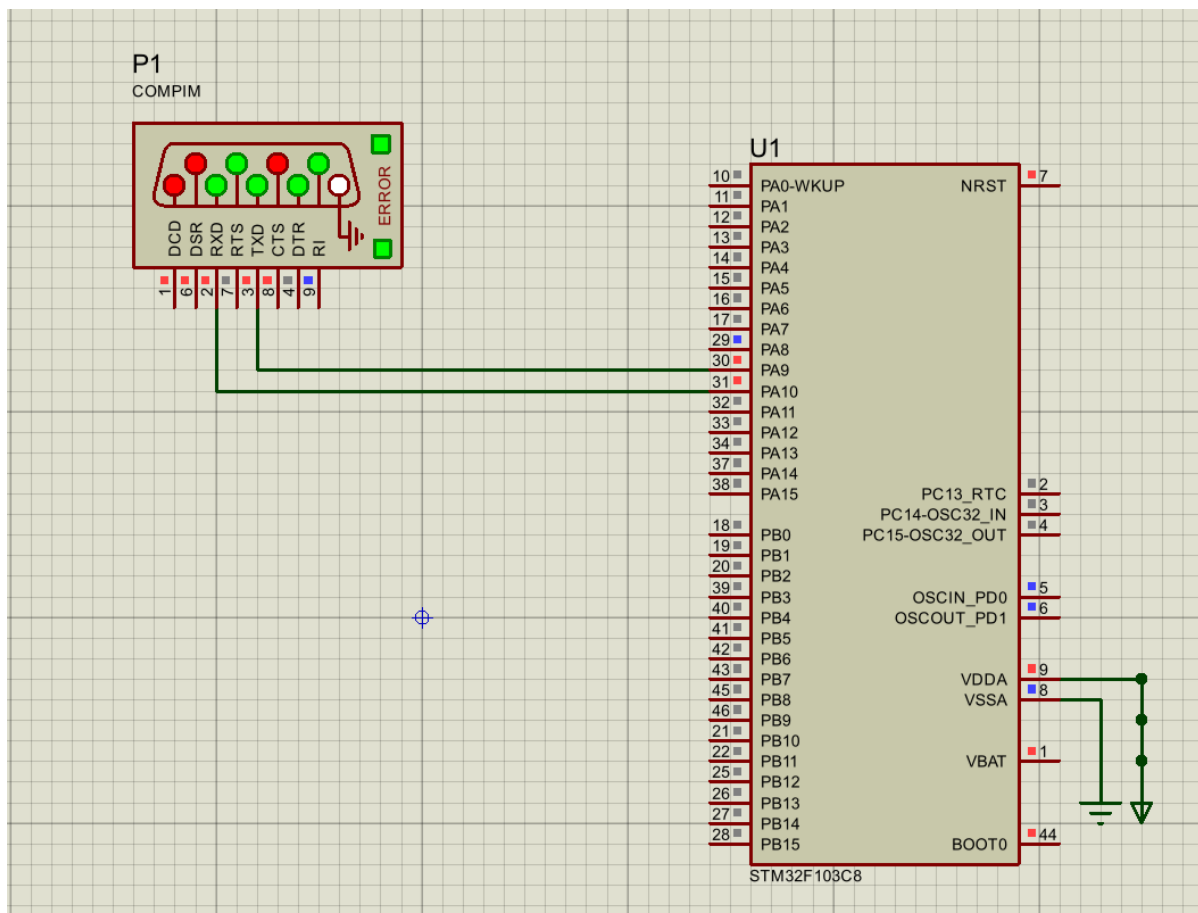
ابتدا مقدار فعلی دندان‌اره در data_to_send ذخیره و از طریق UART ارسال می‌شود. سپس مقدار بعدی محاسبه شده، و یک تأخیر ساده (حلقه for) برای کنترل نرخ ارسال قبل از تکرار بعدی اعمال می‌گردد.

```
/* USER CODE BEGIN 3 */
    uint8_t data_to_send = sawtooth_value;

    if (HAL_UART_Transmit(&huart1, &data_to_send, 1, HAL_MAX_DELAY) !=
HAL_OK)
    {
        Error_Handler();
    }

    sawtooth_value = generate_next_sawtooth(sawtooth_value);

    for (volatile int i = 0; i < 1000; i++);
}
/* USER CODE END 3 */
```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

COM1

COM2

Ports status:	Paired
User session:	Global
Baudrate emulation:	Disabled
Pinout:	Standard

 Edit  Delete

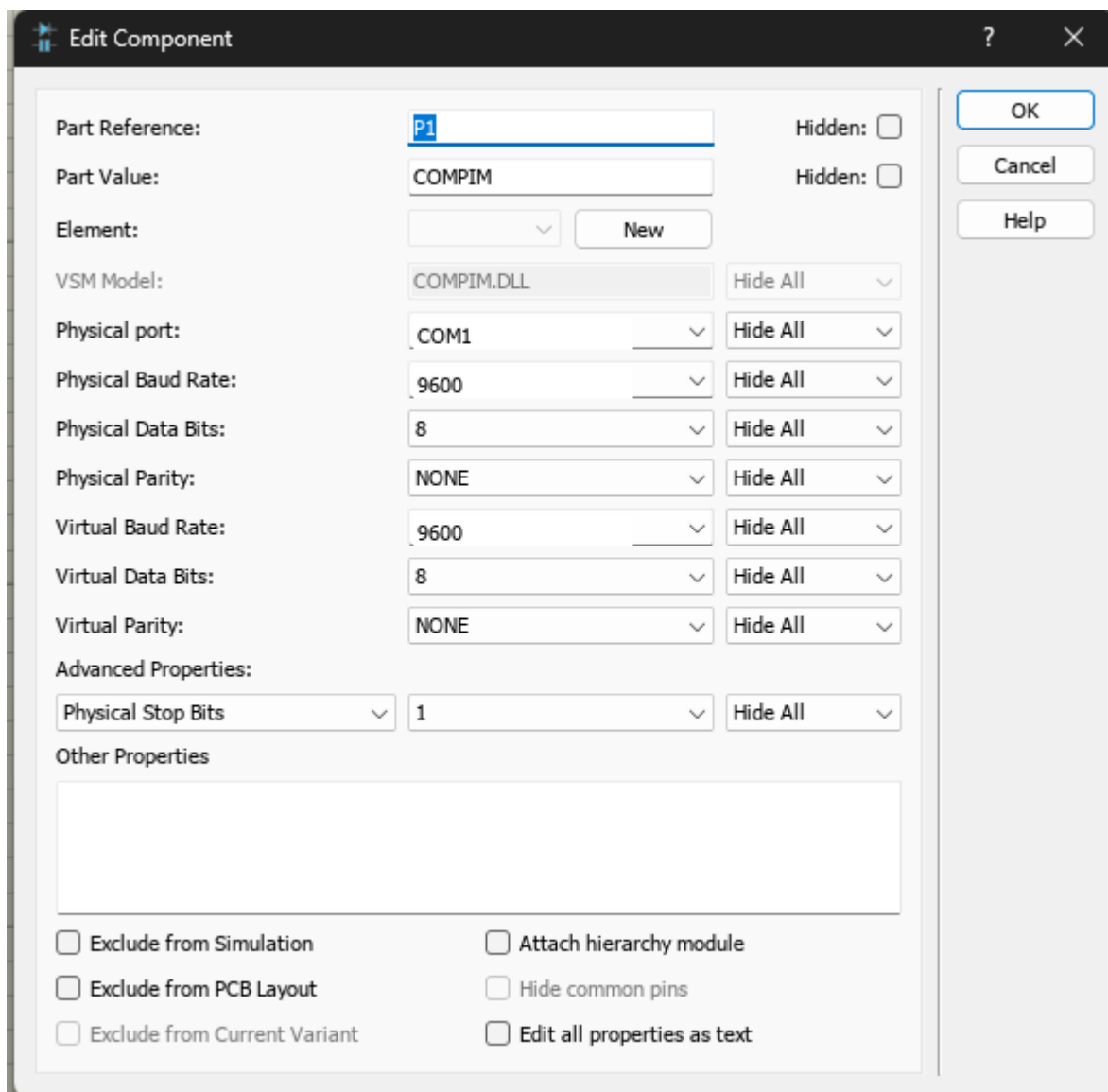
 Break

COM1 (Virtual)

Port status:	Opened
Opened by:	...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe 
Current settings:	9600,N,8,1
Sent / Received:	1.4 KB / 0 bytes

COM2 (Virtual)

Port status:	Opened
Opened by:	C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe 
Current settings:	9600,N,8,1
Sent / Received:	0 bytes / 1.3 KB



کد متلب:

پاک کردن حافظه و صفحه فرمان

```
clear; clc;
```

این دو دستور، تمام متغیرهای قبلی را از حافظه حذف کرده (clear) و صفحه فرمان متلب را پاک می‌کنند (clc) برای شروع تازه و جلوگیری از تداخل متغیرهای قبلی استفاده می‌شود.

تنظیمات پورت سریال و پارامترهای دریافت

```
PORT_NAME = 'COM2';  
BAUD_RATE = 9600;  
  
SAMPLES_PER_CYCLE = 256;  
CYCLES_TO_GET = 3;  
TOTAL_SAMPLES = SAMPLES_PER_CYCLE * CYCLES_TO_GET;
```

- **PORT_NAME**: نام پورت سریال (باید با تنظیمات COMPIIM یا شبیه‌ساز مجازی هماهنگ باشد).
- **BAUD_RATE**: نرخ تبادل داده (باید با مقدار تنظیم شده در STM32CubeMX برابر باشد؛ معمولاً 9600 یا 115200).
- **SAMPLES_PER_CYCLE**: یک سیکل کامل سیگنال دندان‌اره شامل ۲۵۶ مقدار (از ۰ تا ۲۵۵).
- **CYCLES_TO_GET**: تعداد سیکل‌های دندان‌اره‌ای که می‌خواهیم جمع‌آوری کنیم.
- **TOTAL_SAMPLES**: کل نمونه‌هایی که باید دریافت شوند (مثلاً $256 \times 3 = 768$).

بستن اتصالات قبلی (در صورت وجود)

```
if ismember(PORT_NAME, serialportlist("available"))  
    oldPort = serialportfind("Port", PORT_NAME);  
    if ~isempty(oldPort)  
        delete(oldPort);  
    end  
end
```

- ابتدا بررسی می‌شود که آیا پورت مورد نظر در لیست پورت‌های موجود هست.
- اگر قبلاً یک شیء سریال به آن پورت متصل بوده، با `delete` serialportfind پیدا شده و با `delete` بسته می‌شود. این کار از خطای اشغال بودن پورت جلوگیری می‌کند.

ایجاد و پیکربندی شیء سریال

```
try  
    serialObj = serialport(PORT_NAME, BAUD_RATE);  
    serialObj.Timeout = 2;  
catch ME  
    error('Connection failed: %s', ME.message);  
end
```

- با دستور serialport یک ارتباط جدید روی پورت و با نرخ مشخص برقرار می‌شود.
- خاصیت Timeout مدت زمان انتظار برای عملیات خواندن را تعیین می‌کند (اینجا ۲ ثانیه).
- در صورت خطا (مثلاً پورت موجود نیست یا باسد اشتباه است)، برنامه متوقف شده و پیغام خطا نمایش داده می‌شود.

آماده‌سازی آرایه و پنجره نمودار

```
buffer = zeros(1, TOTAL_SAMPLES, 'uint8');
samplesReceived = 0;

fig = figure;
ax = axes('Parent', fig);
plotHandle = plot(ax, NaN, NaN, 'b-');
xlabel(ax, 'Sample Number');
ylabel(ax, 'Value (0-255)');
title(ax, sprintf('Collecting %d samples', TOTAL_SAMPLES));
grid(ax, 'on');
axis(ax, [0 TOTAL_SAMPLES 0 260]);

flush(serialObj);
```

- Buffer: آرایه‌ای از نوع uint8 به طول TOTAL_SAMPLES برای ذخیره داده‌های دریافتی.
- samplesReceived: شمارنده نمونه‌هایی که تاکنون دریافت شده‌اند.
- یک پنجره جدید ایجاد شده و محورها و خط نمودار (با داده اولیه NaN) تنظیم می‌شوند.
- flush(serialObj) بافر ورودی پورت را پاک می‌کند تا داده‌های قدیمی باعث اختلال نشوند.

حلقه اصلی دریافت داده

```
while samplesReceived < TOTAL_SAMPLES
    if ~isgraphics(fig)
        break;
    end
    if serialObj.NumBytesAvailable > 0
        toRead = min(serialObj.NumBytesAvailable, TOTAL_SAMPLES -
samplesReceived);
        newData = read(serialObj, toRead, "uint8");
        if ~isempty(newData)
            startIdx = samplesReceived + 1;
            endIdx = samplesReceived + length(newData);
            buffer(startIdx:endIdx) = newData;
            samplesReceived = endIdx;
            set(plotHandle, 'XData', 1:samplesReceived, 'YData',
buffer(1:samplesReceived));
            drawnow limitrate;
        end
    end
end
```

```

        end
    else
        pause(0.01);
    end
end

```

- تا زمانی که نمونه‌های کافی دریافت نشده، حلقه ادامه دارد.
- اگر کاربر پنجره نمودار را ببندد، حلقه شکسته می‌شود.
- هر بار که تعدادی بایت در بافر ورودی موجود باشد، آنها را خوانده و به آرایه `buffer` اضافه می‌کند.
- پس از هر مرحله، نمودار به‌روزرسانی می‌شود (`drawnow limitrate` set).
- اگر داده‌ای موجود نباشد، یک مکث کوتاه (۰.۰۱ ثانیه) انجام می‌شود تا پردازنده بیش از حد بار نیاید.

نمایش پیام پایان یا توقف زود هنگام

```

if samplesReceived >= TOTAL_SAMPLES
    fprintf('Acquisition complete.\n');
elseif isgraphics(fig)
    fprintf('Stopped early: %d of %d samples received.\n', samplesReceived,
TOTAL_SAMPLES);
end

```

- اگر همه نمونه‌های مورد نظر دریافت شده باشند، پیام "تکمیل شد" چاپ می‌شود.
- در غیر این صورت (اگر پنجره بسته شده باشد یا خطایی رخ داده باشد)، پیام توقف زود هنگام با ذکر تعداد نمونه‌های دریافتی نمایش داده می‌شود.

رسم نهایی نمودار

```

if isgraphics(fig)
    title(ax, sprintf('Final plot - %d samples', samplesReceived));
    set(plotHandle, 'XData', 1:samplesReceived, 'YData',
buffer(1:samplesReceived));
    axis(ax, [0 samplesReceived 0 260]);
else
    if samplesReceived > 0
        figure;
        plot(1:samplesReceived, buffer(1:samplesReceived), 'r-');
        xlabel('Sample Number');
        ylabel('Value (0-255)');
        title(sprintf('Data from STM32 (%d samples)', samplesReceived));
    end
end

```

```

        grid on;
        axis([0 samplesReceived 0 260]);
    end
end

```

- اگر پنجره اصلی هنوز باز باشد، عنوان آن تغییر کرده و داده‌های نهایی به طور کامل در همان پنجره رسم می‌شوند.
- اگر پنجره بسته شده باشد ولی داده‌ای ذخیره شده باشد، یک پنجره جدید باز شده و نمودار با رنگ قرمز رسم می‌شود.
- محدوده محورها متناسب با تعداد نمونه‌های دریافتی تنظیم می‌شود.

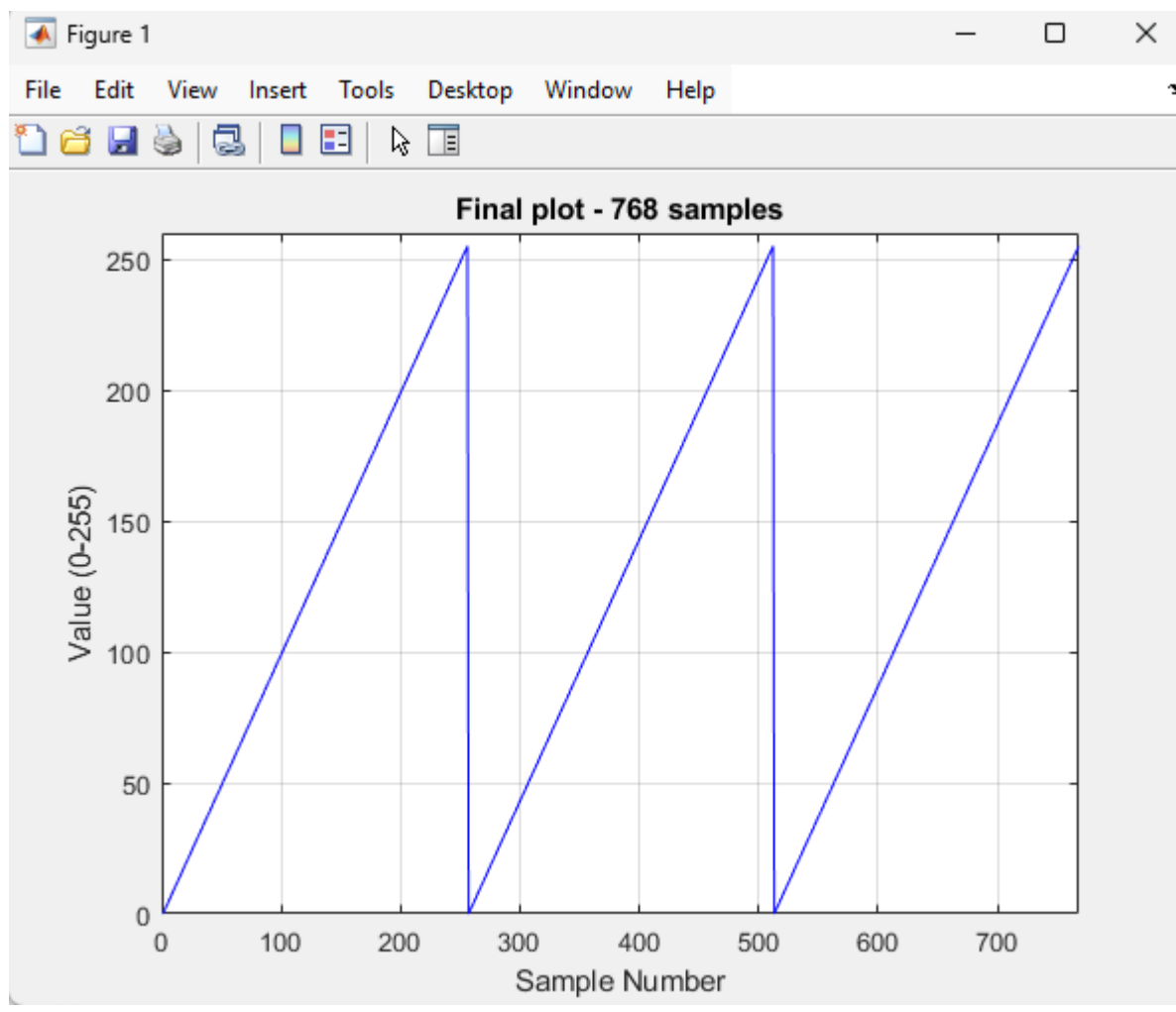
بستن و پاکسازی پورت سریال

```

if exist('serialObj', 'var')
    clear serialObj;
end

```

- اگر متغیر serialObj وجود داشته باشد (یعنی ارتباط برقرار شده بود)، با دستور clear آن را حذف می‌کنیم که باعث بسته شدن خودکار پورت سریال می‌شود.
- این کار از باقی ماندن یک اتصال باز و اشغال پورت جلوگیری می‌کند.



ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

3_IT. شمارنده‌های در میکرو کنترلر ایجاد کنید که اعداد از ۰ تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از ۰ تا ۲۵۵ را مجدد بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندان‌اره‌ای). این اعداد را در همان حین تولید در میکروکنترلر، به کامپیوتر ارسال کرده و در نرم‌افزار Matlab، دریافت نموده و در یک آرایه بریزید و بعد از چند سری دریافت در نرم افزار Matlab، آن را رسم نمایید.

3.2 روش Intrupt

در روش اینترپت، میکروکنترلر بی‌آنکه مدام چک کند (پولینگ)، فقط وقتی داده می‌رسد وقفه ایجاد می‌شود و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

(i) بخش STM32CubeMX

همانند بخش i سوال 2 قسمت Intrupt

(ii) بخش Keil

uart1 ساختار مدیریت پورت سریال است که توسط CubeMX مقداردهی می‌شود. مقدار current_sawtooth. جاری سیگنال دندان‌اره (۰ تا ۲۵۵) را نگه می‌دارد و هر بار یک واحد افزایش می‌یابد tx_buffer_byte. بافری برای ذخیره بایت ارسالی است و transmit_complete_flag که با volatile تعریف شده، وضعیت آمادگی برای ارسال بعدی را نشان می‌دهد.

```
/* USER CODE BEGIN PV */
uint8_t current_sawtooth = 0;
uint8_t tx_buffer_byte;
volatile uint8_t transmit_complete_flag = 1;
/* USER CODE END PV */
```

```

/* USER CODE BEGIN 0 */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        transmit_complete_flag = 1;
    }
}
/* USER CODE END 0 */

```

این تابع به طور خودکار پس از اتمام ارسال هر بایت توسط وقفه صدا زده می‌شود. شرط `if` بررسی می‌کند که وقفه مربوط به USART1 باشد، سپس پرچم `transmit_complete_flag` را برابر ۱ قرار می‌دهد تا حلقه اصلی بداند که می‌تواند بایت بعدی را بفرستد.

در داخل `while`:

اگر پرچم آماده به ارسال برابر ۱ باشد، بلافاصله آن را ۰ می‌کنیم تا از ارسال مجدد جلوگیری شود. سپس مقدار فعلی دندان‌اره در بافر کپی شده و با وقفه ارسال می‌گردد؛ در صورت خطا، پرچم دوباره ۱ می‌شود تا تلاش مجدد انجام شود.

```

while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if (transmit_complete_flag == 1)
    {
        transmit_complete_flag = 0;
        tx_buffer_byte = current_sawtooth;
        if (HAL_UART_Transmit_IT(&huart1, &tx_buffer_byte, 1) != HAL_OK)
        {
            transmit_complete_flag = 1;
        }
    }
    current_sawtooth++;
    HAL_Delay(10);
}

```

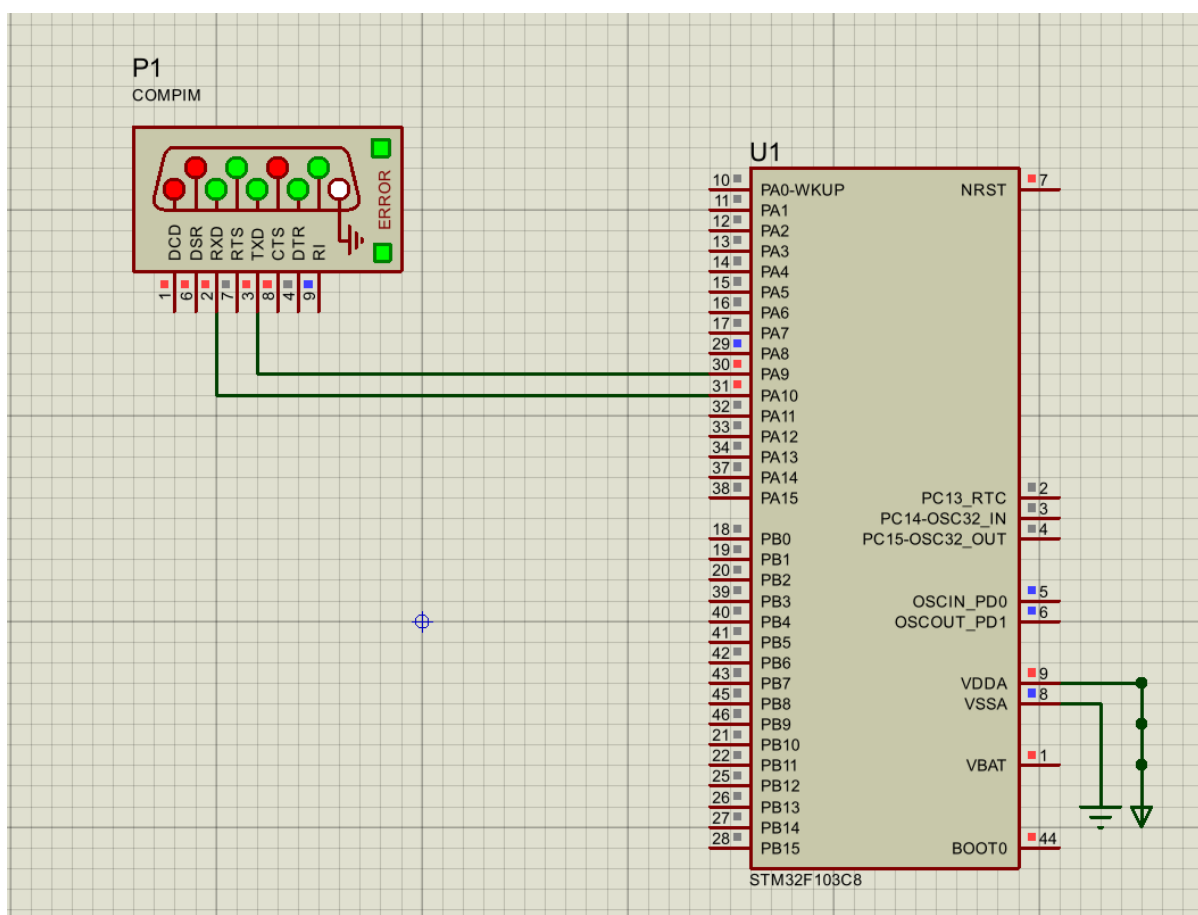
```

/* USER CODE END 3 */
}

```

مقدار دندانه‌اره یک واحد افزایش می‌یابد (به دلیل نوع uint8_t، پس از ۲۵۵ به صفر برمی‌گردد). سپس تأخیر ۱۰ میلی‌ثانیه‌ای اعمال می‌شود تا نرخ ارسال داده قابل کنترل باشد.

iii) بخش شبیه سازی Proteus



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار می‌دهیم

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

- COM1
- COM2

Ports status: Paired

User session: Global

Baudrate emulation: Disabled

Pinout: Standard

Edit Delete Break

COM1 (Virtual)

Port status: Opened

Opened by: ...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe

Current settings: 9600,N,8,1

Sent / Received: 1.4 KB / 0 bytes

COM2 (Virtual)

Port status: Opened

Opened by: C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe

Current settings: 9600,N,8,1

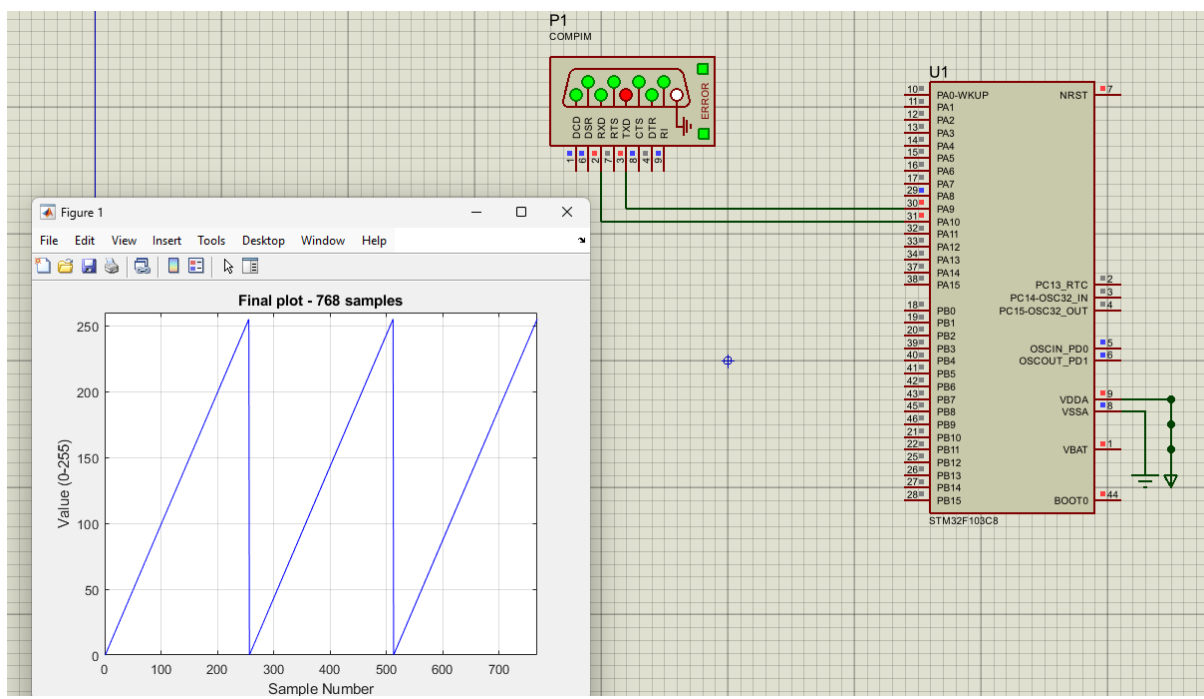
Sent / Received: 0 bytes / 1.3 KB

Edit Component [?] [X]

Part Reference:	P1	Hidden: ☐
Part Value:	COMPIM	Hidden: ☐
Element:	New	
VSM Model:	COMPIM.DLL	Hide All v
Physical port:	COM1	Hide All v
Physical Baud Rate:	9600	Hide All v
Physical Data Bits:	8	Hide All v
Physical Parity:	NONE	Hide All v
Virtual Baud Rate:	9600	Hide All v
Virtual Data Bits:	8	Hide All v
Virtual Parity:	NONE	Hide All v
Advanced Properties:		
Physical Stop Bits	1	Hide All v
Other Properties		
☐ Exclude from Simulation	☐ Attach hierarchy module	
☐ Exclude from PCB Layout	☐ Hide common pins	
☐ Exclude from Current Variant	☐ Edit all properties as text	

کد متلب:

مانند بخش کد متلب سوال 3 قسمت Polling



ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

3_DMA. شمارنده‌ای در میکرو کنترلر ایجاد کنید که اعداد از ۰ تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از ۰ تا ۲۵۵ را مجدداً بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندانه‌ای). این اعداد را در همان حین تولید در میکرو کنترلر، به کامپیوتر ارسال کرده و در نرم افزار Matlab، دریافت نموده و در یک آرایه بریزید و بعد از چند سری دریافت در نرم افزار Matlab، آن را رسم نمایید.

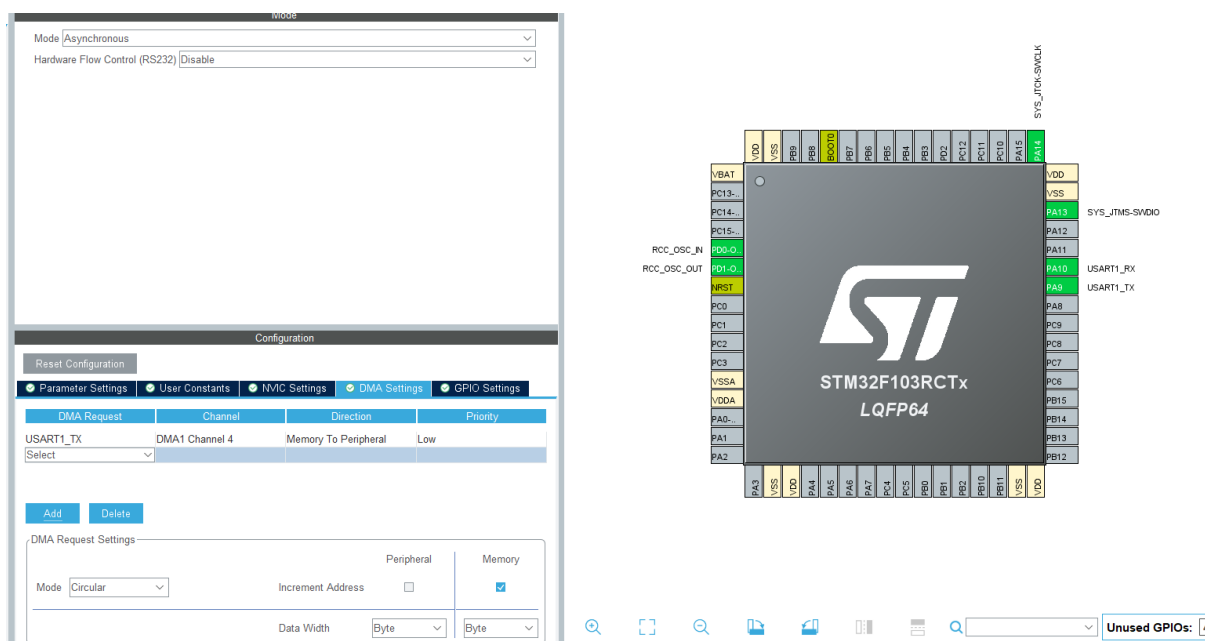
3.3 روش DMA

4 در روش DMA، داده‌ها بدون دخالت CPU و مستقیماً بین پریفرال UART و حافظه جابه‌جا می‌شوند. این روش حتی از وقفه هم کم‌بارتر است، چون CPU فقط در ابتدا و انتهای انتقال بلوک داده درگیر می‌شود، نه برای هر بایت.

(i) بخش STM32CubeMX

مثل بخش i سوال 2 قسمت DMA

با این تفاوت که نوع DMA رو TX می‌زاریم:



(ii) بخش Keil

huart1 و hdma_usart1_tx ساختارهای پیکربندی سخت‌افزاری هستند که توسط CubeMX مقداردهی می‌شوند. sawtooth_generator مقدار جاری سیگنال دندانه‌اره (۰ تا ۲۵۵) را نگه

می‌دارد، `dma_send_buffer` بافری برای ارسال DMA (یک بایت) است، و `dma_transfer_done` که با `volatile` تعریف شده، نشان می‌دهد که انتقال قبلی به پایان رسیده و می‌توان بایت بعدی را فرستاد.

```
/* USER CODE BEGIN PM */
uint8_t sawtooth_generator = 0;
uint8_t dma_send_buffer[1];
volatile uint8_t dma_transfer_done = 1;
/* USER CODE END PM */

/* Private variables -----
*/
UART_HandleTypeDef huart1;
DMA_HandleTypeDef hdma_usart1_rx;
```

این تابع هر بار که انتقال DMA یک بایت به پایان می‌رسد، به طور خودکار توسط HAL فراخوانی می‌شود. با بررسی اینکه وقفه از USART1 آمده، پرچم `dma_transfer_done` را برابر ۱ قرار می‌دهد تا حلقه اصلی بداند بافر آزاد شده و می‌تواند بایت بعدی را برای ارسال آماده کند.

```
/* USER CODE BEGIN PV */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        dma_transfer_done = 1;
    }
}
```

در داخل `while`:

اگر پرچم پایان انتقال قبلی برابر ۱ باشد، آن را ۰ می‌کنیم، مقدار فعلی دندانه‌اره را در بافر DMA کپی می‌کنیم و با `HAL_UART_Transmit_DMA` ارسال را شروع می‌نماییم. در صورت بروز خطا در راه‌اندازی DMA، پرچم دوباره ۱ می‌شود تا تلاش بعدی انجام گیرد.

```
/* USER CODE BEGIN WHILE */
```

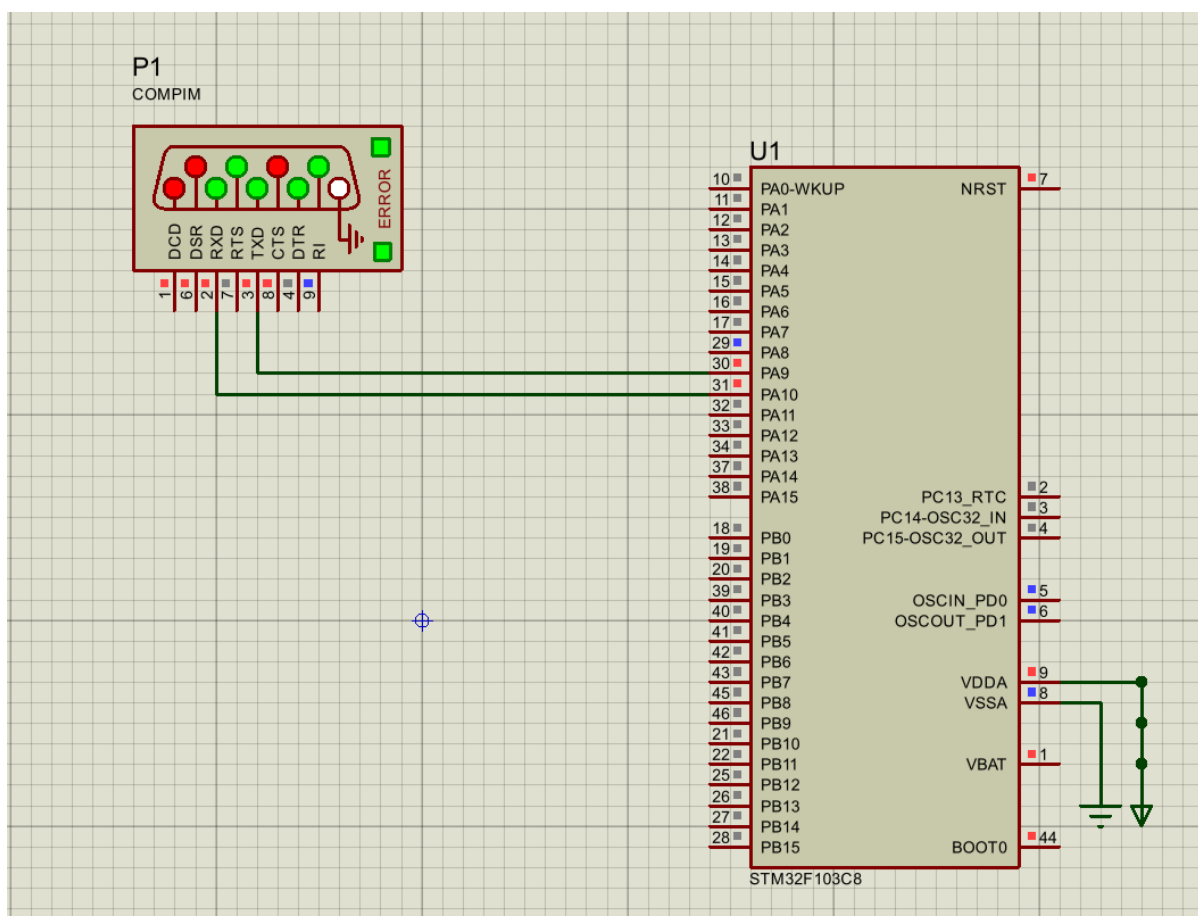
```

while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if (dma_transfer_done == 1)
    {
        dma_transfer_done = 0;
        dma_send_buffer[0] = sawtooth_generator;
        if (HAL_UART_Transmit_DMA(&huart1, dma_send_buffer, 1) != HAL_OK)
        {
            dma_transfer_done = 1;
        }
    }
    sawtooth_generator++;
    HAL_Delay(10);
}
/* USER CODE END 3 */
}

```

شمارنده دندان‌اره یک واحد افزایش می‌یابد (به دلیل نوع `uint8_t`، پس از ۲۵۵ به صفر برمی‌گردد). تأخیر ۱۰ میلی‌ثانیه‌ای نرخ ارسال داده را کنترل می‌کند و از اشباع شدن بافرهای DMA جلوگیری می‌نماید.



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

COM1

COM2

Edit

Delete

Break

Ports status: Paired
User session: Global
Baudrate emulation: Disabled
Pinout: Standard

COM1 (Virtual)

Port status: Opened
Opened by: ...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe
Current settings: 9600,N,8,1
Sent / Received: 1.4 KB / 0 bytes

COM2 (Virtual)

Port status: Opened
Opened by: C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe
Current settings: 9600,N,8,1
Sent / Received: 0 bytes / 1.3 KB

Edit Component

Part Reference: Hidden: ☐

Part Value: Hidden: ☐

Element:

VSM Model:

Physical port:

Physical Baud Rate:

Physical Data Bits:

Physical Parity:

Virtual Baud Rate:

Virtual Data Bits:

Virtual Parity:

Advanced Properties:

Physical Stop Bits

Other Properties

☐ Exclude from Simulation ☐ Attach hierarchy module

☐ Exclude from PCB Layout ☐ Hide common pins

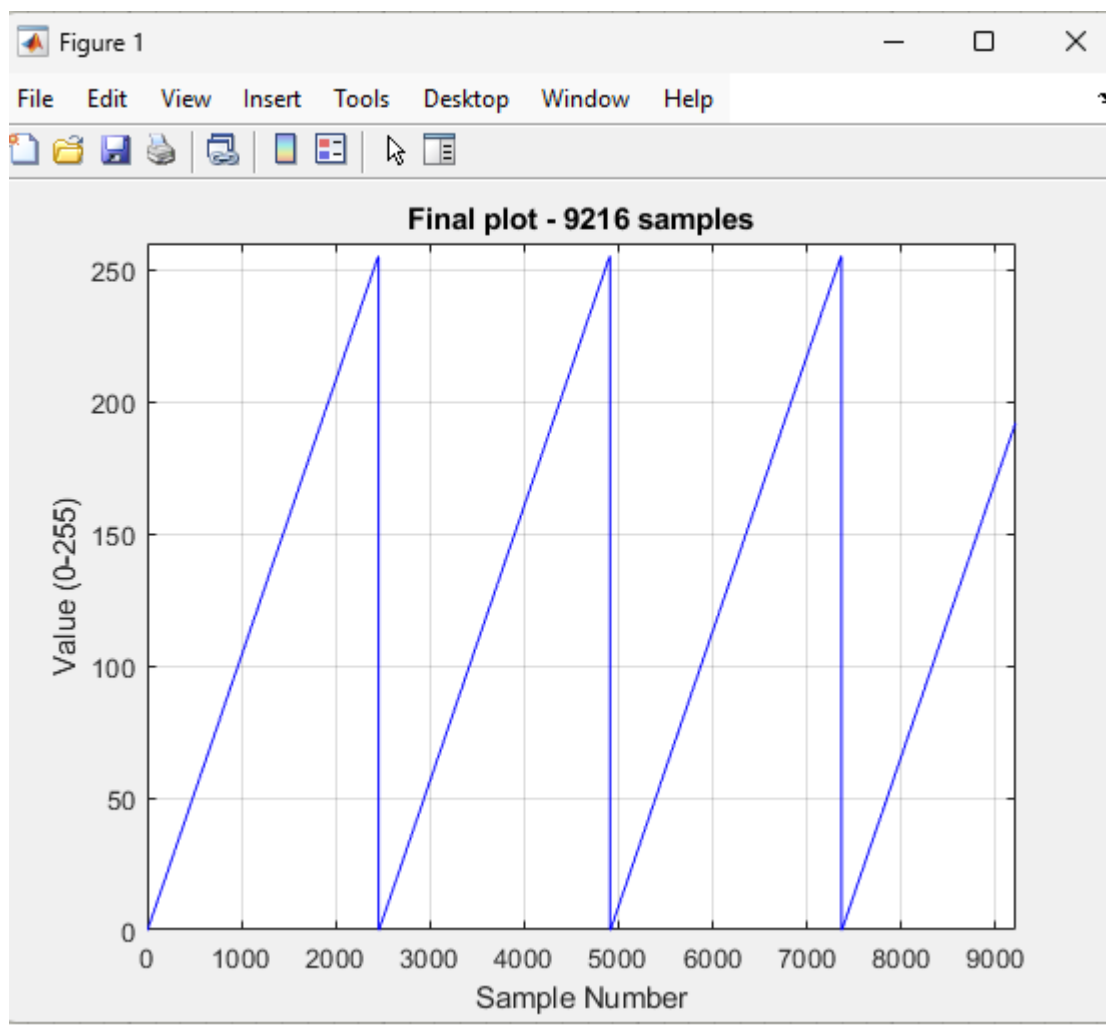
☐ Exclude from Current Variant ☐ Edit all properties as text

کد متلب:

درست مثل قسمت قبل

با این تفاوت که این بار باین تنظیمات داده هارو میگیریم (به علت سرعت بالای DMA):

```
SAMPLES_PER_CYCLE = 1024;
CYCLES_TO_GET = 9;
```



ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

4_Polling. شمارنده ای در کامپیوتر در نرم افزار Matlab ایجاد کنید که اعداد از 0 تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از ۰ تا ۲۵۵ را مجدداً بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندانه اری). این اعداد را در همان حین تولید در نرم افزار Matlab، به میکروکنترلر ارسال کرده و از طریق میکروکنترلر روی LCD، نمایش دهید. (سرعت ارسال و نمایش اعداد روی LCD باید طوری باشد که با چشم قابل تشخیص باشد). دقت: دریافت داده در میکروکنترلر را با ۳ روش، Polling، وقفه و DMA، انجام دهید.

4.1 روش Poling

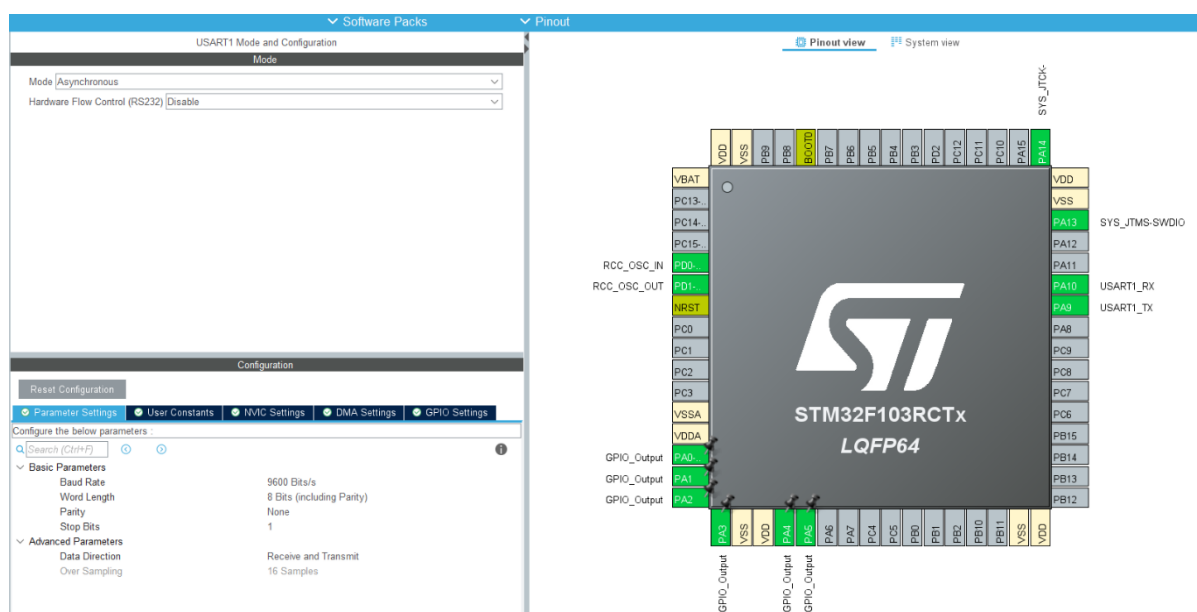
در این روش CPU به طور مداوم وضعیت رجیستر UART را بررسی می کند.

(بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم و پین های 0 تا 5 پورت A را به صورت GPIO Output انتخاب میکنیم.



بخش Keil (ii)

فایل هدر lcd.h به پروژه اضافه می‌شود تا توابع و تعاریف مربوط به نمایشگر LCD در دسترس باشند. این کار برای استفاده از کتابخانه LCD که قبلاً نوشته شده ضروری است.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
/* USER CODE END Includes */
```

پورت و پین‌های میکروکنترلر که به پایه‌های داده (D4 تا D7) و پایه‌های کنترلی (RS و EN) LCD متصل می‌شوند، با دستورات #define مشخص می‌گردند.

```
/* USER CODE BEGIN PTD */
#define LCD_D4_PORT GPIOA
#define LCD_D4_PIN GPIO_PIN_0
#define LCD_D5_PORT GPIOA
#define LCD_D5_PIN GPIO_PIN_1
#define LCD_D6_PORT GPIOA
#define LCD_D6_PIN GPIO_PIN_2
#define LCD_D7_PORT GPIOA
#define LCD_D7_PIN GPIO_PIN_3

#define LCD_RS_PORT GPIOA
#define LCD_RS_PIN GPIO_PIN_4
#define LCD_EN_PORT GPIOA
#define LCD_EN_PIN GPIO_PIN_5
/* USER CODE END PTD */
```

incoming_data مقدار بایتی را که از پورت سریال دریافت می‌شود نگه می‌دارد (۰ تا ۲۵۵).
آرایه display_text برای ساخت رشته متنی که روی LCD نمایش داده می‌شود (مثل "Value: 123") در نظر گرفته شده است.

```
/* USER CODE BEGIN PV */
uint8_t incoming_data = 0;
char display_text[16];
/* USER CODE END PV */
```

ابتدا آرایه‌های پورت و پین داده‌ها و سپس ساختار `Lcd_HandleTypeDef` پر می‌شود و با `Lcd_create` راه‌اندازی LCD در حالت ۴ بیتی انجام می‌گیرد.

در ادامه صفحه LCD پاک شده، مکان‌نما به سطر اول (۰) و ستون اول (۰) رفته و متن

"Mode: POLLING" چاپ می‌شود تا روش ارتباط (پولینگ) اعلام گردد.

```
/* USER CODE BEGIN 2 */

Lcd_PortType data_ports[] = {LCD_D4_PORT, LCD_D5_PORT, LCD_D6_PORT,
LCD_D7_PORT};
Lcd_PinType data_pins[] = {LCD_D4_PIN, LCD_D5_PIN, LCD_D6_PIN, LCD_D7_PIN};

Lcd_HandleTypeDef mylcd;
mylcd = Lcd_create(data_ports, data_pins, LCD_RS_PORT, LCD_RS_PIN,
LCD_EN_PORT, LCD_EN_PIN, LCD_4_BIT_MODE);

Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "Mode: POLLING");
/* USER CODE END 2 */
```

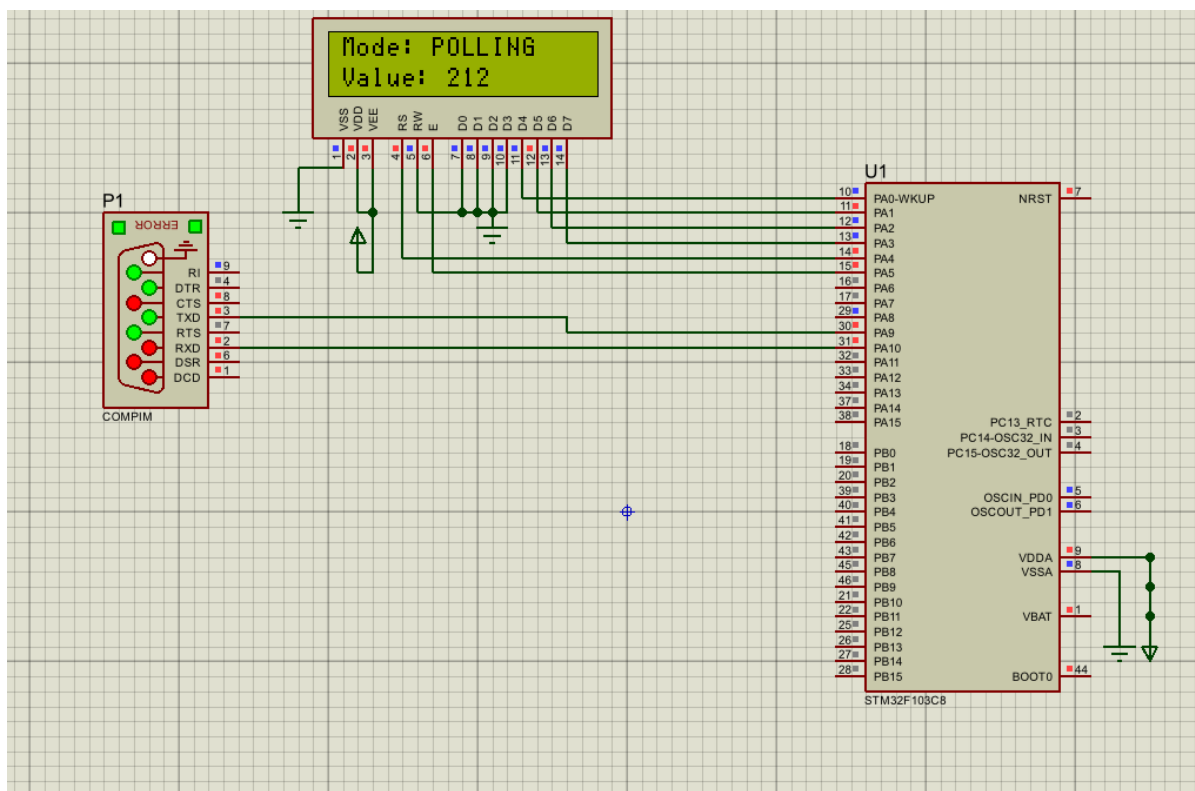
هر بار که یک بایت از پورت سریال (با زمان انتظار ۵۰ میلی‌ثانیه) دریافت شود، مکان‌نما به سطر دوم و ستون اول رفته و مقدار دریافت شده در قالب یک عدد سه رقمی در `display_text` قرار می‌گیرد. سپس این رشته روی LCD نمایش داده می‌شود. حلقه بی‌نهایت این فرایند را تکرار می‌کند و در صورت عدم دریافت داده، کاری انجام نمی‌دهد.

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if (HAL_UART_Receive(&huart1, &incoming_data, 1, 50) == HAL_OK)
    {
        Lcd_cursor(&mylcd, 1, 0);

        sprintf(display_text, "Value: %3d ", incoming_data);
        Lcd_string(&mylcd, display_text);
    }
}
/* USER CODE END 3 */
}
```

iii) بخش شبیه سازی Proteus



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

LM016L LCD را بر بروی برد میزاریم و مطابق شکل پیکربندی می کنیم.

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

- COM1
- COM2

Ports status: Paired

User session: Global

Baudrate emulation: Disabled

Pinout: Standard

Edit Delete Break

COM1 (Virtual)

Port status: Opened

Opened by: ...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe

Current settings: 9600,N,8,1

Sent / Received: 1.4 KB / 0 bytes

COM2 (Virtual)

Port status: Opened

Opened by: C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe

Current settings: 9600,N,8,1

Sent / Received: 0 bytes / 1.3 KB

Edit Component

Part Reference: Hidden: ☐

Part Value: Hidden: ☐

Element:

VSM Model:

Physical port:

Physical Baud Rate:

Physical Data Bits:

Physical Parity:

Virtual Baud Rate:

Virtual Data Bits:

Virtual Parity:

Advanced Properties:

Other Properties

☐ Exclude from Simulation ☐ Attach hierarchy module

☐ Exclude from PCB Layout ☐ Hide common pins

☐ Exclude from Current Variant ☐ Edit all properties as text

کد متلب:

```
clear; clc;

baud_rate = 9600;
stm32_port = serialport("COM2", baud_rate);

disp('Starting transmission. Press Ctrl+C to stop.');
```

```
while true
    for val = 0:255
        write(stm32_port, val, "uint8");
        pause(0.1);
    end
end
```

```

%{
//for making triangle shape we can add this to the code!
for val = 254:-1:1
    write(stm32_port, val, "uint8");
    pause(0.1);
end
%}
end

```

برای تولید موجی که از 255 به 0 مستقیم نرود و یکی یکی کم شود میتوانیم این بخش از کد را از حالت کامنت در بیاوریم.

```

//for making triangle shape we can add this to the code!
for val = 254:-1:1
    write(stm32_port, val, "uint8");
    pause(0.1);
end
%}

```

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

4_IT. شمارنده ای در کامپیوتر در نرم افزار Matlab ایجاد کنید که اعداد از 0 تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از ۲۵۵ تا ۰ را مجدد بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندانان اراهای). این اعداد را در همان حین تولید در نرم افزار Matlab، به میکروکنترلر ارسال کرده واز طریق میکروکنترلر روی LCD، نمایش دهید. (سرعت ارسال و نمایش اعداد روی LCD باید طوری باشد که با چشم قابل تشخیص باشد). دقت: دریافت داده در میکروکنترلر را با ۳ روش، Polling، وقفه و DMA، انجام دهید.

4.2 روش IT

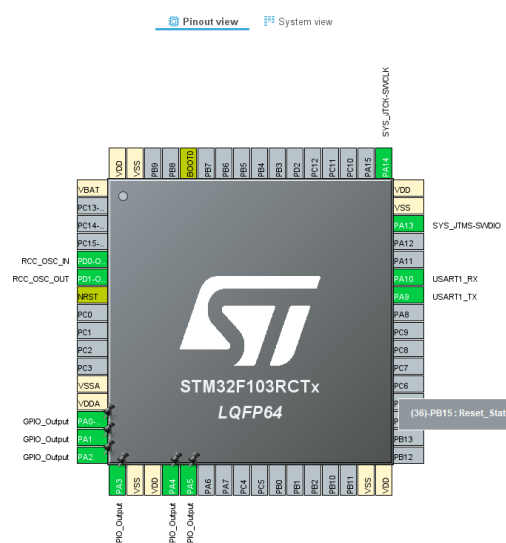
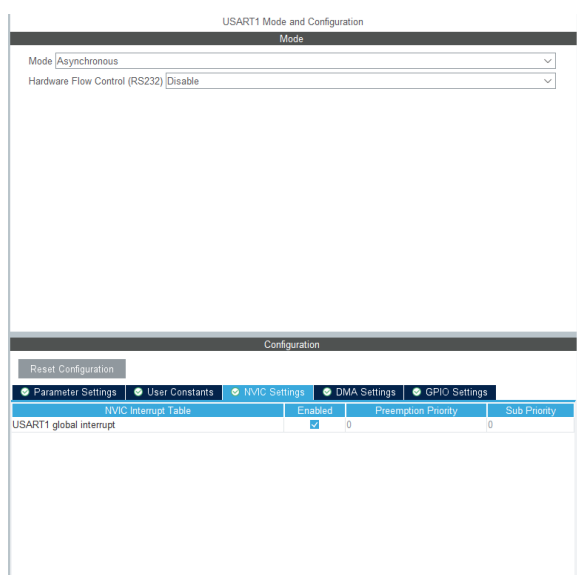
در روش اینترایت، میکروکنترلر بی آنکه مدام چک کند (پولینگ)، فقط وقتی داده می رسد وقفه ایجاد می شود و CPU در بقیه زمان آزاد است و درگیر نمی شود.

(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم و تیک اینترایت را میزنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم و پین های 0 تا 5 پورت A را GPIO OUTPUT انتخاب میکنیم



iii) بخش Keil

فایل هدر lcd.h به پروژه اضافه می‌شود تا توابع و تعاریف مربوط به نمایشگر LCD در دسترس باشند. این کار برای استفاده از کتابخانه LCD که قبلاً نوشته شده ضروری است.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
/* USER CODE END Includes */
```

پورت و پین‌های میکروکنترلر که به پایه‌های داده (D4 تا D7) و پایه‌های کنترلی (RS و EN) LCD متصل می‌شوند، با دستورات #define مشخص می‌گردند.

```
/* USER CODE BEGIN PTD */
#define LCD_D4_PORT GPIOA
#define LCD_D4_PIN GPIO_PIN_0
#define LCD_D5_PORT GPIOA
#define LCD_D5_PIN GPIO_PIN_1
#define LCD_D6_PORT GPIOA
#define LCD_D6_PIN GPIO_PIN_2
#define LCD_D7_PORT GPIOA
#define LCD_D7_PIN GPIO_PIN_3

#define LCD_RS_PORT GPIOA
#define LCD_RS_PIN GPIO_PIN_4
#define LCD_EN_PORT GPIOA
#define LCD_EN_PIN GPIO_PIN_5
/* USER CODE END PTD */
```

rx_byte مقدار بایتی را که از پورت سریال دریافت می‌شود ذخیره می‌کند. پرچم data_ready با کلمه کلیدی volatile تعریف شده تا به کامپایلر بگوید این متغیر در تابع وقفه تغییر می‌کند و هر بار باید از حافظه خوانده شود.

آرایه lcd_buffer برای ساخت رشته متنی (مانند "Val: 123") که روی LCD نمایش داده می‌شود، استفاده می‌گردد.

```

/* Private variables -----
*/
UART_HandleTypeDef huart1;

/* USER CODE BEGIN PV */
Lcd_HandleTypeDef mylcd;
uint8_t rx_byte = 0;
volatile uint8_t data_ready = 0;
char lcd_buffer[16];
/* USER CODE END PV */

```

این تابع به صورت خودکار هر بار که یک بایت به طور کامل از طریق UART دریافت می‌شود، فراخوانی می‌شود. در آن پرچم data_ready را برابر ۱ قرار می‌دهیم تا حلقه اصلی بداند داده جدید آماده است و سپس وقفه را دوباره فعال می‌کنیم (HAL_UART_Receive_IT) تا بایت بعدی نیز قابل دریافت باشد.

```

/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    // Verify the interrupt came from the correct UART port (USART1)
    if (huart->Instance == USART1)
    {
        data_ready = 1; // Set the flag so the main loop knows data arrived

        // CRITICAL: In STM32 HAL, interrupts turn off after receiving.
        // We must re-arm the interrupt to catch the next number from MATLAB!
        HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
    }
}
/* USER CODE END 0 */

```

ابتدا آرایه‌های پورت و پین داده‌ها ساخته شده، سپس نمایشگر LCD با تابع Lcd_create مقداردهی اولیه می‌شود و پیام "Mode: INTERRUPT" در سطر اول نمایش داده می‌شود. در انتها با HAL_UART_Receive_IT اولین وقفه دریافت راه‌اندازی می‌شود تا میکروکنترلر بدون اشغال شدن CPU منتظر بایت اول از MATLAB بماند.

```

/* USER CODE BEGIN 2 */
Lcd_PortType lcd_ports[] = {LCD_D4_PORT, LCD_D5_PORT, LCD_D6_PORT,
LCD_D7_PORT};
Lcd_PinType lcd_pins[] = {LCD_D4_PIN, LCD_D5_PIN, LCD_D6_PIN, LCD_D7_PIN};

```

```
// Initialize the LCD
mylcd = Lcd_create(lcd_ports, lcd_pins, LCD_RS_PORT, LCD_RS_PIN, LCD_EN_PORT,
LCD_EN_PIN, LCD_4_BIT_MODE);

Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "Mode: INTERRUPT");

// Arm the UART to listen for the VERY FIRST byte from MATLAB
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
/* USER CODE END 2 */
```

در حلقه بی‌نهایت، مدام پرچم `data_ready` چک می‌شود. اگر برابر ۱ باشد، یعنی داده جدید از MATLAB رسیده است. پس از پاک کردن پرچم، مکان‌نما به سطر دوم رفته و مقدار دریافت شده با قالب سه رقمی مثلاً `Val: 005` روی LCD نمایش داده می‌شود.

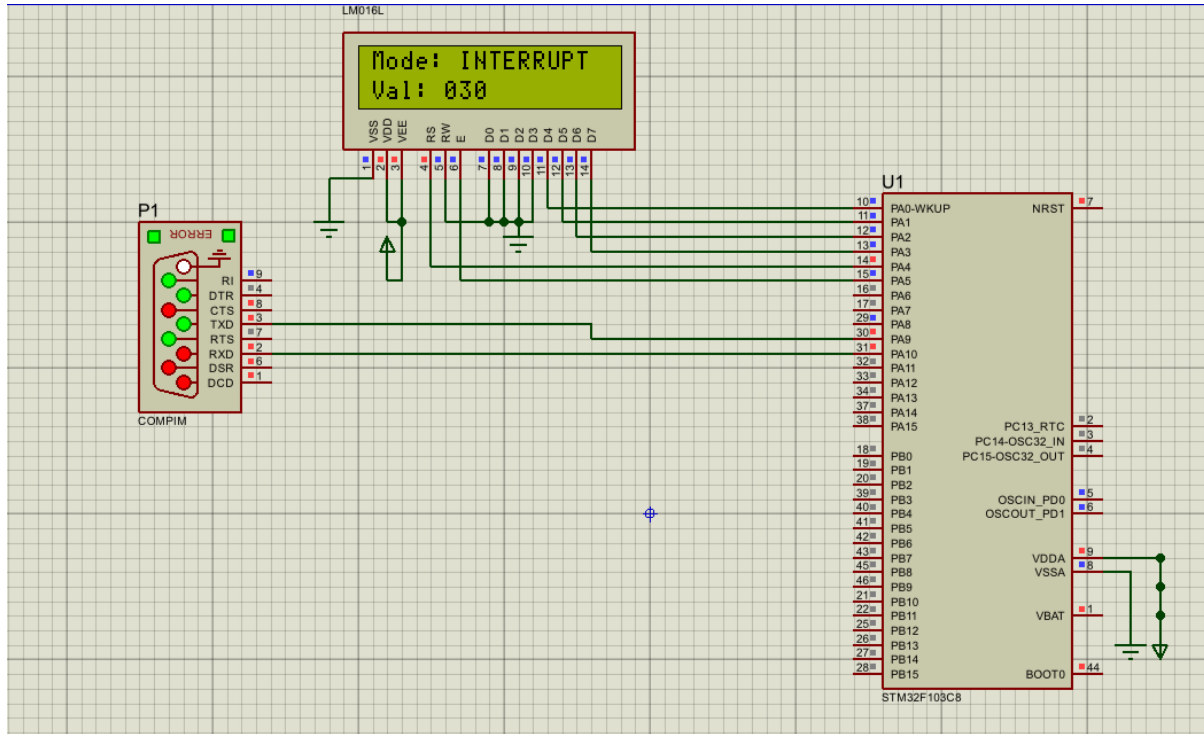
در زمان عدم دریافت داده، CPU کاملاً آزاد است و می‌تواند کارهای دیگر (مثل خواندن سنسور) را انجام دهد – این مزیت اصلی روش وقفه نسبت به پولینگ است

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    if (data_ready == 1)
    {
        data_ready = 0; // Lower the flag immediately

        Lcd_cursor(&mylcd, 1, 0);

        // Format the received byte (0-255) with leading zeros if desired
        // %03d means 5 becomes "005", keeping the LCD characters perfectly
aligned
        sprintf(lcd_buffer, "Val: %03d ", rx_byte);
        Lcd_string(&mylcd, lcd_buffer);
    }
}
/* USER CODE END 3 */
```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

LM016L LCD را بر بروی برد میزایم و مطابق شکل پیکربندی می کنیم.

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در آخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

COM1

COM2

Edit

Delete

Break

Ports status: Paired
User session: Global
Baudrate emulation: Disabled
Pinout: Standard

COM1 (Virtual)

Port status: Opened
Opened by: ...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe
Current settings: 9600,N,8,1
Sent / Received: 1.4 KB / 0 bytes

COM2 (Virtual)

Port status: Opened
Opened by: C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe
Current settings: 9600,N,8,1
Sent / Received: 0 bytes / 1.3 KB

Edit Component

Part Reference: Hidden: ☐

Part Value: Hidden: ☐

Element:

VSM Model:

Physical port:

Physical Baud Rate:

Physical Data Bits:

Physical Parity:

Virtual Baud Rate:

Virtual Data Bits:

Virtual Parity:

Advanced Properties:

Other Properties

☐ Exclude from Simulation ☐ Attach hierarchy module

☐ Exclude from PCB Layout ☐ Hide common pins

☐ Exclude from Current Variant ☐ Edit all properties as text

کد متلب:

```
clear; clc;

baud_rate = 9600;
stm32_port = serialport("COM2", baud_rate);

disp('Starting transmission. Press Ctrl+C to stop.');
```

```
while true
    for val = 0:255
        write(stm32_port, val, "uint8");
        pause(0.1);
    end
end
```

```

%{

//for making triangle shape we can add this to the code!
for val = 254:-1:1
    write(stm32_port, val, "uint8");
    pause(0.1);
end
%}
end

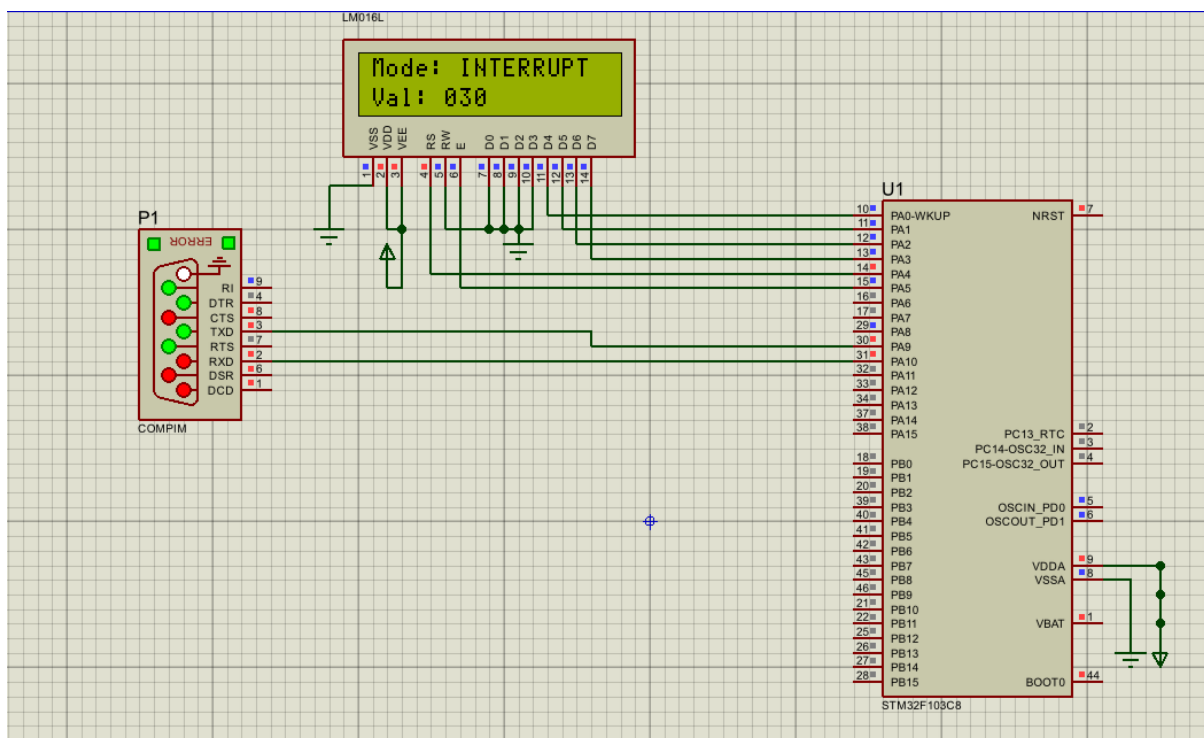
```

برای تولید موجی که از 255 به 0 مستقیم نرود و یکی یکی کم شود میتوانیم این بخش از کد را از حالت کامنت در بیاوریم.

```

//for making triangle shape we can add this to the code!
for val = 254:-1:1
    write(stm32_port, val, "uint8");
    pause(0.1);
end
%}

```



ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

4_DMA. شمارنده ای در کامپیوتر در نرم افزار Matlab ایجاد کنید که اعداد از 0 تا ۲۵۵ را بصورت متوالی تولید کند وقتی به ۲۵۵ رسید از ۲۵۵ تا ۰ را مجدد بصورت متوالی تولید نماید و این کار را بصورت متوالی پشت سر هم انجام دهد (تولید سیگنال دندان‌اره‌ای). این اعداد را در همان حین تولید در نرم‌افزار Matlab، به میکروکنترلر ارسال کرده و از طریق میکروکنترلر روی LCD، نمایش دهید. (سرعت ارسال و نمایش اعداد روی LCD باید طوری باشد که با چشم قابل تشخیص باشد). دقت: دریافت داده در میکروکنترلر را با ۳ روش، Polling، وقفه و DMA، انجام دهید.

4.3 روش DMA

در روش DMA، داده‌ها بدون دخالت CPU و مستقیماً بین پریفرال UART و حافظه جابه‌جا می‌شوند. این روش حتی از وقفه هم کم‌بارتر است، چون CPU فقط در ابتدا و انتهای انتقال بلوک داده درگیر می‌شود، نه برای هر بایت.

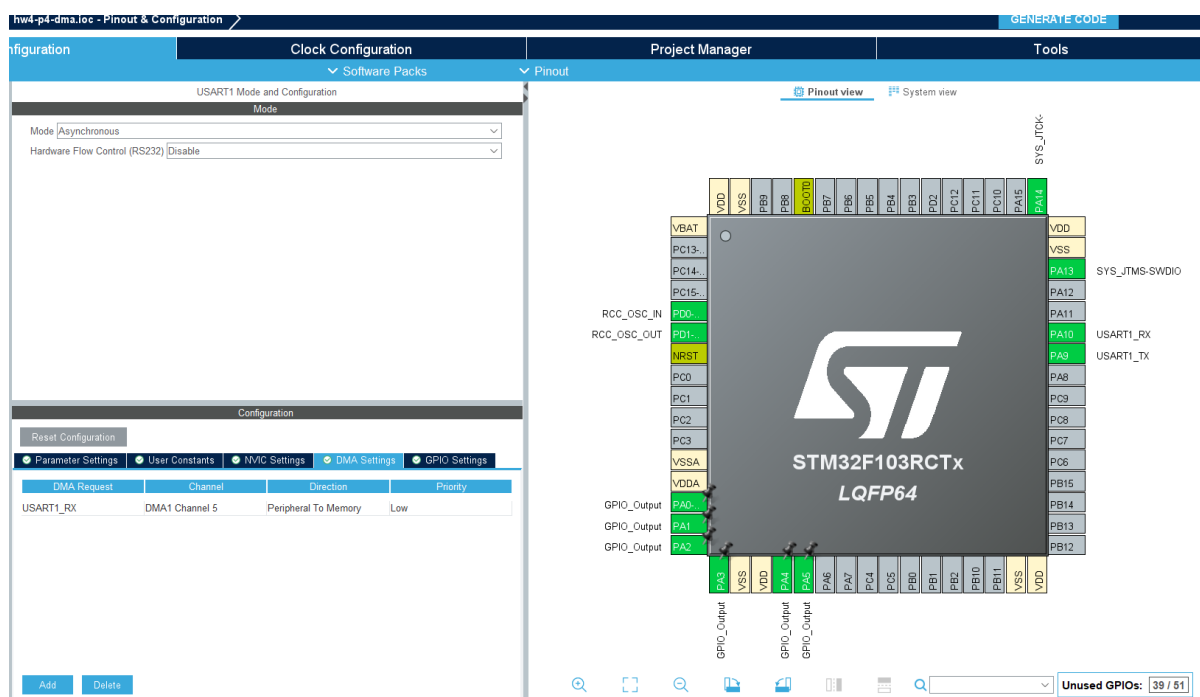
(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می‌کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم و تیک DMA را می‌زنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم و پین‌های 0 تا 5 پورت A را GPIO

OUTPUT انتخاب میکنیم



فایل هدر lcd.h به پروژه اضافه می‌شود تا توابع و تعاریف مربوط به نمایشگر LCD در دسترس باشند. این کار برای استفاده از کتابخانه LCD که قبلاً نوشته شده ضروری است.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
/* USER CODE END Includes */
```

پورت و پین‌های میکروکنترلر که به پایه‌های داده (D4 تا D7) و پایه‌های کنترلی (RS و EN) LCD متصل می‌شوند، با دستورات #define مشخص می‌گردند.

```
/* USER CODE BEGIN PTD */
#define LCD_D4_PORT GPIOA
#define LCD_D4_PIN GPIO_PIN_0
#define LCD_D5_PORT GPIOA
#define LCD_D5_PIN GPIO_PIN_1
#define LCD_D6_PORT GPIOA
#define LCD_D6_PIN GPIO_PIN_2
#define LCD_D7_PORT GPIOA
#define LCD_D7_PIN GPIO_PIN_3

#define LCD_RS_PORT GPIOA
#define LCD_RS_PIN GPIO_PIN_4
#define LCD_EN_PORT GPIOA
#define LCD_EN_PIN GPIO_PIN_5
/* USER CODE END PTD */
```

hdma_usart1_rx ساختار کنترل کننده DMA برای دریافت از UART است (نه ارسال). متغیر dma_rx_byte محلی در حافظه است که DMA مستقیماً بایت دریافتی را در آن می‌نویسد و dma_data_ready یک پرچم volatile برای اطلاع به حلقه اصلی از رسیدن داده جدید می‌باشد.

```
/* Private variables -----
*/
UART_HandleTypeDef huart1;
DMA_HandleTypeDef hdma_usart1_rx;

/* USER CODE BEGIN PV */
Lcd_HandleTypeDef mylcd;
uint8_t dma_rx_byte;
volatile uint8_t dma_data_ready = 0;
char display_buffer[16];
```

```
/* USER CODE END PV */
```

این تابع هر بار که DMA انتقال یک بایت از UART به حافظه را به پایان برساند، فراخوانی می‌شود. در آن پرچم `dma_data_ready` را برابر ۱ قرار می‌دهیم تا حلقه اصلی بداند داده جدید آماده است. اگر در CubeMX حالت DMA را روی **Circular** تنظیم کرده باشید، DMA خودکار دوباره شروع می‌شود و نیازی به فراخوانی دستی `HAL_UART_Receive_DMA` نیست.

```
/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        // Tell the main loop that new data is sitting in the memory
        dma_data_ready = 1;

        // NOTE: If you set DMA to "Circular" in CubeMX, you DO NOT need
        // to call HAL_UART_Receive_DMA here again. It restarts automatically!
        // If you used "Normal" mode, you would need to re-arm it here.
    }
}
/* USER CODE END 0 */
```

ابتدا نمایشگر LCD مقداردهی شده و پیام "Mode: DMA RX" در سطر اول نمایش داده می‌شود. سپس با `HAL_UART_Receive_DMA` سخت‌افزار DMA را برای دریافت یک بایت از UART و نوشتن مستقیم در آدرس `dma_rx_byte` راه‌اندازی می‌کنیم – این کار فقط یک بار انجام می‌شود و DMA در پس‌زمینه بدون دخالت CPU کار می‌کند.

```
/* USER CODE BEGIN 2 */
Lcd_PortType lcd_ports[] = {LCD_D4_PORT, LCD_D5_PORT, LCD_D6_PORT,
LCD_D7_PORT};
Lcd_PinType lcd_pins[] = {LCD_D4_PIN, LCD_D5_PIN, LCD_D6_PIN, LCD_D7_PIN};

mylcd = Lcd_create(lcd_ports, lcd_pins, LCD_RS_PORT, LCD_RS_PIN, LCD_EN_PORT,
LCD_EN_PIN, LCD_4_BIT_MODE);
```

```

Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "Mode: DMA RX");

HAL_UART_Receive_DMA(&huart1, &dma_rx_byte, 1);
/* USER CODE END 2 */

```

در حلقه بی‌نهایت، مرتباً پرچم `dma_data_ready` بررسی می‌شود. وقتی DMA داده جدیدی دریافت کند و پرچم توسط `callback` برابر ۱ شود، پرچم پاک شده، مقدار دریافت شده (با قالب سه رقمی) در سطر دوم LCD نمایش داده می‌شود.

در این روش، CPU کاملاً آزاد است و حتی می‌تواند پردازش‌های سنگین دیگر را انجام دهد، زیرا انتقال داده توسط DMA و بدون دخالت پردازنده انجام می‌شود.

```

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

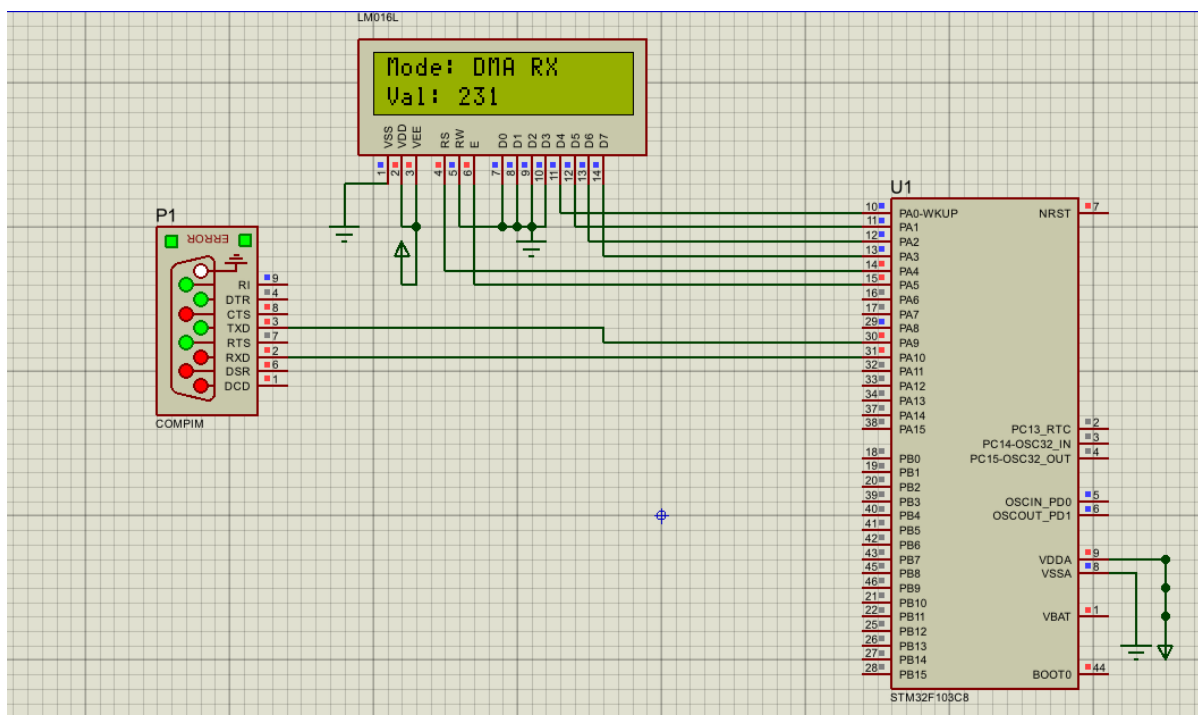
    /* USER CODE BEGIN 3 */
    if (dma_data_ready == 1)
    {
        // Lower the flag
        dma_data_ready = 0;

        // Move to the second line of the LCD
        Lcd_cursor(&mylcd, 1, 0);

        // Format the number received by the DMA controller
        sprintf(display_buffer, "Val: %03d ", dma_rx_byte);
        Lcd_string(&mylcd, display_buffer);
    }
}
/* USER CODE END 3 */
}

```

بخش شبیه سازی Proteus



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

LM016L LCD را بر بروی برد میزاریم و مطابق شکل پیکربندی می کنیم.

COMPIM را در محیط Proteus قرار می دهیم و پین های Rx و Tx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

COM1
COM2

 Edit  Delete

 Break

Ports status:	Paired
User session:	Global
Baudrate emulation:	Disabled
Pinout:	Standard

COM1 (Virtual)

Port status:	Opened
Opened by:	...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe 
Current settings:	9600,N,8,1
Sent / Received:	1.4 KB / 0 bytes

COM2 (Virtual)

Port status:	Opened
Opened by:	C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe 
Current settings:	9600,N,8,1
Sent / Received:	0 bytes / 1.3 KB

Edit Component

Part Reference: Hidden: ☐

Part Value: Hidden: ☐

Element:

VSM Model:

Physical port:

Physical Baud Rate:

Physical Data Bits:

Physical Parity:

Virtual Baud Rate:

Virtual Data Bits:

Virtual Parity:

Advanced Properties:

Other Properties

☐ Exclude from Simulation ☐ Attach hierarchy module

☐ Exclude from PCB Layout ☐ Hide common pins

☐ Exclude from Current Variant ☐ Edit all properties as text

کد متلب:

```
clear; clc;

baud_rate = 9600;
stm32_port = serialport("COM2", baud_rate);

disp('Starting transmission. Press Ctrl+C to stop.');
```

```
while true
    for val = 0:255
        write(stm32_port, val, "uint8");
        pause(0.1);
    end
end
```

```

%{

//for making triangle shape we can add this to the code!
for val = 254:-1:1
    write(stm32_port, val, "uint8");
    pause(0.1);
end
%}
end

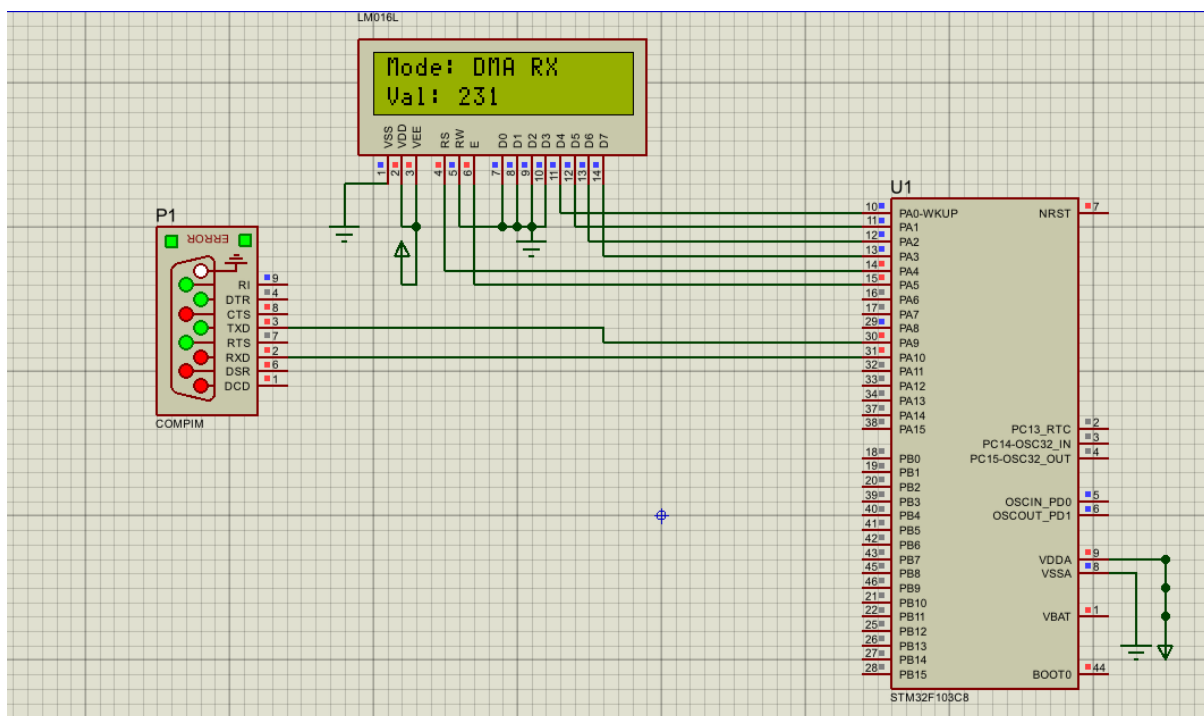
```

برای تولید موجی که از 255 به 0 مستقیم نرود و یکی یکی کم شود میتوانیم این بخش از کد را از حالت کامنت در بیاوریم.

```

//for making triangle shape we can add this to the code!
for val = 254:-1:1
    write(stm32_port, val, "uint8");
    pause(0.1);
end
%}

```



ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

5_Polling. اسم خودتان را به تعداد ۱۰ بار، از میکروکنترلر به نرم افزار Matlab، ارسال نمایید آن را در نرم افزار Matlab، دریافت و نمایش دهید.

5.1 روش Poling

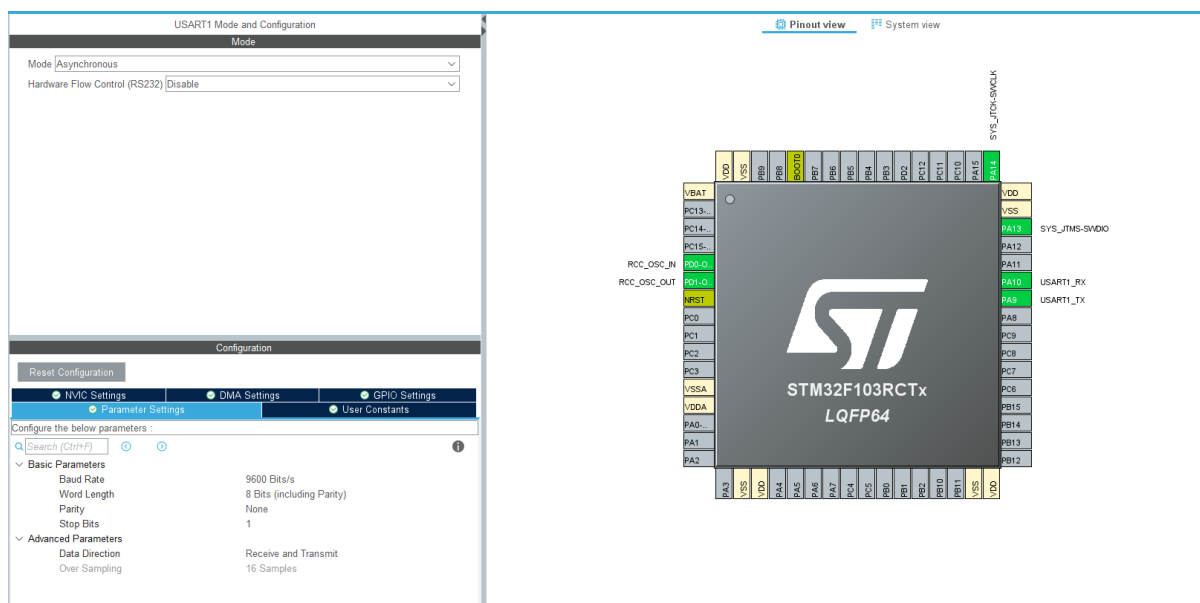
در این روش CPU به طور مداوم وضعیت رجیستر UART را بررسی می کند.

(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



فایل هدر string.h برای استفاده از تابع strlen (محاسبه طول رشته) اضافه شده است. رشته myName شامل نام کاربر به همراه یک کاراکتر خط جدید (\n) است که به عنوان مشخصه پایان پیام در متلب عمل می‌کند.

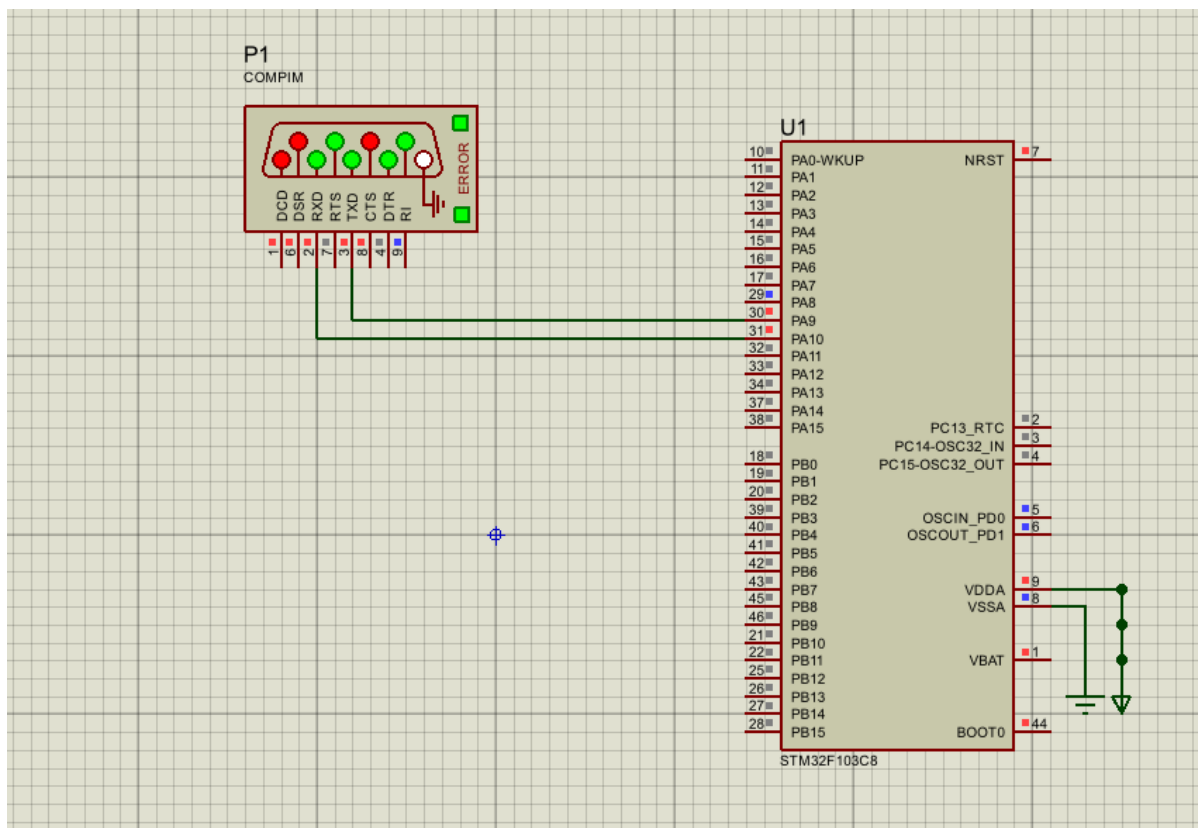
```
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */
```

ابتدا ۵۰۰ میلی‌ثانیه تأخیر ایجاد می‌شود تا پس از ریست برد، سیستم تثبیت شود. سپس در یک حلقه ۱۰ تایی، رشته نام (به همراه \n) از طریق UART ارسال گردیده و هر بار ۲۰۰ میلی‌ثانیه صبر می‌کند تا متلب فرصت پردازش داشته باشد.

```
/* USER CODE BEGIN PV */
char myName[] = "Reza Nadimi\n";
/* USER CODE END PV */
```

```
/* USER CODE BEGIN 2 */
HAL_Delay(500);
for (int i = 0; i < 10; i++)
{
    HAL_UART_Transmit(&huart1, (uint8_t*)myName, strlen(myName),
    HAL_MAX_DELAY);

    HAL_Delay(200);
}
/* USER CODE END 2 */
```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

COM1

COM2

Ports status:	Paired
User session:	Global
Baudrate emulation:	Disabled
Pinout:	Standard

 Edit  Delete

 Break

COM1 (Virtual)

Port status:	Opened
Opened by:	...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe 
Current settings:	9600,N,8,1
Sent / Received:	1.4 KB / 0 bytes

COM2 (Virtual)

Port status:	Opened
Opened by:	C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe 
Current settings:	9600,N,8,1
Sent / Received:	0 bytes / 1.3 KB

Edit Component

Part Reference: Hidden: ☐

Part Value: Hidden: ☐

Element:

VSM Model:

Physical port:

Physical Baud Rate:

Physical Data Bits:

Physical Parity:

Virtual Baud Rate:

Virtual Data Bits:

Virtual Parity:

Advanced Properties:

Other Properties

☐ Exclude from Simulation ☐ Attach hierarchy module

☐ Exclude from PCB Layout ☐ Hide common pins

☐ Exclude from Current Variant ☐ Edit all properties as text

کد متلب:

```
clear; clc;
```

```
baud_rate = 9600;
stm32_port = serialport("COM2", baud_rate);
configureTerminator(stm32_port, "LF");
```

دستورات `clear` و `clc` حافظه و صفحه فرمان را پاک می‌کنند. پورت سریال با نرخ 9600 روی COM2 باز شده و پایان خط (terminator) به خط جدید (LF) تنظیم می‌شود تا متلب بداند هر پیام کجا تمام می‌شود.

```
stm32_port.Timeout = 30;
```

```
disp('Waiting for data from the microcontroller...');
disp('Press the RESET button on your STM32 board now!');
```

```
disp('-----');
flush(stm32_port);
```

زمان انتظار برای دریافت داده به ۳۰ ثانیه افزایش یافته تا کاربر فرصت فشردن دکمه Reset روی برد STM32 را داشته باشد. `flush(stm32_port)` بافر ورودی را پاک می‌کند تا نویزهای احتمالی ناشی از ریست شدن برد نادیده گرفته شوند.

```
for count = 1:10
    received_text = readline(stm32_port);
    fprintf('Message %d: %s\n', count, received_text);
end
```

در یک حلقه ۱۰ تایی، هر بار تابع `readline` یک خط کامل (تا رسیدن به کاراکتر خط جدید) را از پورت خوانده و در متغیر `received_text` ذخیره می‌کند. سپس شماره پیام و متن دریافت شده در صفحه فرمان چاپ می‌شود.

```
disp('-----');
disp('Successfully received all 10 messages.');
```

پس از دریافت موفق ۱۰ پیام، یک پیام تأیید در خروجی نمایش داده می‌شود. (پورت سریال به طور خودکار با خروج از برنامه بسته می‌شود.)

```
Press the RESET button on your STM32 board now!
```

```
-----
Message 1: Reza Nadimi
Message 2: Reza Nadimi
Message 3: Reza Nadimi
Message 4: Reza Nadimi
Message 5: Reza Nadimi
Message 6: Reza Nadimi
Message 7: Reza Nadimi
Message 8: Reza Nadimi
Message 9: Reza Nadimi
Message 10: Reza Nadimi
-----
```

```
Successfully received all 10 messages.
```

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

IT_5. اسم خودتان را به تعداد ۱۰ بار، از میکروکنترلر به نرمافزار Matlab، ارسال نمایید
آن را در نرمافزار Matlab، دریافت و نمایش دهید.

5.2 روش IT

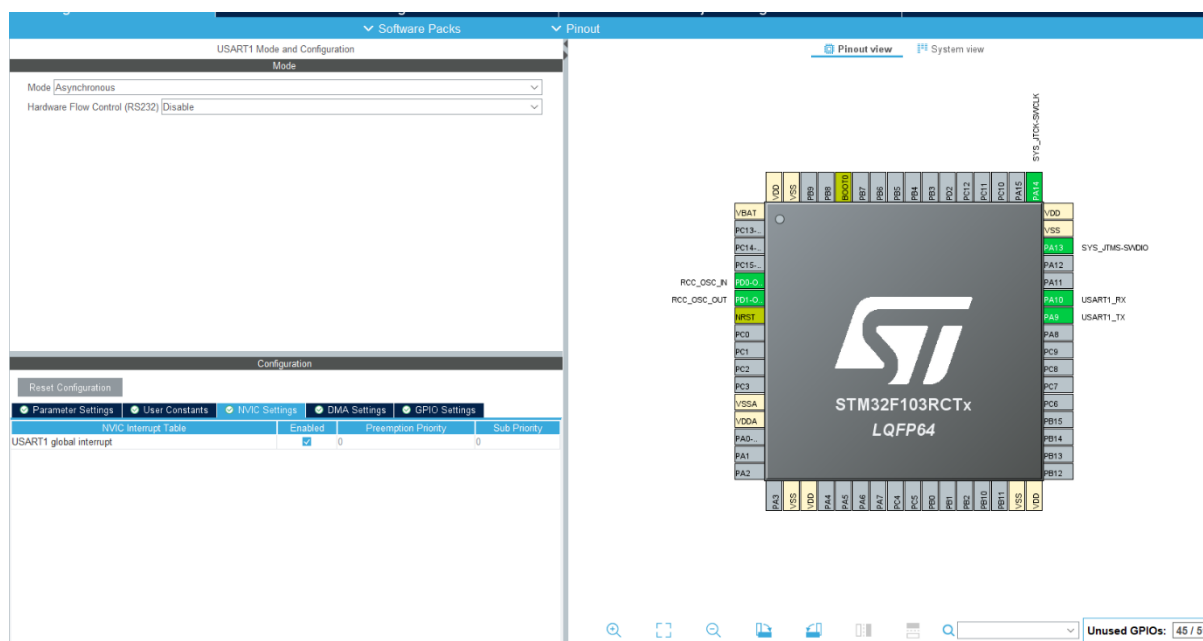
در روش اینترپت، میکروکنترلر بی‌آنکه مدام چک کند (پولینگ)، فقط وقتی داده می‌رسد وقفه ایجاد می‌شود و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

بخش (i) STM32CubeMX

میکرو کنترلر STM32F103RCT و در نرم افزار انتخاب میکنیم.

UART1 رو فعال می کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم و تیک اینترپرت را میزنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم



(iii) بخش Keil

رشته myName شامل نام کاربر و یک کاراکتر خط جدید (\n) است تا متلب بتواند پایان هر پیام را تشخیص دهد. پرچم tx_ready از نوع volatile با مقدار اولیه ۱ نشان می‌دهد که فرستنده UART آماده ارسال بایت بعدی است.

```
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
char myName[] = "Reza Nadimi\n";

// Flag to track if UART is busy or free.
// 1 = Ready to send, 0 = Busy sending
volatile uint8_t tx_ready = 1;
/* USER CODE END PV */
```

این تابع به صورت خودکار هر بار که ارسال یک بسته (در اینجا کل رشته) به صورت وقفه‌ای به پایان می‌رسد، فراخوانی می‌شود. در آن بررسی می‌کند که وقفه مربوط به USART1 باشد و سپس پرچم tx_ready را برابر ۱ قرار می‌دهد تا حلقه اصلی بداند سخت‌افزار آزاد است و می‌تواند پیام بعدی را ارسال کند.

```
/* USER CODE BEGIN 0 */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        tx_ready = 1;
    }
}
/* USER CODE END 0 */
```

پس از یک تأخیر ۱ ثانیه‌ای برای تثبیت پورت USB، حلقه ۱۰ بار اجرا می‌شود. در هر تکرار ابتدا پرچم tx_ready را ۰ می‌کنیم، سپس ارسال رشته را به صورت وقفه آغاز می‌کنیم و در یک حلقه while منتظر می‌مانیم تا callback پرچم را دوباره ۱ کند (یعنی ارسال کامل شود). در نهایت ۲۰۰ میلی‌ثانیه تأخیر ایجاد می‌شود تا متلب فرصت پردازش پیام را داشته باشد.

```
/* USER CODE BEGIN 2 */
HAL_Delay(1000);

for (int i = 0; i < 10; i++)
```

```

{
    tx_ready = 0;

    HAL_UART_Transmit_IT(&huart1, (uint8_t*)myName, strlen(myName));

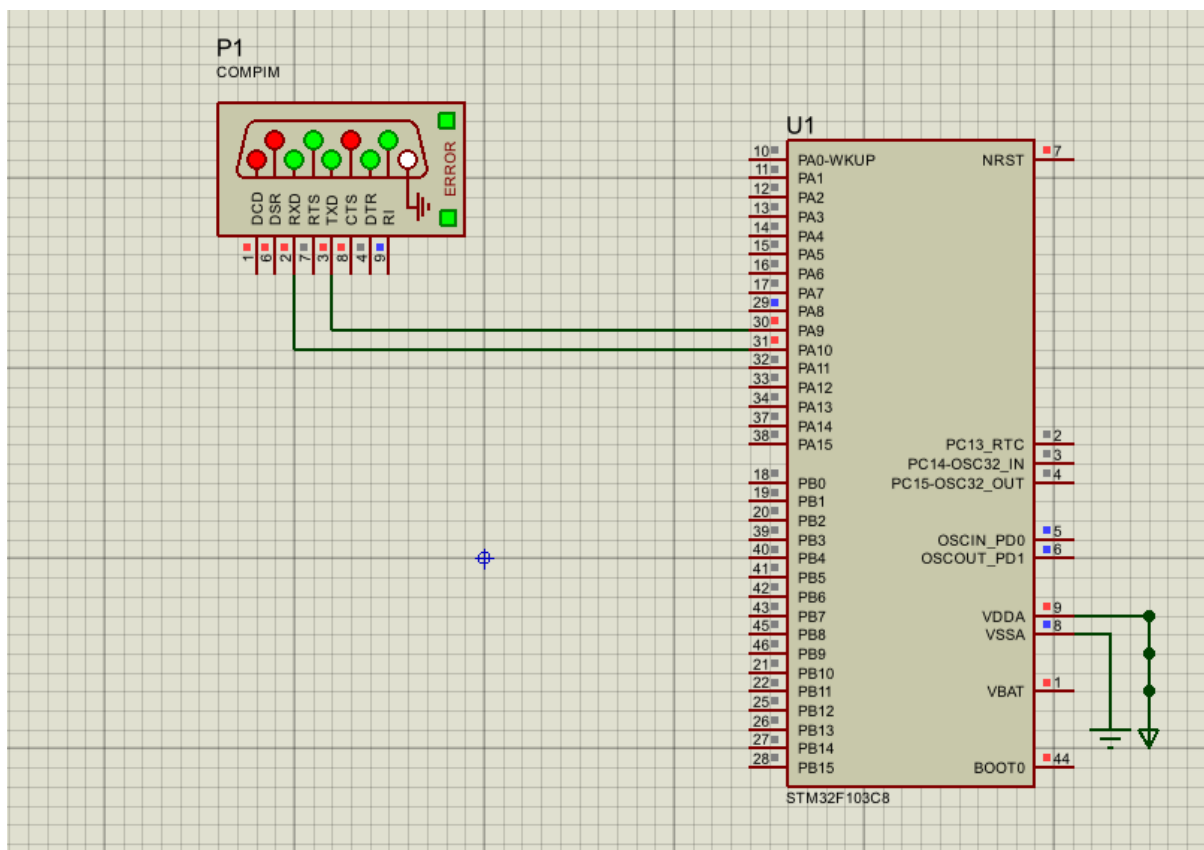
    while (tx_ready == 0)
    {
    }

    HAL_Delay(200);
}

/* USER CODE END 2 */

```

(iii) بخش شبیه سازی Proteus



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

The screenshot shows the 'Virtual Serial Port Manager' application. On the left, under 'ALL BUNDLES', there is a section 'Paired (1)' containing two entries: 'COM1' and 'COM2'. The main area displays the configuration for these ports. At the top right, there are buttons for 'Edit', 'Delete', and 'Break'. Below this, a table-like structure shows the status and settings for each port.

Port	Status	User session	Baudrate emulation	Pinout
COM1 (Virtual)	Opened	Global	Disabled	Standard
COM2 (Virtual)	Opened	Global	Disabled	Standard

Port	Port status	Opened by	Current settings	Sent / Received
COM1 (Virtual)	Opened	...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe	9600,N,8,1	1.4 KB / 0 bytes
COM2 (Virtual)	Opened	C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe	9600,N,8,1	0 bytes / 1.3 KB

Edit Component ? X

Part Reference:	P1	Hidden: ☐
Part Value:	COMPIM	Hidden: ☐
Element:	New	
VSM Model:	COMPIM.DLL	Hide All v
Physical port:	COM1	Hide All v
Physical Baud Rate:	9600	Hide All v
Physical Data Bits:	8	Hide All v
Physical Parity:	NONE	Hide All v
Virtual Baud Rate:	9600	Hide All v
Virtual Data Bits:	8	Hide All v
Virtual Parity:	NONE	Hide All v
Advanced Properties:		
Physical Stop Bits v	1	Hide All v
Other Properties		
☐ Exclude from Simulation	☐ Attach hierarchy module	
☐ Exclude from PCB Layout	☐ Hide common pins	
☐ Exclude from Current Variant	☐ Edit all properties as text	

کد متلب:

همان کد قسمت Polling

```
Press the RESET button on your STM32 board now!
```

```
-----  
Message 1: Reza Nadimi  
Message 2: Reza Nadimi  
Message 3: Reza Nadimi  
Message 4: Reza Nadimi  
Message 5: Reza Nadimi  
Message 6: Reza Nadimi  
Message 7: Reza Nadimi  
Message 8: Reza Nadimi  
Message 9: Reza Nadimi  
Message 10: Reza Nadimi  
-----
```

```
Successfully received all 10 messages.
```

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

5_DMA. اسم خودتان را به تعداد ۱۰ بار، از میکروکنترلر به نرمافزار Matlab، ارسال نمایید آن را در نرمافزار Matlab، دریافت و نمایش دهید.

5.3 روش DMA

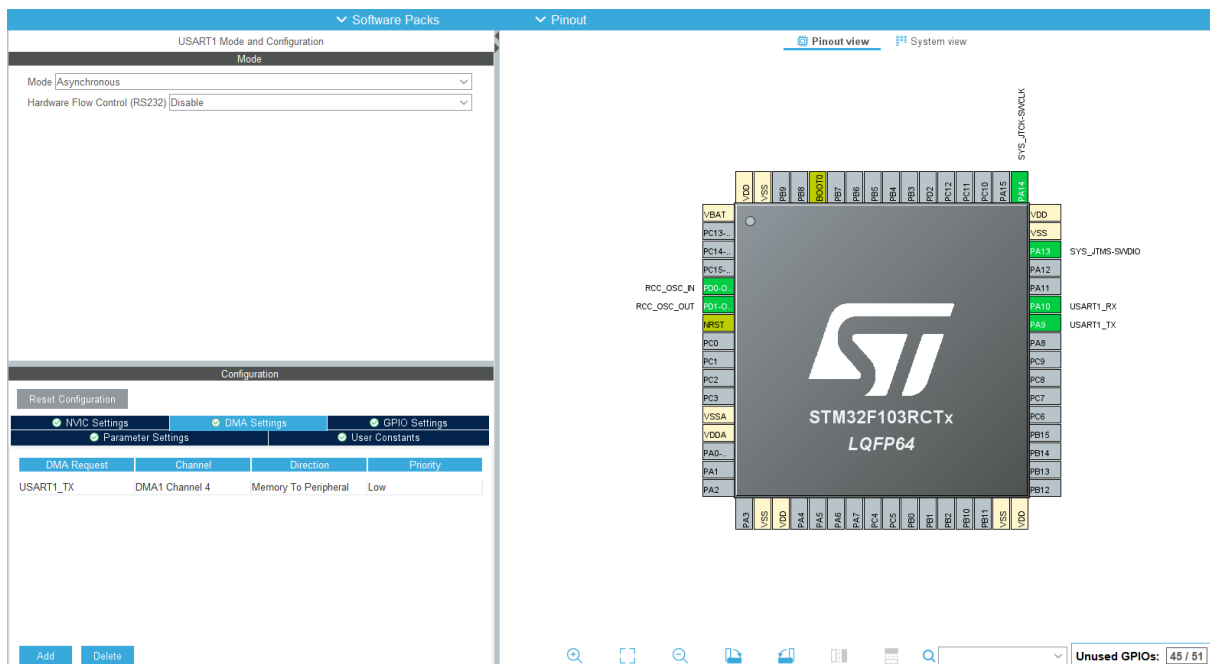
در روش DMA، داده‌ها بدون دخالت CPU و مستقیماً بین پریفرال UART و حافظه جابه‌جا می‌شوند. این روش حتی از وقفه هم کم‌بارتر است، چون CPU فقط در ابتدا و انتهای انتقال بلوک داده درگیر می‌شود، نه برای هر بایت.

(i) بخش STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم و تیک DMA را میزنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم، DMA رو در حالت نرمال قرار می دهیم.



بخش Keil (iii)

رشته myName شامل نام کاربر و کاراکتر خط جدید (\n) است که متلب با استفاده از آن پایان هر پیام را تشخیص می‌دهد. پرچم dma_tx_ready از نوع volatile و مقدار اولیه ۱ نشان می‌دهد که کنترلر DMA برای ارسال بسته بعدی آماده است.

```
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */
/* USER CODE BEGIN PV */
char myName[] = "Reza Nadimi\n";

// Flag to track DMA status: 1 = Ready, 0 = Busy transferring
volatile uint8_t dma_tx_ready = 1;
/* USER CODE END PV */
```

این تابع به صورت خودکار پس از اتمام انتقال یک بسته (کل رشته) توسط DMA فراخوانی می‌شود. بررسی می‌کند که وقفه مربوط به USART1 باشد، سپس پرچم dma_tx_ready را برابر ۱ قرار می‌دهد تا حلقه اصلی بداند DMA آزاد شده و می‌تواند بسته بعدی را ارسال کند.

```
/* USER CODE BEGIN 0 */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        dma_tx_ready = 1;
    }
}
/* USER CODE END 0 */
```

پس از تأخیر ۱ ثانیه برای تثبیت پورت USB ، حلقه ۱۰ بار اجرا می‌شود. در هر تکرار پرچم dma_tx_ready را می‌کنیم، سپس با HAL_UART_Transmit_DMA به کنترلر DMA دستور ارسال رشته را می‌دهیم و در یک حلقه while منتظر می‌مانیم تا callback پرچم را دوباره ۱ کند (یعنی ارسال کامل شود). در انتها ۲۰۰ میلی‌ثانیه تأخیر می‌دهیم تا متلب فرصت پردازش داشته باشد.

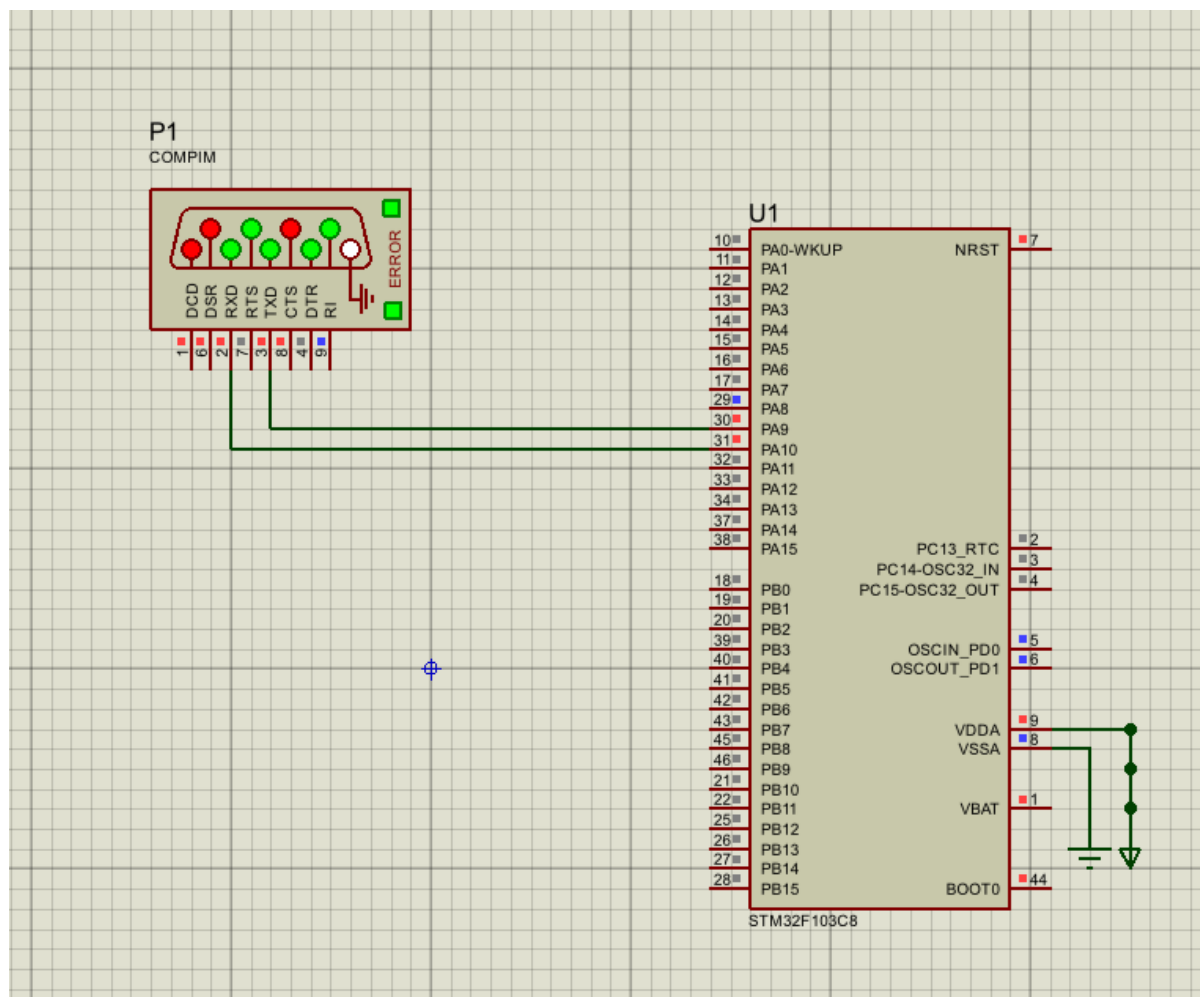
```
/* USER CODE BEGIN 2 */
HAL_Delay(1000);

for (int i = 0; i < 10; i++)
{
    dma_tx_ready = 0;

    HAL_UART_Transmit_DMA(&huart1, (uint8_t*)myName, strlen(myName));

    while (dma_tx_ready == 0)
    {
    }

    HAL_Delay(200);
}
/* USER CODE END 2 */
```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

COM1

COM2

Ports status:	Paired
User session:	Global
Baudrate emulation:	Disabled
Pinout:	Standard

 Edit  Delete

 Break

COM1 (Virtual)

Port status:	Opened
Opened by:	...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe 
Current settings:	9600,N,8,1
Sent / Received:	1.4 KB / 0 bytes

COM2 (Virtual)

Port status:	Opened
Opened by:	C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe 
Current settings:	9600,N,8,1
Sent / Received:	0 bytes / 1.3 KB

Edit Component ? X

Part Reference:	P1	Hidden: ☐
Part Value:	COMPIM	Hidden: ☐
Element:	New	
VSM Model:	COMPIM.DLL	Hide All v
Physical port:	COM1	Hide All v
Physical Baud Rate:	9600	Hide All v
Physical Data Bits:	8	Hide All v
Physical Parity:	NONE	Hide All v
Virtual Baud Rate:	9600	Hide All v
Virtual Data Bits:	8	Hide All v
Virtual Parity:	NONE	Hide All v
Advanced Properties:		
Physical Stop Bits v	1	Hide All v
Other Properties		
☐ Exclude from Simulation	☐ Attach hierarchy module	
☐ Exclude from PCB Layout	☐ Hide common pins	
☐ Exclude from Current Variant	☐ Edit all properties as text	

کد متلب:

مثل بخش Polling

```
Command Window
Press the RESET button on your STM32 board now!
-----
Message 1: Reza Nadimi
Message 2: Reza Nadimi
Message 3: Reza Nadimi
Message 4: Reza Nadimi
Message 5: Reza Nadimi
Message 6: Reza Nadimi
Message 7: Reza Nadimi
Message 8: Reza Nadimi
Message 9: Reza Nadimi
Message 10: Reza Nadimi
-----
Successfully received all 10 messages.
```

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

6_Polling. اسم خودتان را به تعداد ۱۰ بار، از نرمافزار Matlab به میکروکنترلر، ارسال نمایید آن را در میکروکنترلر، دریافت و روی LCD نمایش دهید

6.1 روش Poling

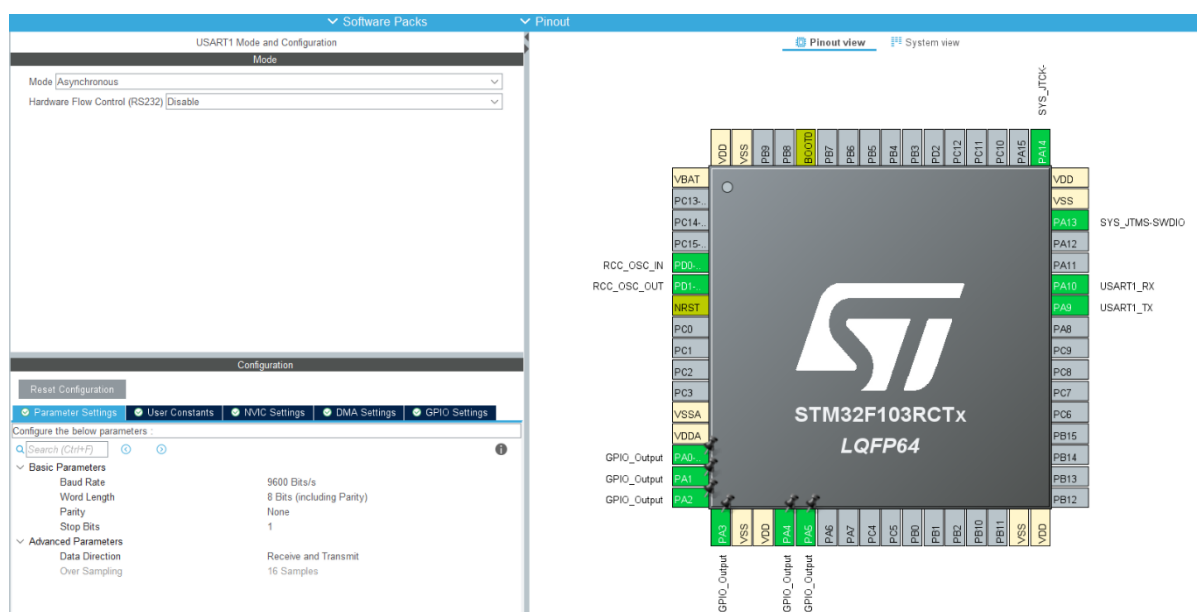
در این روش CPU به طور مداوم وضعیت رجیستر UART را بررسی می کند.

بخش (i) STM32CubeMX

میکرو کنترلر STM32F103RCT رو در نرم افزار انتخاب میکنیم.

UART1 رو فعال مے کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم و پین های 0 تا 5 پورت A را به صورت GPIO Output انتخاب میکنیم.



(ii) بخش Keil

فایل‌های هدر lcd.h برای راه‌اندازی نمایشگر، string.h برای توابع رشته‌ای مانند memset و stdio.h برای sprintf اضافه شده‌اند. تعریف پین‌های LCD نیز با دستورات #define انجام شده تا کد مستقل از سخت‌افزار باشد.

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */
```

پورت و پین‌های میکروکنترلر که به پایه‌های داده (D4 تا D7) و پایه‌های کنترلی (RS و EN) LCD متصل می‌شوند، با دستورات #define مشخص می‌گردند.

```
/* USER CODE BEGIN PTD */
#define LCD_D4_PORT GPIOA
#define LCD_D4_PIN GPIO_PIN_0
#define LCD_D5_PORT GPIOA
#define LCD_D5_PIN GPIO_PIN_1
#define LCD_D6_PORT GPIOA
#define LCD_D6_PIN GPIO_PIN_2
#define LCD_D7_PORT GPIOA
#define LCD_D7_PIN GPIO_PIN_3

#define LCD_RS_PORT GPIOA
#define LCD_RS_PIN GPIO_PIN_4
#define LCD_EN_PORT GPIOA
#define LCD_EN_PIN GPIO_PIN_5
/* USER CODE END PTD */
```

RX_BUFFER_SIZE حداکثر طول رشته دریافتی (مثلاً ۶۴ کاراکتر) را مشخص می‌کند. UART_INTERCHAR_TIMEOUT زمان انتظار بین دو کاراکتر متوالی را ۲۰۰ میلی‌ثانیه تعریف می‌کند تا بتوان پایان یک رشته را تشخیص داد.

```
/* USER CODE BEGIN PD */
#define RX_BUFFER_SIZE 64
#define UART_INTERCHAR_TIMEOUT 200 // 200ms timeout
/* USER CODE END PD */
```

آرایه rx_buffer رشته دریافتی از پورت سریال را ذخیره می‌کند lcd_display برای ساخت رشته نهایی که روی LCD نمایش داده می‌شود در نظر گرفته شده است.

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef mylcd;
char rx_buffer[RX_BUFFER_SIZE];
char lcd_display[16];
/* USER CODE END PV */
```

ابتدا LCD مقداردهی شده و در سطر اول "Waiting for" و در سطر دوم "MATLAB Data..." نمایش داده می‌شود تا کاربر بداند سیستم آماده دریافت است. سپس حلقه اصلی ۱۰ بار اجرا خواهد شد.

```
/* USER CODE BEGIN 2 */
Lcd_PortType lcd_ports[] = {LCD_D4_PORT, LCD_D5_PORT, LCD_D6_PORT,
LCD_D7_PORT};
Lcd_PinType lcd_pins[] = {LCD_D4_PIN, LCD_D5_PIN, LCD_D6_PIN, LCD_D7_PIN};

mylcd = Lcd_create(lcd_ports, lcd_pins, LCD_RS_PORT, LCD_RS_PIN, LCD_EN_PORT,
LCD_EN_PIN, LCD_4_BIT_MODE);

HAL_Delay(100);
Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "Waiting for");
Lcd_cursor(&mylcd, 1, 0);
Lcd_string(&mylcd, "MATLAB Data...");
```

ابتدا بافر پاک می‌شود. برای اولین کاراکتر بی‌نهایت منتظر می‌مانیم (HAL_MAX_DELAY) و سپس کاراکترهای بعدی را با زمان‌آوت ۲۰۰ میلی‌ثانیه می‌خوانیم. اگر کاراکتر newline (\n) برسد یا زمان‌آوت منقضی شود، حلقه پایان می‌یابد و رشته با \0 خاتمه داده می‌شود.

```
for (int count = 1; count <= 10; count++)
{
    uint16_t idx = 0;

    memset(rx_buffer, 0, RX_BUFFER_SIZE);
```

```

    if (HAL_UART_Receive(&huart1, (uint8_t*)&rx_buffer[idx], 1, HAL_MAX_DELAY)
== HAL_OK)
    {
        idx++;

        while (idx < RX_BUFFER_SIZE - 1)
        {
            if (HAL_UART_Receive(&huart1, (uint8_t*)&rx_buffer[idx], 1,
UART_INTERCHAR_TIMEOUT) == HAL_OK)
            {
                if (rx_buffer[idx] == '\n' || rx_buffer[idx] == '\r')
                {
                    break;
                }
                idx++;
            }
            else
            {
                break;
            }
        }
    }

    rx_buffer[idx] = '\0';

```

صفحه LCD پاک شده، در سطر اول شماره پیام و در سطر دوم نام دریافتی چاپ می‌شود. تأخیر ۱.۵ ثانیه‌ای باعث می‌شود کاربر فرصت خواندن نام را داشته باشد و از ارسال سریع پیام بعدی توسط متلب جلوگیری نشود.

```

if (idx > 0)
{
    Lcd_clear(&mylcd);

    sprintf(lcd_display, "Name %d/10:", count);
    Lcd_cursor(&mylcd, 0, 0);
    Lcd_string(&mylcd, lcd_display);

    Lcd_cursor(&mylcd, 1, 0);
    Lcd_string(&mylcd, rx_buffer);

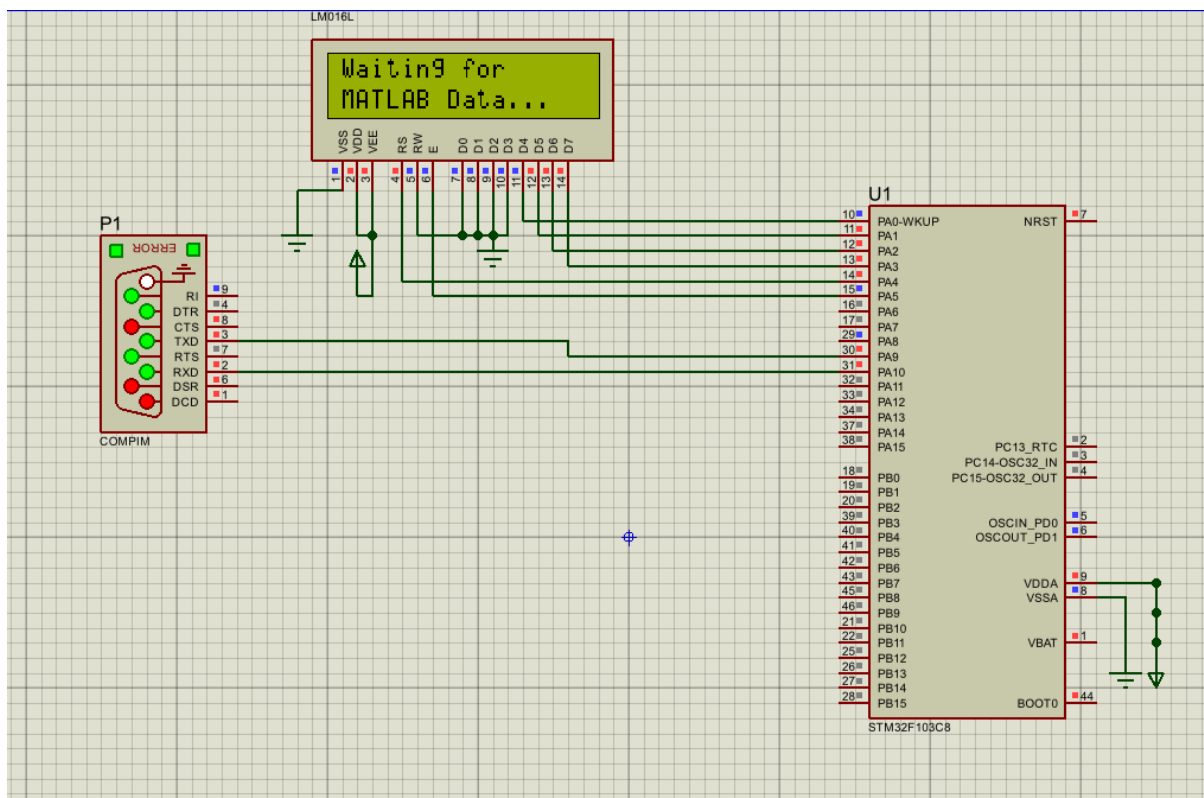
    HAL_Delay(1500);
}

```

پس از دریافت ۱۰ نام، صفحه LCD پاک شده و پیام "All 10 Names" در سطر اول و "Received! :)" در سطر دوم نمایش داده می‌شود تا پایان کار به کاربر اطلاع داده شود.

```
// After all 10 loops are finished
Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "All 10 Names");
Lcd_cursor(&mylcd, 1, 0);
Lcd_string(&mylcd, "Received! :)");
/* USER CODE END 2 */
```

(iii) بخش شبیه سازی Proteus



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهم

LM016L LCD را بر بروی برد میزاییم و مطابق شکل پیکربندی می کنیم.

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

- COM1
- COM2

Ports status: Paired

User session: Global

Baudrate emulation: Disabled

Pinout: Standard

Edit Delete Break

COM1 (Virtual)

Port status: Opened

Opened by: ...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe

Current settings: 9600,N,8,1

Sent / Received: 1.4 KB / 0 bytes

COM2 (Virtual)

Port status: Opened

Opened by: C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe

Current settings: 9600,N,8,1

Sent / Received: 0 bytes / 1.3 KB

Edit Component

Part Reference: Hidden: ☐

Part Value: Hidden: ☐

Element:

VSM Model:

Physical port:

Physical Baud Rate:

Physical Data Bits:

Physical Parity:

Virtual Baud Rate:

Virtual Data Bits:

Virtual Parity:

Advanced Properties:

Other Properties

☐ Exclude from Simulation ☐ Attach hierarchy module

☐ Exclude from PCB Layout ☐ Hide common pins

☐ Exclude from Current Variant ☐ Edit all properties as text

کد متلب:

```
clear; clc;

baud_rate = 9600;
stm32_port = serialport("COM2", baud_rate);

pause(1);
```

حافظه و صفحه فرمان پاک شده و پورت سریال روی COM2 با نرخ 9600 باز می‌شود. تأخیر ۱ ثانیه‌ای به STM32 فرصت می‌دهد تا پس از ریست، مقداردهی اولیه را کامل کند.

```

disp('Starting transmission to STM32...');
disp('-----');

my_name = "Reza Nadimi";

for count = 1:10

    text_to_send = sprintf('%s\n', my_name);

    write(stm32_port, text_to_send, "string");

    fprintf('Sent %d/10: %s\n', count, my_name);

    pause(2.5);

end

```

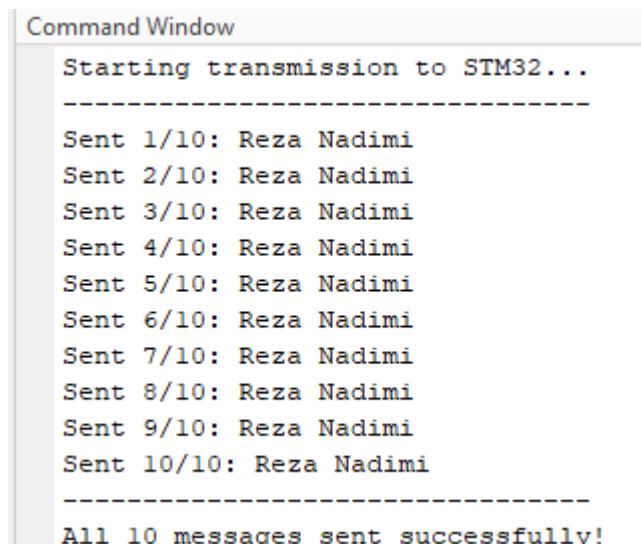
در یک حلقه ۱۰ تایی، رشته نام به همراه کاراکتر خط جدید (\n) ساخته و از طریق پورت سریال ارسال می‌شود. fprintf وضعیت ارسال را در صفحه فرمان نشان می‌دهد و pause(2.5) صبر می‌کند تا STM32 فرصت پردازش و نمایش نام روی LCD را داشته باشد (متناسب با تأخیر ۱.۵ ثانیه در میکرو).

```

disp('-----');
disp('All 10 messages sent successfully!');

```

پس از اتمام حلقه، پیام موفقیت در خروجی چاپ می‌شود. (پورت سریال با خروج از برنامه به طور خودکار بسته می‌شود).



```

Command Window

Starting transmission to STM32...
-----
Sent 1/10: Reza Nadimi
Sent 2/10: Reza Nadimi
Sent 3/10: Reza Nadimi
Sent 4/10: Reza Nadimi
Sent 5/10: Reza Nadimi
Sent 6/10: Reza Nadimi
Sent 7/10: Reza Nadimi
Sent 8/10: Reza Nadimi
Sent 9/10: Reza Nadimi
Sent 10/10: Reza Nadimi
-----
All 10 messages sent successfully!

```

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

IT_6 اسم خودتان را به تعداد ۱۰ بار، از نرمافزار Matlab به میکروکنترلر، ارسال نمایید
آن را در میکروکنترلر، دریافت و روی LCD نمایش دهید

6.2 روش IT

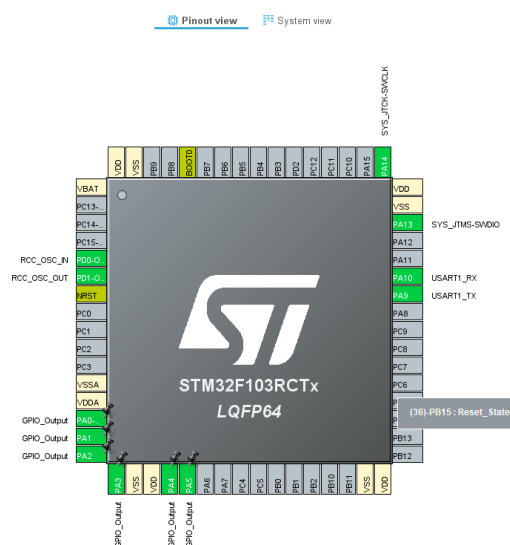
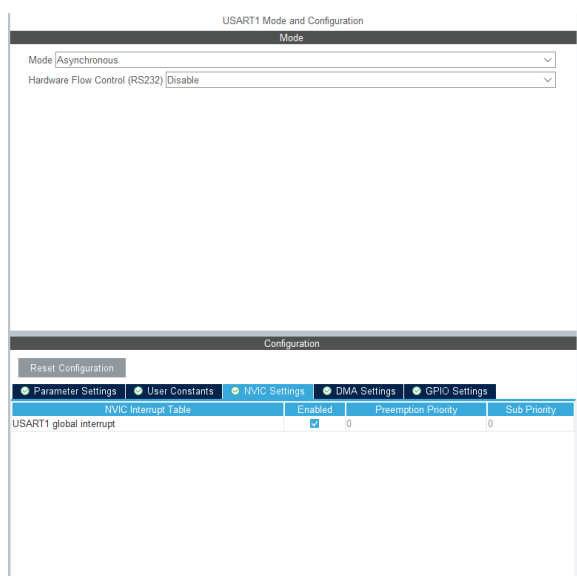
در روش اینترپیت، میکروکنترلر بی‌آنکه مدام چک کند (پولینگ)، فقط وقتی داده می‌رسد وقفه ایجاد می‌شود و CPU در بقیه زمان آزاد است و درگیر نمی‌شود.

بخش STM32CubeMX

میکرو کنترلر STM32F103RCT دو در نرم افزار انتخاب میکنیم.

UART1 رو فعال می کنیم و BaudRate رو روی 9600 Bits/s تنظیم میکنیم و تیک اینترپرت را میزنیم.

پروگرامر رو به حالت Serial Wire و کلاک رو تنظیم میکنیم و پین های 0 تا 5 پورت A را GPIO OUTPUT انتخاب میکنیم



رشته دریافتی را کاراکتر به کاراکتر ساخته و rx_byte بایت جاری را نگه می‌دارد rx_index. موقعیت بعدی در بافر را نشان می‌دهد و string_ready پرچمی است که پس از دریافت کامل یک نام (دیدن \n) توسط وقفه تنظیم می‌شود تا حلقه اصلی آن را پردازش کند.

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef mylcd;
char rx_buffer[64];
uint8_t rx_byte;
volatile uint16_t rx_index = 0;
volatile uint8_t string_ready = 0;
char lcd_display[16];
/* USER CODE END PV */
```

هر بار یک بایت می‌رسد، این تابع خودکار فراخوانی می‌شود. اگر بایت \n یا \r باشد، رشته را با \0 خاتمه داده و پرچم string_ready را ۱ می‌کند و وقفه را غیرفعال می‌گذارد. در غیر این صورت، بایت را در بافر ذخیره کرده و وقفه را برای دریافت بایت بعدی دوباره فعال می‌کند.

```
/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (rx_byte == '\n' || rx_byte == '\r')
        {
            rx_buffer[rx_index] = '\0';
            string_ready = 1;
        }
        else
        {
            if (rx_index < 63)
            {
                rx_buffer[rx_index++] = rx_byte;
            }

            HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
        }
    }
}
/* USER CODE END 0 */
```

LCD پاک شده و پیام آماده‌به‌کار نمایش داده می‌شود. هرگونه خطای باقیمانده از UART پاک شده و اولین وقفه دریافت راه‌اندازی می‌شود تا منتظر اولین کاراکتر از MATLAB بماند.

```
/* USER CODE BEGIN 2 */
Lcd_PortType lcd_ports[] = {LCD_D4_PORT, LCD_D5_PORT, LCD_D6_PORT,
LCD_D7_PORT};
Lcd_PinType lcd_pins[] = {LCD_D4_PIN, LCD_D5_PIN, LCD_D6_PIN, LCD_D7_PIN};

mylcd = Lcd_create(lcd_ports, lcd_pins, LCD_RS_PORT, LCD_RS_PIN, LCD_EN_PORT,
LCD_EN_PIN, LCD_4_BIT_MODE);

HAL_Delay(100);
Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "Waiting for");
Lcd_cursor(&mylcd, 1, 0);
Lcd_string(&mylcd, "MATLAB Data IT..");

int name_count = 1;

// 1. Clear any hardware errors that happened during startup
__HAL_UART_CLEAR_OREFLAG(&huart1);
__HAL_UART_FLUSH_DRREGISTER(&huart1);

// 2. Arm the interrupt to listen for the VERY FIRST letter of the first name
HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
```

CPU در این حلقه بی‌کار می‌ماند تا پرچم `string_ready` توسط وقفه تنظیم شود. سپس نام روی LCD نمایش داده می‌شود، ۱.۵ ثانیه تأخیر برای خواندن، شمارنده افزایش می‌یابد و بافر برای نام بعدی پاک می‌گردد. پیش از فعال‌سازی مجدد وقفه، بافر UART شستشو می‌شود تا داده‌های ناخواسته از بین بروند.

```
while (name_count <= 10)
{
    if (string_ready == 1)
    {
        string_ready = 0; // Lower the flag

        if (rx_index > 0)
        {
            Lcd_clear(&mylcd);

            sprintf(lcd_display, "Name %d/10:", name_count);
            Lcd_cursor(&mylcd, 0, 0);
            Lcd_string(&mylcd, lcd_display);

            Lcd_cursor(&mylcd, 1, 0);
            Lcd_string(&mylcd, rx_buffer);
        }
    }
}
```

```

        HAL_Delay(1500);

        name_count++;
    }

    // Reset our buffer variables for the next incoming name
    memset(rx_buffer, 0, sizeof(rx_buffer));
    rx_index = 0;

    if (name_count <= 10)
    {
        // Clear any garbage that arrived while we were sleeping in
        HAL_Delay
        __HAL_UART_CLEAR_OREFLAG(&huart1);
        __HAL_UART_FLUSH_DRREGISTER(&huart1);

        // Re-arm the interrupt to listen for the first letter of the NEXT
        name!
        HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
    }
}
}

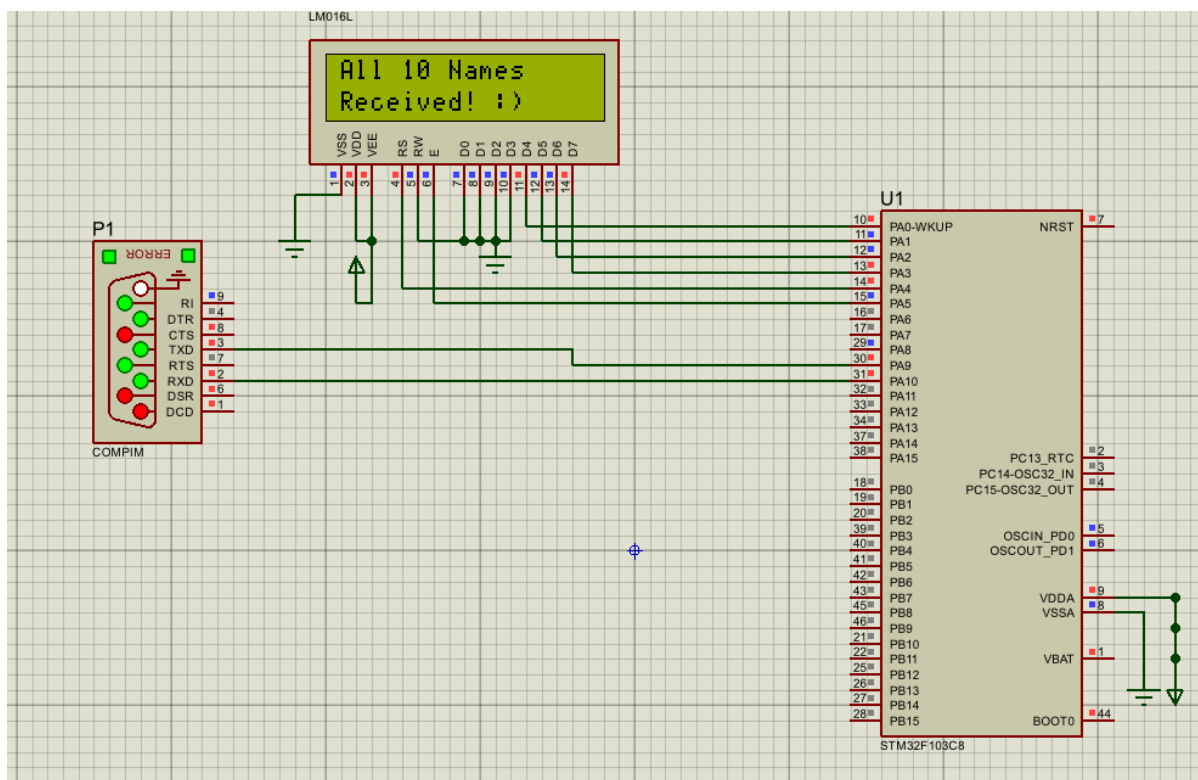
```

پس از دریافت ۱۰ نام، پیام موفقیت روی LCD نمایش داده می‌شود. میکروکنترلر وارد یک حلقه بی‌نهایت خالی می‌شود و هیچ عملیات دیگری انجام نمی‌دهد.

```

// Finished
Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "All 10 Names");
Lcd_cursor(&mylcd, 1, 0);
Lcd_string(&mylcd, "Received! :)");
/* USER CODE END 2 */

```



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

LM016L LCD را بر بروی برد میزاریم و مطابق شکل پیکربندی می کنیم.

COMPIP را در محیط Proteus قرار می دهیم وپین های Rx وTx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

COM1
COM2

 Edit  Delete

 Break

Ports status:	Paired
User session:	Global
Baudrate emulation:	Disabled
Pinout:	Standard

COM1 (Virtual)

Port status:	Opened
Opened by:	...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe 
Current settings:	9600,N,8,1
Sent / Received:	1.4 KB / 0 bytes

COM2 (Virtual)

Port status:	Opened
Opened by:	C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe 
Current settings:	9600,N,8,1
Sent / Received:	0 bytes / 1.3 KB

Edit Component ? X

Part Reference:	P1	Hidden: ☐
Part Value:	COMPIM	Hidden: ☐
Element:	New	
VSM Model:	COMPIM.DLL	Hide All v
Physical port:	COM1	Hide All v
Physical Baud Rate:	9600	Hide All v
Physical Data Bits:	8	Hide All v
Physical Parity:	NONE	Hide All v
Virtual Baud Rate:	9600	Hide All v
Virtual Data Bits:	8	Hide All v
Virtual Parity:	NONE	Hide All v
Advanced Properties:		
Physical Stop Bits v	1	Hide All v
Other Properties		
☐ Exclude from Simulation	☐ Attach hierarchy module	
☐ Exclude from PCB Layout	☐ Hide common pins	
☐ Exclude from Current Variant	☐ Edit all properties as text	

کد متلب:

مثل بخش Polling

```
Command Window
Starting transmission to STM32...
-----
Sent 1/10: Reza Nadimi
Sent 2/10: Reza Nadimi
Sent 3/10: Reza Nadimi
Sent 4/10: Reza Nadimi
Sent 5/10: Reza Nadimi
Sent 6/10: Reza Nadimi
Sent 7/10: Reza Nadimi
Sent 8/10: Reza Nadimi
Sent 9/10: Reza Nadimi
Sent 10/10: Reza Nadimi
-----
All 10 messages sent successfully!
```

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می باشد.

آرایه rx_buffer با طول ۲۰ بایت برای دریافت داده‌های پد شده از MATLAB در نظر گرفته شده است. پرچم dma_ready از نوع volatile نشان می‌دهد که DMA چه زمانی دقیقاً ۲۰ بایت را دریافت کرده و حلقه اصلی می‌تواند آن را پردازش کند.

```
/* USER CODE BEGIN PV */
Lcd_HandleTypeDef mylcd;
char rx_buffer[20]; // We will tell MATLAB to always send exactly 20
characters
volatile uint8_t dma_ready = 0;
char lcd_display[16];
/* USER CODE END PV */
```

این تابع به طور خودکار هر بار که DMA دقیقاً ۲۰ بایت (مقدار مشخص شده در HAL_UART_Receive_DMA) را از UART دریافت کرده و در بافر ذخیره کند، فراخوانی می‌شود. در آن بررسی می‌شود که وقفه مربوط به USART1 باشد و سپس پرچم dma_ready را برابر ۱ قرار می‌دهد تا حلقه اصلی از آماده بودن داده مطلع شود.

```
/* USER CODE BEGIN 0 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        dma_ready = 1; // Raise the flag!
    }
}
/* USER CODE END 0 */
```

LCD راه‌اندازی شده و پیام آماده‌به‌کار در حالت DMA نمایش داده می‌شود. هرگونه خطای باقیمانده از UART و رجیستر داده نیز پاک می‌گردد تا از دریافت داده‌های ناخواسته جلوگیری شود.

```
/* USER CODE BEGIN 2 */
Lcd_PortType lcd_ports[] = {LCD_D4_PORT, LCD_D5_PORT, LCD_D6_PORT,
LCD_D7_PORT};
Lcd_PinType lcd_pins[] = {LCD_D4_PIN, LCD_D5_PIN, LCD_D6_PIN, LCD_D7_PIN};

mylcd = Lcd_create(lcd_ports, lcd_pins, LCD_RS_PORT, LCD_RS_PIN, LCD_EN_PORT,
LCD_EN_PIN, LCD_4_BIT_MODE);
```

```

HAL_Delay(100);
Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "Waiting for");
Lcd_cursor(&mylcd, 1, 0);
Lcd_string(&mylcd, "MATLAB (DMA)...");

// Clear any hardware errors before we start
__HAL_UART_CLEAR_OREFLAG(&huart1);
__HAL_UART_FLUSH_DRREGISTER(&huart1);

```

در هر تکرار، پرچم صفر شده، بافر پاک می‌شود و DMA راه‌اندازی می‌گردد تا دقیقاً ۲۰ بایت دریافت کند CPU. در یک حلقه while منتظر می‌ماند تا callback پرچم را ۱ کند؛ سپس نام روی LCD نمایش داده شده و ۱.۵ ثانیه تأخیر برای خواندن وجود دارد. در نهایت بافر UART دوباره شستشو می‌شود تا برای نام بعدی آماده گردد.

```

for (int name_count = 1; name_count <= 10; name_count++)
{
    dma_ready = 0;
    memset(rx_buffer, 0, sizeof(rx_buffer));

    HAL_UART_Receive_DMA(&huart1, (uint8_t*)rx_buffer, 20);

    while (dma_ready == 0)
    {
        // Waiting...
    }

    // DMA is done! Display the name.
    Lcd_clear(&mylcd);

    sprintf(lcd_display, "Name %d/10:", name_count);
    Lcd_cursor(&mylcd, 0, 0);
    Lcd_string(&mylcd, lcd_display);

    Lcd_cursor(&mylcd, 1, 0);
    Lcd_string(&mylcd, rx_buffer);

    // Wait so you can read it on the screen
    HAL_Delay(1500);

    // Flush the register again just in case MATLAB sent extra garbage data
    __HAL_UART_CLEAR_OREFLAG(&huart1);
    __HAL_UART_FLUSH_DRREGISTER(&huart1);
}

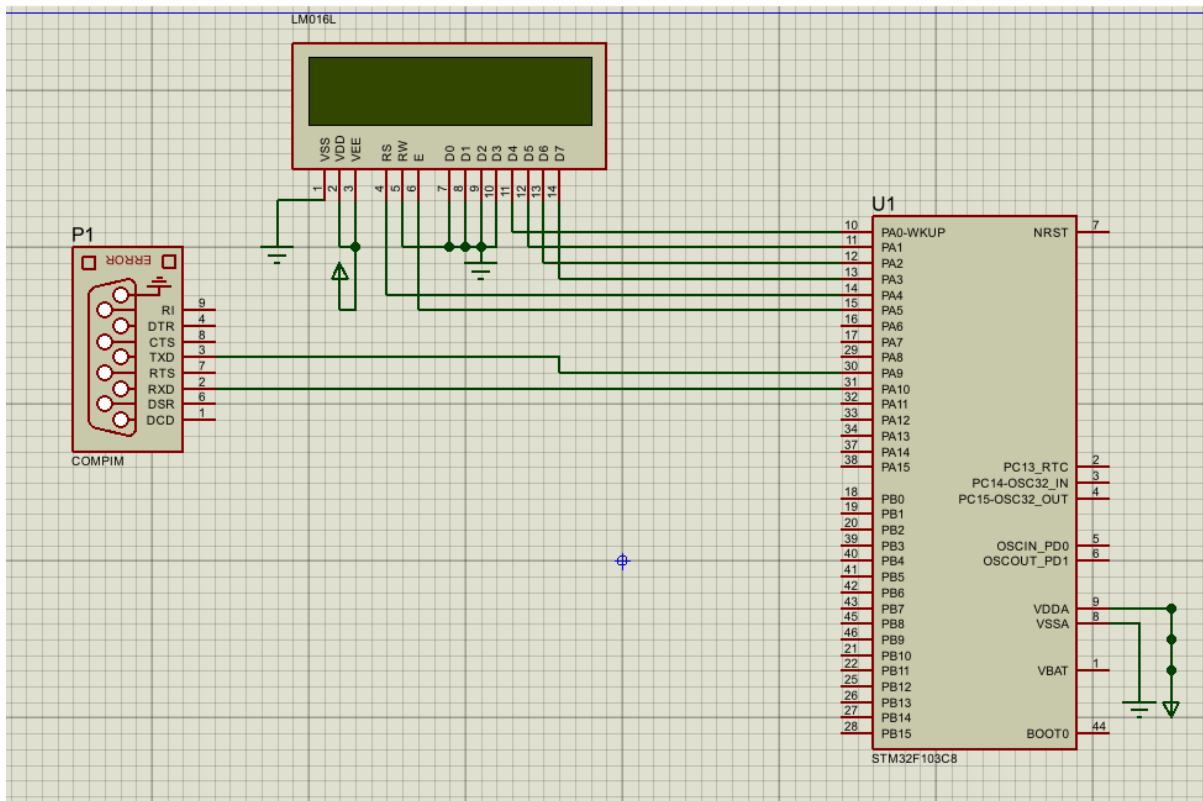
```

در هر تکرار، پرچم صفر شده، بافر پاک می‌شود و DMA راه‌اندازی می‌گردد تا دقیقاً ۲۰ بایت دریافت کند CPU. در یک حلقه while منتظر می‌ماند تا callback پرچم را ۱ کند؛ سپس نام روی LCD نمایش داده شده و ۱.۵ ثانیه تأخیر برای خواندن وجود دارد. در نهایت بافر UART دوباره شستشو می‌شود تا برای نام بعدی آماده گردد.

```
// Finished
Lcd_clear(&mylcd);
Lcd_cursor(&mylcd, 0, 0);
Lcd_string(&mylcd, "All 10 Names");
Lcd_cursor(&mylcd, 1, 0);
Lcd_string(&mylcd, "DMA Success! ");
/* USER CODE END 2 */
```

پس از دریافت ۱۰ نام، پیام موفقیت روی LCD نمایش داده می‌شود و میکروکنترلر وارد یک حلقه بی‌نهایت خالی می‌گردد تا هیچ عملیات دیگری انجام ندهد.

iii) بخش شبیه سازی Proteus



میکروکنترلر STM32F103C8 (چون کنترلر اصلی ما در داخل Proteus موجود نبود) رو در محیط شبیه سازی قرار میدهیم

LM016L LCD را بر بروی برد میزاریم و مطابق شکل پیکربندی می کنیم.

COMPIM را در محیط Proteus قرار می دهیم وپین های Rx و Tx آن را به میکروکنترلر وصل می کنیم.

در اخر هم فایل Compile شده HEX. رو به کنترلر می دهیم.

4. بخش Matlab و Virtual serial port pro

یک کانیکشن pair می سازی

ALL BUNDLES

▼ Paired (1)

COM1

COM2

Ports status: Paired

User session: Global

Baudrate emulation: Disabled

Pinout: Standard

Edit Delete

Break

COM1 (Virtual)

Port status: Opened

Opened by: ...rogram Files\Labcenter Electronics\Proteus 9 Professional\Bin\PDS.exe

Current settings: 9600,N,8,1

Sent / Received: 1.4 KB / 0 bytes

COM2 (Virtual)

Port status: Opened

Opened by: C:\Program Files\MATLAB\R2024b\bin\win64\MATLAB.exe

Current settings: 9600,N,8,1

Sent / Received: 0 bytes / 1.3 KB

Edit Component

?
X

Part Reference:

P1

Hidden: ☐

Part Value:

COMPIM

Hidden: ☐

Element:

New

VSM Model:

COMPIM.DLL

Hide All

Physical port:

COM1

Hide All

Physical Baud Rate:

9600

Hide All

Physical Data Bits:

8

Hide All

Physical Parity:

NONE

Hide All

Virtual Baud Rate:

9600

Hide All

Virtual Data Bits:

8

Hide All

Virtual Parity:

NONE

Hide All

Advanced Properties:

Physical Stop Bits

1

Hide All

Other Properties

☐ Exclude from Simulation
☐ Attach hierarchy module

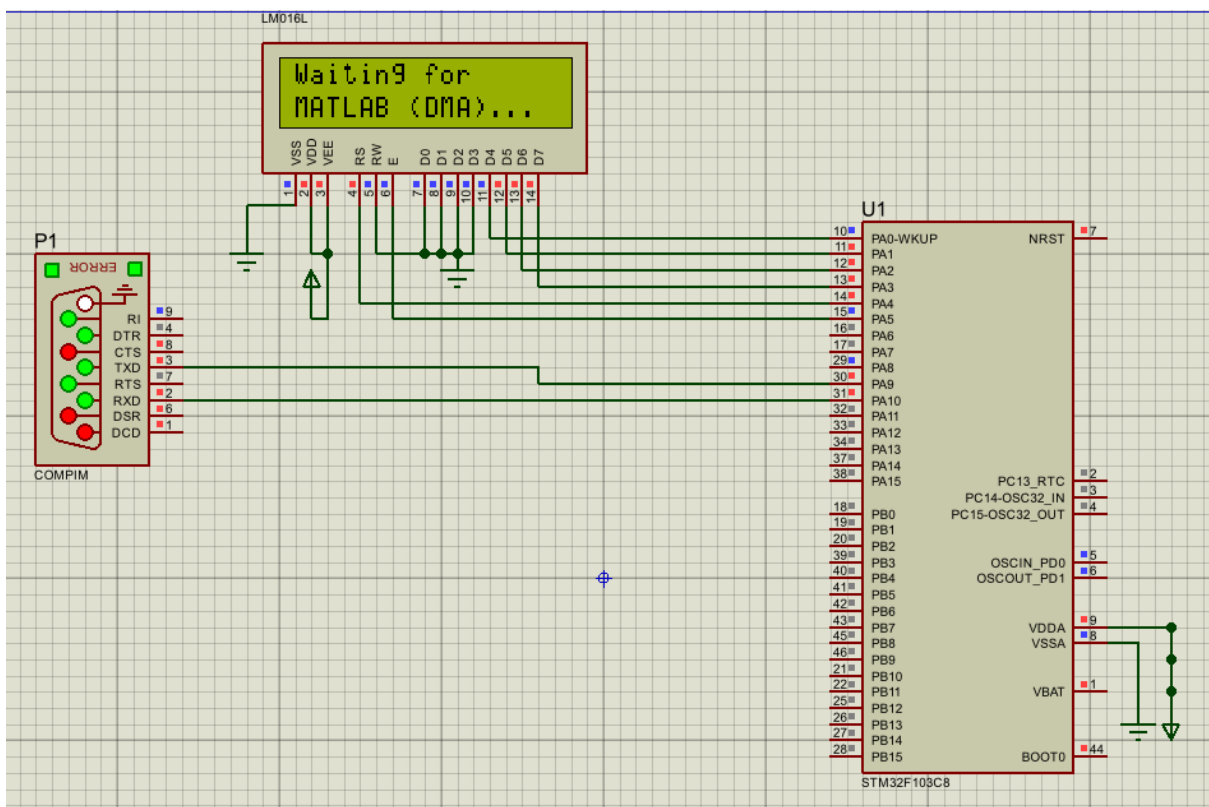
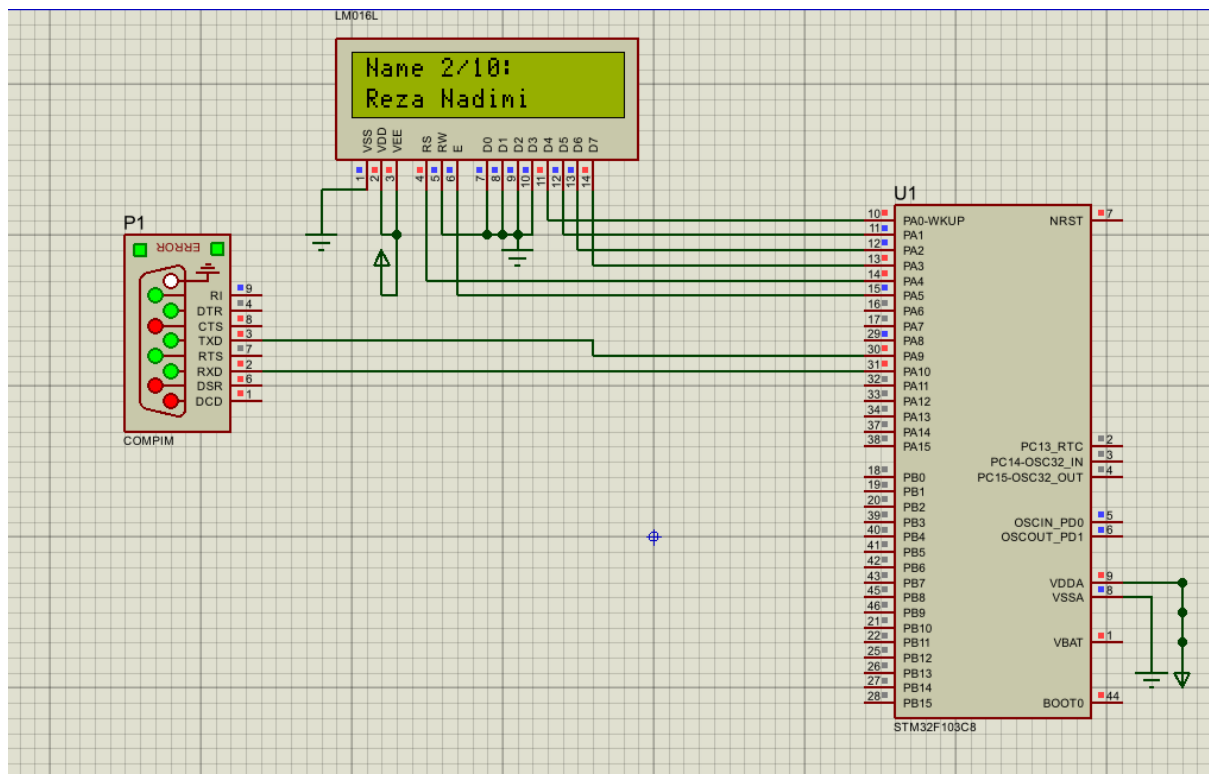
☐ Exclude from PCB Layout
☐ Hide common pins

☐ Exclude from Current Variant
☐ Edit all properties as text

OK

Cancel

Help



کد متلب:

محیط متلب پاک شده، پورت سریال با نرخ 9600 روی COM2 باز می‌شود و یک ثانیه تأخیر برای تثبیت اتصال در نظر گرفته می‌شود.

```
clear; clc;

baud_rate = 9600;
stm32_port = serialport("COM2", baud_rate);

pause(1);

disp('Starting transmission to STM32 (DMA Padded)...');
disp('-----');

my_name = "Reza Nadimi";

for count = 1:10

    % '%-19s' means: Treat my_name as a string, left-justify it,
    % and pad it with blank spaces until it is exactly 19 characters long.
    % The '\n' adds the 20th character!
    text_to_send = sprintf('%-19s\n', my_name);

    write(stm32_port, text_to_send, "string");

    fprintf('Sent %d/10: [%s]\n', count, strtrim(text_to_send));

    pause(2.5);

end

disp('-----');
disp('All 10 messages sent successfully!');
```

قالب '%-19s\n' باعث می‌شود رشته نام در ۱۹ کاراکتر چپ‌چین شده و باقیمانده با فضا پر شود؛ سپس \n بیستمین کاراکتر را می‌سازد. ارسال انجام شده و ۲.۵ ثانیه صبر می‌کند تا STM32 فرصت نمایش نام روی LCD را داشته باشد (با تأخیر ۱.۵ ثانیه‌ای در میکرو هماهنگ شده است). پس از اتمام حلقه، پیغام موفقیت در خروجی چاپ می‌شود. (پورت سریال با اتمام برنامه به طور خودکار بسته می‌شود).

ویدیوی شبیه سازی در داخل پوشه تمرین موجود می‌باشد.